

The Complete Full Stack Website Development



Dibuat Oleh:
Akbar Anggit Pambudi
Bandung
2021

Daftar Isi

1. HTML

- Dasar 1
- Dasar 2
- Link html (a href)
- 1. Tabel html dasar
- 2. Tabel (Cellspacing = untuk memberi spasi antar kolom)
- Form html

2. CSS

- Inline CSS
- Internal CSS
- External CSS
- Class dan id selector (id = #namaid{}) dan (class = .namaclass{})
- Menambahkan Icon pada title
- Div (untuk mengelompokan)
- Padding, margin, border
- Display (inline, inline-block, block, none)
- Position (Static, relative, absolute, fixed)
- Penggunaan google font
- Font sizing (mengatur jarak spasi antar kata dibawahnya)
- Float dan clear
- Text-decoration: none; (menghilangkan garis bawah)
- a:hover{}

3. Bootstrap

- Instalasi bootstrap
- Praktek Bootstrap

4. Web Desain

- Teori Warna
- Typografi (Jenis Font)
- User Interface
- User Experience

5. JAVASCRIPT ES6

- Javascript VS Java
- Alert Dan Prompt
- Tipe Data
- Variabel
- Length (namaVariabel.length)
- Slice(x,y) (namaVariabel.slice(2,3))
- toUpperCase() (namaVariabel.toUpperCase())
- toLowerCase() (namaVariabel.toLowerCase())
- Perhitungan Dasar (penambahan, pengurangan, pembagian, perkalian, modulo (mencari sisa dari hasil pembagian (%)))
- Increment (kenaikan / tambah 1) Dan Decrement (Pengurangan 1)
- Function (create dan call) Dan console.log
- Parameter Function (Math.floor() / Pembulatan kebawah)
- Parameter Function (Math.round() / Pembulatan keterdekatan)
- Parameter Function (Math.pow() / Perhitungan Pangkat)
- Parameter Function (Math.random / Gacha Angka / Mengacak Angka)
- **Kasus (jika batas umur 90 tahun berapa sisa umur anda sekarang baik hari bulan tahun. Cat: 1 tahun = 360 hari, 1 tahun = 12 bulan)**
- Return (mengembalikan nilai dalam function)
- **Kasus (Menghitung Berat Badan Ideal (BMI))**
- Statement IF Else (Jika Benar dan Jika Salah)
- Komperator (===, !==, >, <, >=, <=, &&, ||, !)
- Array (length array, push (menambahkan) , pop (menghapus) , includes)
- **Kasus FizzBuzz (jika habis dibagi 3 = "Fizz", Jika habis dibagi 5 ="Buzz")**
- **Kasus Acak Nama**
- Loop (menjalankan pada batas tertentu) (while (memeriksa keadaan yang selalu benar jika salah berhenti)) (for (menjalankan terus walau salah))
- Switch (case, default)

6. DOM (Document Object Model)

- Penggunaan javascript (sebaris, internal External)
- Pengenalan DOM
- Selector (1. Get Elements Dan 2. Query Selector (Rekomendasi))
- Dom Style
- Dom Manipulasi (Add Class, Remove Class)
- Dom Manipulasi (innerHTML dan textContent)
- Dom Attribute (Attributes, Get Attribute, Set Attribute)
- **Kasus Dadu Acak (ketika direfresh dadu akan acak)**
- AddEventListener (Memberikan respon ketika target diklik)
- **Kasus Kalkulator Sederhana (function, retrun)**
Javascript (Tambahan)
- Play audio javascript
- Contructor Function (This)
- Contructor Function 2 (function didalam function)
- Callback Function
- Set Time Out (animasi web)

7. JQUERY (Perpustakaan untuk JavaScript seperti halnya bootstrap untuk Css)

- Perbedaan Dasar JQuery Dan JavaScript
- Instalasi CDN Script JQuery Pada Html
- Selector JQuery
- Manipulasi (Style, tambah kelas (addClass), cek nama kelas (hasClass))
- Manipulasi text (text vs textContent Dan html vs innerHtml)
- Attribute (attr)
- Event / addEventListener
- Menambahkan element (p, h1, button, dst) (before(), after(), prepend(), append())
- Animasi JQuery (slideup, toogle, fadeout, fadein, dst)

8. Command Terminal (Hyper)

- Instalasi
- Konfigurasi Hyper
- Baris Perintah (mkdir, mkdir .Secrets, ls -a, cd, rm, touch, dst...)

9. NODE.JS

- Instalasi Node.Js
- Menjalankan file js dengan node di terminal
- Node REPL (Read, Evaluation, Print, Loop) (menggunakan perintah node untuk menjalankan hyper seperti console yang ada di chrome)
- Menerapkan modul atau paket (require()) dan menyalin file jadi 2
- Instalasi NPM (Node Package Manager) Pada Folder
- Penggunaan Code NPM orang (Contoh Npm = Superheroes)

10. EXPRESS.JS (Express library untuk Node seperti Bootstrap)

- Instalasi Express.js
- Instalasi Nodemon (agar ketika jalankan file js di node restartnya otomatis)
- Route (localhost:3000/nama, localhost:3000/alamat)
- Post, Get Dan Body-Parser (Dalam kasus kalkulator)
- **Latihan Membuat Kalkulator BMI**

11. API (Application Programming Interface)

- Contoh menerapkan Api cuaca
- Penggunaan API Dengan Postman
- Praktek API didalam node.js html

12. GIT & GITHUB

- Comand terminal git
- Membuat Repository GitHub
- Remote Git Dan GitHub
- Membuat file .gitignore (untuk menyembunyikan file yang diinginkan)
- Branching dan merging
- Branching Dan Merging

13. EJS (Embedded JavaScript Templating)

- Latihan jika hari ini libur dan kerja
- Install EJS Package Dan Penggunaan Ejs (npm install ejs)
- Latihan membuat to do list
- Perbedaan var, let, const
- Memotong file Ejs menjadi beberapa bagian <%- include(namaFileEjs) -%>
- Lodash (low case / upper case di link)
- Potong text (substring(1,3))
- **Tantangan Membuat Blog (File2 > 12. Ejs > 5. Tantangan membuat blog)**

14. DATABASE SQL (Structured Query Language)

- Pengenalan Database (SQL VS NON SQL)
- Membuat Database, Table, Primary key (CREATE)
- Menambahkan data kedalam table yang telah dibuat (INSERT INTO)
- Menampilkan Database (SELECT)
- Menampilkan Database dengan Kondisi (Select dan Where)
- Menyisipkan data tertentu pada table (Update)
- Menambahkan nama kolom pada table yang telah dibuat (ALTER TABLE)
- Menghapus data pada table (DELETE)
- Relationship foreign key (hubungan antar tabel) dan inner join

15. DATABASE MONGODB (NON SQL)

- Instalasi mongoDB di windows (<https://medium.com/@LondonAppBrewery/how-to-download-install-mongodb-on-windows-4ee4b3493514>)
- CRUD MongoDB
- Relationship mongoDB
- MongoDB with Node js (versi manual) (masih masalah)

16. PENGGUNAAN MONGODB DENGAN MOONGOSE DALAM NODE JS

- Instalasi mongoose dan tamplate
- Read mongoose di node js (find)
- Validasi Skema
- Update (Merubah Data Tertentu)
- Delete (Menghapus Data Tertentu)
- Relationship
- Kalo ga bisa connect pake =
mongoose.connect("mongodb://127.0.0.1:27017/namaDatabaseBaru",
{useNewUrlParser:true});
- Kasus Penerapan Database MongoDB, Express Route Parameter, Lodash

17. Penggunaan MongoDB Dengan MongoDB Atlas

- Create Cluster MongoDB Atlas
- Mengatur akun cluster dan menghubungkan Mongo db Atlas dengan hyper
- Membuat database dan kolom
- Cara menghubungkan MongoDB atlas dengan node.js

18. REST (Representational State Transfer)

- Pengenalan (rest = salahs atu gaya standar membangun API)
- Penggunaan ROBO 3T (menggunakan aplikasi untuk MongoDB)

19. Autentication & Security

- Menerapkan Tamplate node js
- Tingkat 1 (Login dengan email dan password)
- Tingkat 2 (Encipsi database) (Teknik Caesar) (dotenv = untuk menyimpan api key / hal rahasia)
- Tingkat 3 (Hash)
- Tingkat 4 (Hash + salting dengan bcrypt) password lebih susah ditebak
- Tingkat 5 Penerapan Passport (Cookies & Session) & Hash + salting (masalah ga support)
- Tingkat 6 authen google (masalah ga support)

20. REACT JS

- Pengenalan React
- Implementasi React (Dasar)
- **Latihan Membuat Note Sederhana**
- Props (memanggil nama attribute fungsi)
- **Latihan Props**
- Map data (Pemanggilan data array di app.jsx Mirip forEach)
- Perbedaan Map, Filter, Find, FindIndex, Reduce
- Arrow Function (=>) (Mempersingkat Kodingan function())
- **Latihan Membuat Note Lanjutan**
- Kondisi Rendering
- Kondisi Rendering Dengan Ternary Operator (kondisi ? IF true : Else False)
- Kondisi Rendering Dengan Ternary Operator dan And Operator (&&)
- State (Deklaratif VS Impresif)
- Hooks (useState)
- Latihan hooks useState (toLocaleTimeString)
- Destructuring Javascript ES6
- Event Handling React (onClick, onMouseOver, onMouseOut)
- Form React (Effect Button)
- State complex (Hooks)
- Latihan State Complex Sendiri

- Javascript ES6 Spread (`var = [2, 4, 4, ...spread]`)

REFERENSI

ACCOUNT		
Nama	Email	Password
Course.net	Akbaranggitpambudi96@gmail.com	Inda@123
Udemy	Akbaranggitpambudi96@gmail.com	Inda@123
GitHub (user: Akbaranggit)	Akbaranggitpambudi96@gmail.com	Inda@123
email	belajarcepat@gmail.com	Embem@123
Postman (user: Anggit96)	belajarcepat@gmail.com	Embem@123
Mongo Atlas	Akbaranggitpambudi96@gmail.com	Embem@123
Robo 3T	Akbaranggitpambudi96@gmail.com	Embem@123

NO	Keterangan	Referensi
1	https://drive.google.com/file/d/1jT9gMFb5dvdeWZPgHTPLkYyrtsM7BavE/view?usp=sharing	Silabus referensi pelatihan
2	https://codepen.io/pen/	Koding online html, css, js
HTML		
3	W3schools html	https://www.w3schools.com/html/default.asp
4	MDN html	https://developer.mozilla.org/en-US/docs/Web/HTML
5	Devdocs HTML	https://devdocs.io/
6	Emoji title	https://emojipedia.org/
7	Crop image circle and more	https://imageonline.co/
8	Mdn type input form	https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input
9	MDN (dd, dl, dt)	https://developer.mozilla.org/en-US/docs/Web/HTML/Element/dl
CSS		
9	Colorhunt = memilih warna yang cocok	https://colorhunt.co/
10	Pesticide for Chrome = membantu melihat kotak	https://chrome.google.com/webstore/detail/pesticide-for-chrome/bakpbgckdnepkmkeaiomhmfncnejndkbi
11	Mdn css reference	https://developer.mozilla.org/en-US/docs/Web/CSS/Reference
12	Favicon cc = untuk membuat lambing pada title	https://www.favicon.cc/
13	Google font	https://fonts.google.com/
14	Kumpulan ikon	https://www.flaticon.com/
15	Button generator	https://css3buttongenerator.com/

BOOTSTRAP		
16	Tamplate penggunaan bootstrap	https://getbootstrap.com/docs/5.3/getting-started/introduction/
17	Desain web / App (balsamiq)	https://balsamiq.cloud/
18	CDN bootstrap	https://getbootstrap.com/docs/5.3/getting-started/introduction/
WEB DESAIN		
18	Kombinasi warna (adobe color)	https://color.adobe.com/create/color-wheel
19	Kombinasi warna (color hunt co)	https://colorhunt.co/
JAVASCRIPT ES6		
20	MDN (Math.pow(), Math.floor())	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math
DOM (DOCUMENT OBJECT MODEL)		
21	Pohon html (document, html, head, body)	https://chrome.google.com/webstore/detail/html-tree-generator/dlbbmhhaadfnbbdnjalilhakfmiffeg
22	DOM style	https://www.w3schools.com/jsref/dom_obj_style.asp
23	Menambahkan event Ketika target diklik	https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener
24	Typen event	https://developer.mozilla.org/en-US/docs/Web/Events
25	Play sound javascript	https://stackoverflow.com/questions/9419263/how-to-play-audio
26	Type event keyboard	https://www.w3schools.com/jsref/event_onkeydown.asp
27	Set Time Out (Animasi web)	https://www.w3schools.com/js/js_timing.asp
JQUERY		
28	CDN Script JQuery	https://developers.google.com/speed/libraries#jquery
29	MINIFIER (JS DAN CSS) agar file lebih kecil	https://www.minifier.org/
30	Animasi JQuery	https://api.jquery.com/category/effects/
Hyper (Command Terminal)		
31	Download Hyper	https://hyper.is/#installation
32	Download Git Bash (Bourne Again Shell)	https://git-scm.com/downloads
33	Tamplate Konfigurasi Hyper	https://gist.github.com/cocoonapky/404220405435b3d0373e37ec43e54a23
34	Type baris perintah	https://www.learnenough.com/command-line-tutorial

NODE.JS		
35	Download Node.JS	https://nodejs.org/en/
36	Dokumentasi Node.JS	https://nodejs.org/api/
37	NPM	https://www.npmjs.com/
EXPRESS.JS		
38	Dokumentasi Express	https://expressjs.com/
39	Code status browser (404. 200,)	https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
API (Application Programming Interface)		
40	Contoh Api cuaca	https://openweathermap.org/current
41	Postman	https://www.postman.com/downloads/
43	Json formatted viewer	https://chrome.google.com/webstore/detail/json-viewer-pro/eifflpmocdbdmejbjaopkhhbfmdgjicc
GIT & GITHUB		
44	Tampalate gitignore	https://github.com/github/gitignore/blob/main/Swift.gitignore
EJS (Embedded JavaScript Templating)		
45	Documentasi EJS	https://ejs.co/#install
46	Setup EJS	https://github.com/mde/ejs/wiki/Using-EJS-with-Express
47	Lodash (_.lowerCase(req.params.title))	https://lodash.com/
Database (SQL)		
48	Document SQL syntax	https://www.w3schools.com/sql/
49	Untuk demo menggunakan SQL	https://sqliteonline.com/#fiddle-5bbdbae7288bo2ajn2wly03
50	Data Type	https://www.w3schools.com/sql/sql_datatypes.asp
51	Insert into (memasukan data ke table yang dibuat)	https://www.w3schools.com/sql/sql_insert.asp
MONGODB		
52	Cara install mongoDB	https://medium.com/@LondonAppBrewery/how-to-download-install-mongodb-on-windows-4ee4b3493514
53	Download MongoDB	https://www.mongodb.com/try/download/community
54	Doc CRUD	https://www.mongodb.com/docs/manual/crud/
55	Doc MongoDB	https://www.w3schools.com/mongodb/mongodb_mongosh_create_database.php
56	Operator (\$gte, \$gt)	https://www.mongodb.com/docs/manual/reference/operator/query/
57	Mongodb driver (node js, ruby, php)	https://www.mongodb.com/docs/drivers/

Mongoose (Mongodb)		
58	Dokumentasi Mongoose	https://mongoosejs.com/
59	Update, delete, find	https://mongoosejs.com/docs/api/model.html
MongoDB With mongoDB Atlas		
60	MongoDB Atlas	https://www.mongodb.com/try
Restfull API		
61	Robot 3T (Penggunaan aplikasi untuk Mongodb)	https://robomongo.org/
Autentication & Security		
62	Criyptii	https://cryptii.com/
63	Untuk mngecek berapa bnyak orang yang gunakan password tersebut	http://password-checker.online-domain-tools.com/
64	Npm bcrypt	https://www.npmjs.com/package/bcrypt
React js		
65	Code sand box (coding online)	http://codesandbox.io/
66	Document react js	https://reactjs.org/docs/create-a-new-react-app.html
67	Symbol copyright	https://fsymbols.com/copyright/
68	React devTools	https://chrome.google.com/webstore/detail/react-developer-tools/fmkadmapgofadopljbjfkapdkoienihi?hl=en
69	Map, Filter, Find FindIndex, reduce	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/filter
70	Hooks (useState)	https://reactjs.org/docs/hooks-reference.html#usestate
71	Event Html	https://www.w3schools.com/tags/ref_event_attributes.asp

HTML (HyperText Markup Language)

- **Dasar 1 HTML**

Template html dapat diketik dengan “ ! ” (tanda seru) kemudian enter

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dasar html</title>
  <style>
    hr{
      height: 3px; /* height = menagatur ketinggian garis / ktbalan*/
      background-color: grey;
    }
  </style>
</head>
<body>
  <center> <!--center untuk mengatur bagian didalamnya dengan format ditengah-->
  <hr noshade="10px"> <!-- noshade = untuk memberikan bayangan pada garis (hr) -->
  <h1>Belajar Awal Html</h1>
  <h3>By</h3>
  <h2>Akbar Anggit Pambudi</h2>
  <hr noshade="3px">
  </center>
</body>
</html>
```

Hasil



- Dasar 2 html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dasar 2 HTML</title>
</head>
<body>
   <!-- img src untuk menambahkan gambar dan alt untuk mengisi jika gambar tdk ada -->
  <h1> Akbar Anggit Pambudi</h1>
  <p> <em> saya adalah <strong> developper js </strong> </em> </p>
  <br>
  <hr>
  <h2> Skill</h2>
  <ul> <!-- membuat list tanpa urutan dan ol = untuk membuat list berurut -->
    <li>javascript</li> <!-- li = isi dari list -->
    <li>react js</li>
    <li> front end</li>
  </ul>
  <h2> Hobi</h2>
  <ul>
    <li>belajar</li>
    <li>olahraga</li>
  </ul>
</body>
</html>
```

Hasil



- **Link HTML (a href)**

File html 1

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta http-equiv="X-UA-Compatible" content="IE=edge">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>link</title>
8 </head>
9 <body>
10   <h1>makanan favorit</h1>
11   <h3> <a href="ambil dari sini/sini.html"> klik disini </a> </h3>
12   <!-- a href = link artikel atau link html lainnya -->
13 </body>
14 </html>
```

Hasil

makanan favorit

[klik disini](#)

Klik disini ngelink ke file html 2

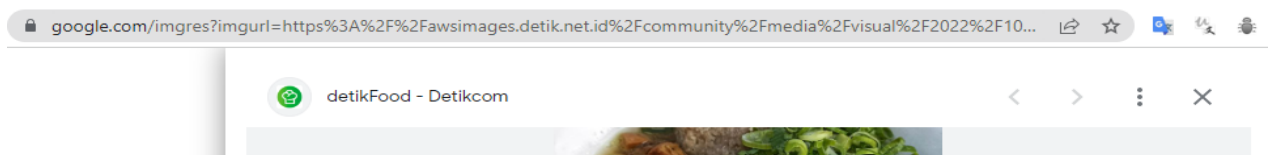
File html 2

```
<HTML>  
link > ambil dari sini > sini.html > html > body > ul  
  
1    <!DOCTYPE html>  
2    <html lang="en">  
3      <head>  
4        <meta charset="UTF-8">  
5        <meta http-equiv="X-UA-Compatible" content="IE=edge">  
6        <meta name="viewport" content="width=device-width, initial-scale=1.0">  
7        <title>linknya</title>  
8      </head>  
9      <body>  
10       <ul>  
11         <li> <a href="https://www.google.com/imgres?imgurl=https%3A%2F%2Fawsimages.detik.net.id%  
12           <!-- a href = mengambil gambar online -->  
13         </li>  
14       </ul>  
15     </body>  
16   </html>
```

Hasil

- Mie ayam

Mie ayam ngelink ke website



- 1. Tabel html dasar

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1>tabel</h1>
  <table border="1px"> <!-- membuat tabel --> <!--border = garis tabel-->
    <thead> <!-- bagian kepala tabel -->
      <tr> <!--tr = membuat baris -->
        <th> No </th> <!-- th = membuat judul tabel garis tebal -->
        <th> Nama </th>
      </tr>
    </thead>
    <tbody> <!-- membuat isi tabel -->
      <tr>
        <td> 1 </td> <!-- membuat isi tabel -->
        <td> akbar </td>
      </tr>
      <tr>
        <td> 2 </td> <!-- membuat isi tabel -->
        <td> anggit </td>
      </tr>
    </tbody>
    <tfoot> <!-- bagian bawah tabel -->
      <th> ini bagian bawah</th>
    </tfoot>
  </table>
</body>
</html>
```

Hasil

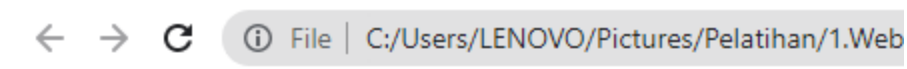
Tabel html

No	Nama
1	akbar
2	anggit
ini bagian bawah	

- 2. Tabel (Cellspacing = mengatur spasi antar kolom)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Tabel Cellspacing</title>
</head>
<body>
  <h2>Contoh penggunaan tabel dan cel spacing</h2>
  <table cellspacing="10" border="1px">
    <!-- cellspacing = untuk mengatur antar kolom -->
    <tr>
      <td>  </td>
      <td> <p>saya adalah web devlopper </p> </td>
    </tr>
  </table>
</body>
</html>
```

Hasil



Contoh penggunaan tabel dan cel spacing

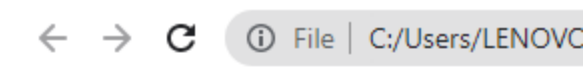


- **Form HTML**

Tipe – tipe input = <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input>

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form</title>
</head>
<body>
  <h1>Membuat Form</h1>
  <br>
  <form action="mailto:akbaranggitpambudi"> <!-- action = yang dilakukan form
ini setelah submit -->
    <label for=""> Nama</label>
    <input type="text">
    <br>
    <label for=""> Tanggal lahir</label>
    <input type="date" name="" id="">
    <br>
    <label for=""> data benar</label>
    <input type="checkbox" name="" id="">
    <br>
    <input type="submit" value="submit">
  </form>
</body>
</html>
```

Hasil



Membuat Form

Nama

Tanggal lahir

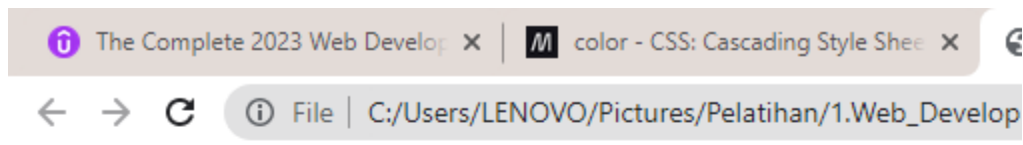
data benar ☒

CSS (Cascading Style Sheet)

- Inline CSS

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>inline css</title>
</head>
<body>
  <h1 style="background-color: aqua; font-size: 100px;"> Tes CSS</h1>
  <!-- inline = mengatur css pada satu baris html dengan tambah style -->
</body>
</html>
```

Hasil



Tes CSS

- **Internal CSS**

➔ Mengatur desain pada file html tetapi tidak pada baris koding

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Internal CSS</title>

  <style> /* style ini untuk mndesain terpisah dari baris koding */

  h1{
    color: gold;
  }
  body{
    background-color: antiquewhite;
  }

  </style>

</head>
<body>
  <h1> Akbar Anggit Pambudi </h1>
  <hr>
  <h3> Seorang developer </h3>
</body>
</html>
```

Hasil



- **Eksternal CSS**

Mengatur desain html diluar file html dengan membuat file css

File html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title> file html </title>

  <link rel="stylesheet" href="style.css">
  <!-- link ini untuk memanggil file css yang dibuat diluar file html -->

</head>
<body>
  <h1> Akbar Anggit Pambudi </h1>
  <hr>
  <h3> Seorang developers </h3>
</body>
</html>
```

File css

```
h1{
  font-size: 100px;
}
body{
  background-color: aquamarine;
}
```

Hasil



- **Class dan id selector css**

Jika mengatur css menggunakan class maka gunakan (titik didepannya), contoh : .sapi{}

Jika mengatur css menggunakan id maka gunakan (tanda pagar didepannya), contoh :

#sapi{}

File html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>class selector</title>

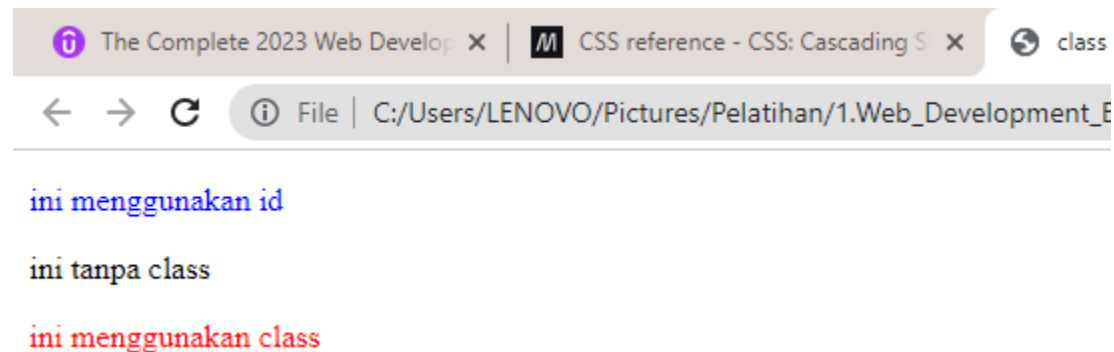
  <link rel="stylesheet" href="style.css"> <!-- link external css -->

</head>
<body>
  <!-- class berfungsi membedakan dari yang lain misal ada 3 paragraf -->
  <p id="id1"> ini menggunakan id </p>
  <p> ini tanpa class </p>
  <p class="kelas2"> ini menggunakan class </p>
</body>
</html>
```

File css external

```
/* untuk mengatur css kelas gunakan (titik didepannya) */
/* untuk mengatur id css gunakan (tanda pagar didepannya) */
#id1{
  color: blue;
}
.kelas2{
  color: red;
}
```

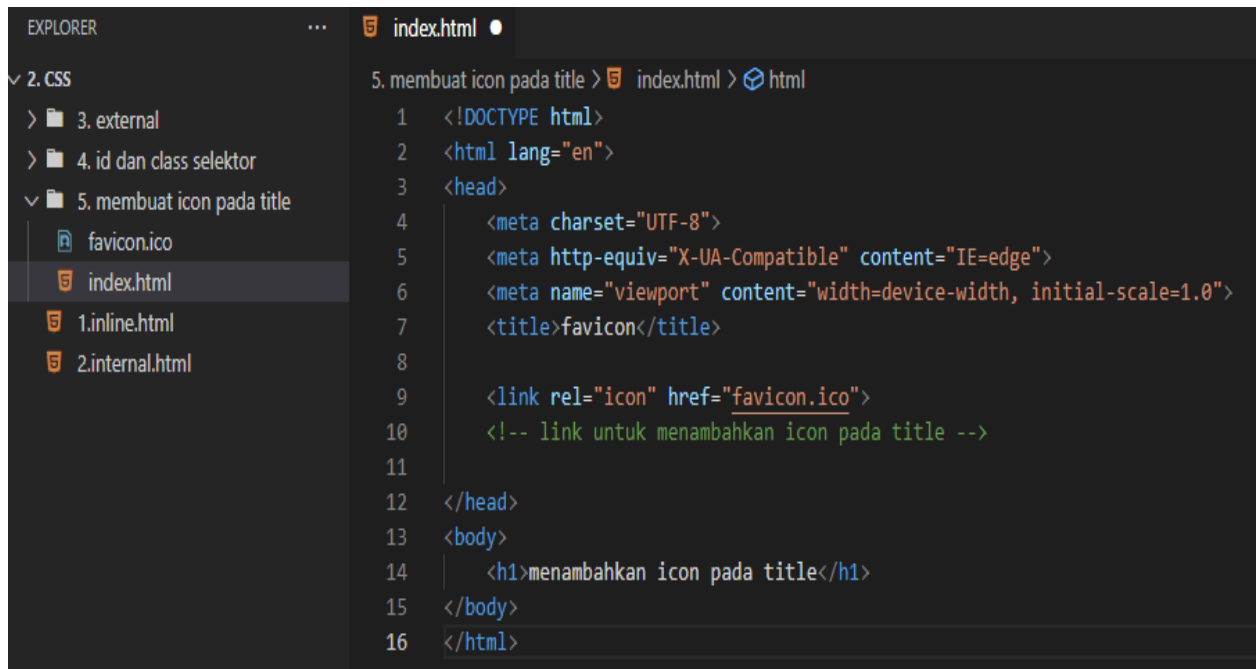
Hasil



- **Menambahkan icon pada title**

Buat icon di favicon cc = <https://www.favicon.cc/>

File html



```
5. membuat icon pada title > index.html > html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta http-equiv="X-UA-Compatible" content="IE=edge">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>favicon</title>
8
9      <link rel="icon" href="favicon.ico">
10     <!-- link untuk menambahkan icon pada title -->
11
12 </head>
13 <body>
14     <h1>menambahkan icon pada title</h1>
15 </body>
16 </html>
```

Hasil



menambahkan icon pada title

- **Div**

Div = digunakan untuk mengelompokkan agar mudah didesain

File html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DIV</title>

</head>
<body>
  <div> <!-- div untuk mengelompokkan agar lebih l=mudah mendesain -->
    <h1> Akbar Anggit Pambudi</h1>
    <p> Seorang programmer </p>
  </div>
</body>
</html>
```

Hasil

← → ↻ File | C:/Users/LENOVO/Pictures/Pelatihan/1.Web_Development_Bootcamp/file%202/2.%20css/6.%20div/inddex.html

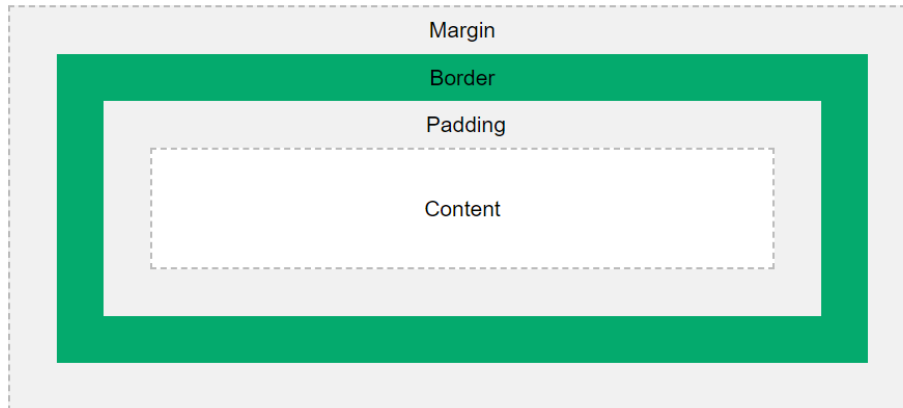
Akbar Anggit Pambudi
Seorang programmer

- **Padding, margin, border**

Padding = jarak antara konten dengan garis border

Border = garis tepi pada konten

Margin = jarak antara border dengan halaman luar



File html

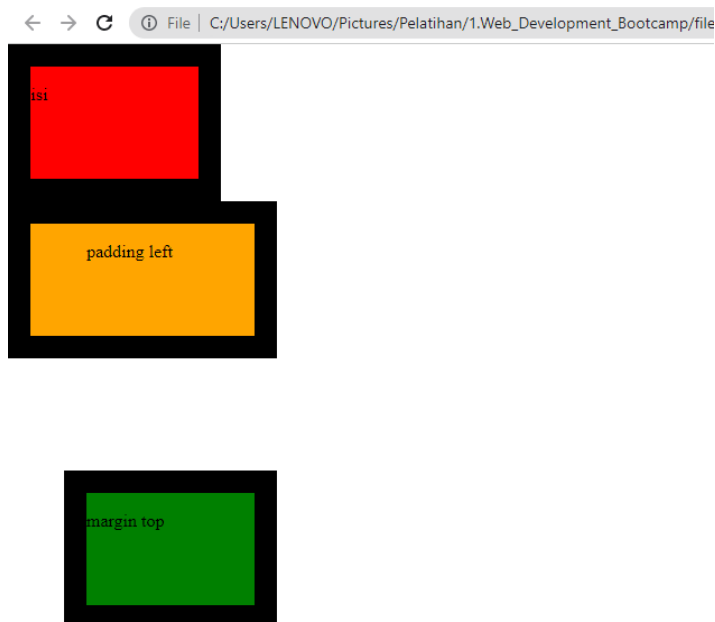
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>document</title>

  <link rel="stylesheet" href="style.css">
  <!-- css internal -->
</head>
<body>
  <div class="pertama">
    <p>isi</p>
  </div>
  <div class="kedua">
    <p>padding left </p>
  </div>
  <div class="ketiga">
    <p>margin top </p>
  </div>
</body>
</html>
```

File css eksternal

```
body{
  margin: 0;
}
.pertama{
  background-color: red;
  width: 150px;
  height: 100px;
}
.kedua{
  background-color: orange;
  width:150px;
  height:100px;
  padding-left: 50px;
}
.ketiga{
  background-color: green;
  width:150px;
  height:100px;
  margin-top: 100px;
  margin-left: 50px;
}
div{
  border: 20px solid;
}
```

Hasil



- **Display**

Block = mengambil semua sisi kanan dan kiri jadi tidak ada elemen yg disebelah

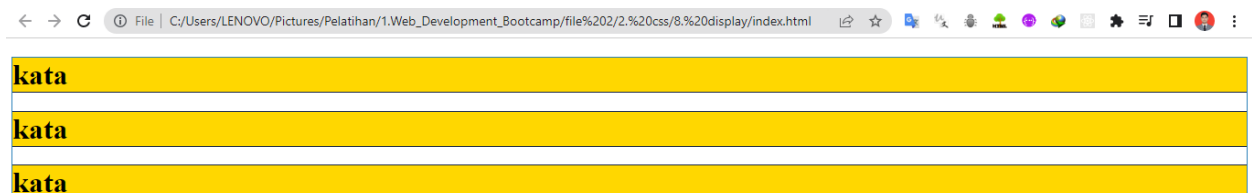
Contoh: <p>, <h1-6>, <div>, list

File html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Display</title>

  <style>
    h1{
      background-color: gold;
    }
  </style>
  <!-- css internal -->
</head>
<body>
  <h1>kata</h1>
  <h1>kata</h1>
  <h1>kata</h1>
</body>
</html>
```

Hasil



Inline = sebaris / tidak mengambil semua lebarnya

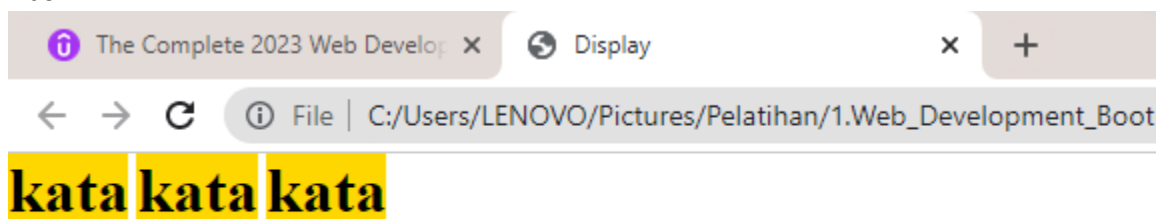
Contoh: , , <a>

File Html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Display</title>

  <style>
    h1{
      background-color: gold;
      display: inline;
      width: 100px;
    }
    body{
      margin: 0;
    }
  </style>
  <!-- css internal -->
</head>
<body>
  <h1>kata</h1>
  <h1>kata</h1>
  <h1>kata</h1>
</body>
</html>
```

Hasil



Inline-Block

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Display</title>

  <style>
    h1{
      background-color: gold;
      display: inline-block;
      width: 100px;
    }
    body{
      margin: 0;
    }
  </style>
  <!-- css internal -->
</head>
<body>
  <h1>kata</h1>
  <h1>kata</h1>
  <h1>kata</h1>
</body>
</html>
```

Hasil



kata **kata** **kata**

None = menghilangkan elemen / menganggap elemen tidak ada

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Display</title>

  <style>
    h1{
      background-color: gold;
      width: 100px;
    }
    body{
      margin: 0;
    }
    .kata{
      display: none;
    }
  </style>
  <!-- css internal -->
</head>
<body>
  <h1>kata</h1>
  <h1 class="kata">kata</h1>
  <h1>kata</h1>
</body>
</html>
```

Hasil



kata

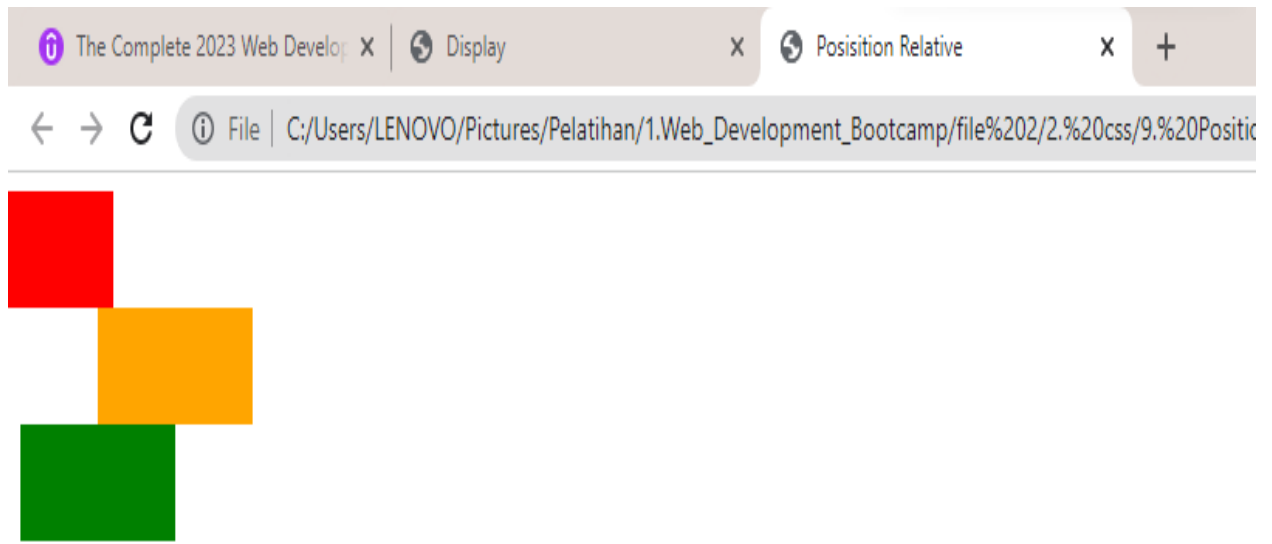
kata

- **Position**
Relative

File Html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Position Relative</title>
  <style>
    .satu{
      background-color: red;
      width: 100px;
      height: 50px;
      position: relative;
      right: 40px;
    }
    .dua{
      background-color: orange;
      width: 100px;
      height: 50px;
      position: relative;
      left: 50px;
    }
    .tiga{
      background-color: green;
      width: 100px;
      height: 50px;
      position: relative;
    }
  </style>
</head>
<body>
  <div class="satu">
  </div>
  <div class="dua">
  </div>
  <div class="tiga">
  </div>
</body>
</html>
```

Hasil

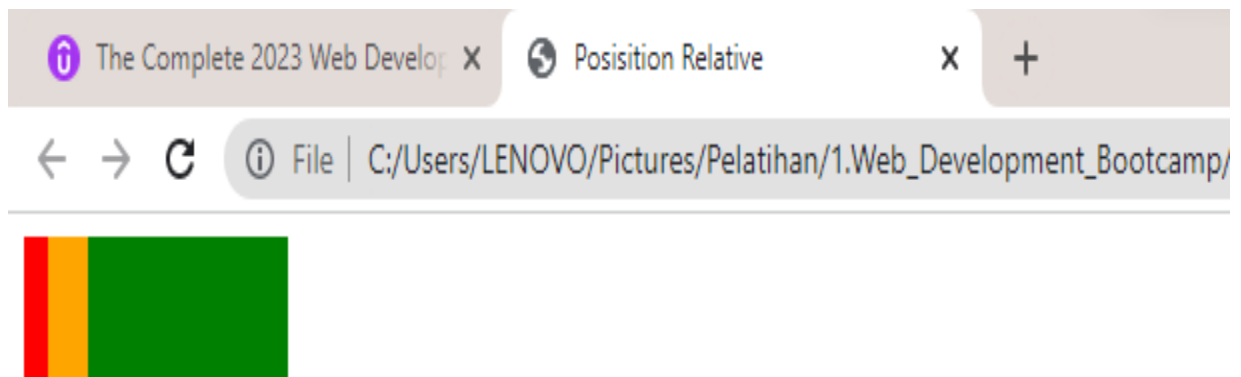


Absolute

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Position Relative</title>
  <style>
    .satu{
      background-color: red;
      width: 100px;
      height: 50px;
      position: absolute;
    }
    .dua{
      background-color: orange;
      width: 100px;
      height: 50px;
      position: absolute;
      left: 20px;
    }
    .tiga{
      background-color: green;
      width: 100px;
      height: 50px;
    }
  </style>
</head>
<body>
  <div class="satu"></div>
  <div class="dua"></div>
  <div class="tiga"></div>
</body>
</html>
```

```
        position: absolute;
        left: 40px;
    }
</style>
</head>
<body>
    <div class="satu">
    </div>
    <div class="dua">
    </div>
    <div class="tiga">
    </div>
</body>
</html>
```

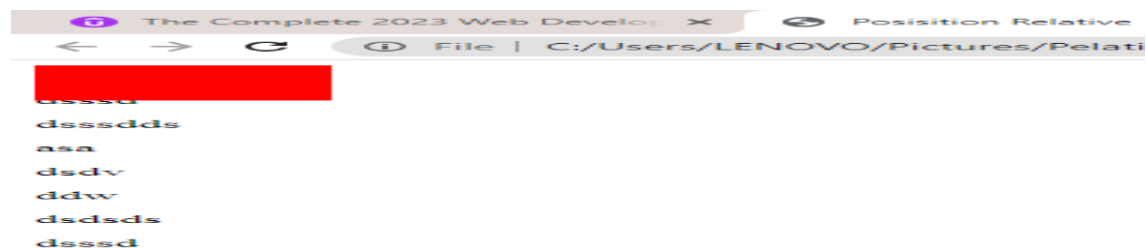
Hasil



Fixed = jadi kalo dscroll gambar yang diatas tetap ikut File html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Position Relative</title>
  <style>
    .satu{
      background-color: red;
      width: 100px;
      height: 50px;
      position: fixed;
    }
    .dua{
      background-color: orange;
      width: 100px;
      height: 50px;
      position: relative;
      left: 100px;
    }
  </style>
</head>
<body>
  <div class="satu">
  </div>
  <div class="dua">
  </div>
  <p>dsv</p><p>ddw</p><p>dssds</p><p>dsssd</p><p>dssdds</p><p>asa</p>
  <p>dsv</p><p>ddw</p><p>dssds</p><p>dsssd</p><p>dssdds</p><p>asa</p>
  <p>dsv</p><p>ddw</p><p>dssds</p><p>dsssd</p><p>dssdds</p><p>asa</p>
  <p>dsv</p><p>ddw</p><p>dssds</p><p>dsssd</p><p>dssdds</p><p>asa</p>
  <p>dsv</p><p>ddw</p><p>dssds</p><p>dsssd</p><p>dssdds</p><p>asa</p>
</body>
</html>
```

Hasil



- **Penggunaan google font**

Pilih font dan copi link

Styles

Type here to preview text

48px

Regular 400

Whereas recognition of the inherent dignity

Remove Regular 400

Add more styles Remove all

☒ <link> ☐ @import

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Zeyada&display=swap" rel="stylesheet">
```

File html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>penggunaan font google</title>

  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Zeyada&display=swap"
rel="stylesheet">
  <!-- link google font -->

  <style>
    h1{
      font-family: 'Zeyada', cursive;
    }
  </style>

</head>
<body>
  <h1>Ini pake google font</h1>
  <h3>Ini tidak pake google font</h3>
</body>
</html>
```

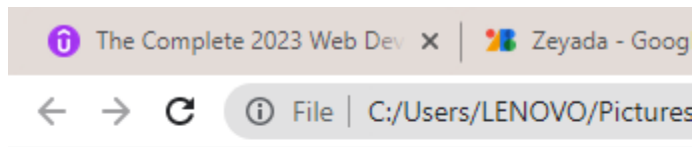
Font family liat dari

```
swap" rel="stylesheet">
```

CSS rules to specify families

```
font-family: 'Zeyada', cursive;
```

Hasil



Ini pake google font

Ini tidak pake google font

- **Font Sizing dan mengatur antar kata dibawahnya**

16px = 100% = 1em

Penggunaan em lebih dinamis

Line-height: 2; = line-height digunakan untuk mengatur jarak antar kata dibawahnya

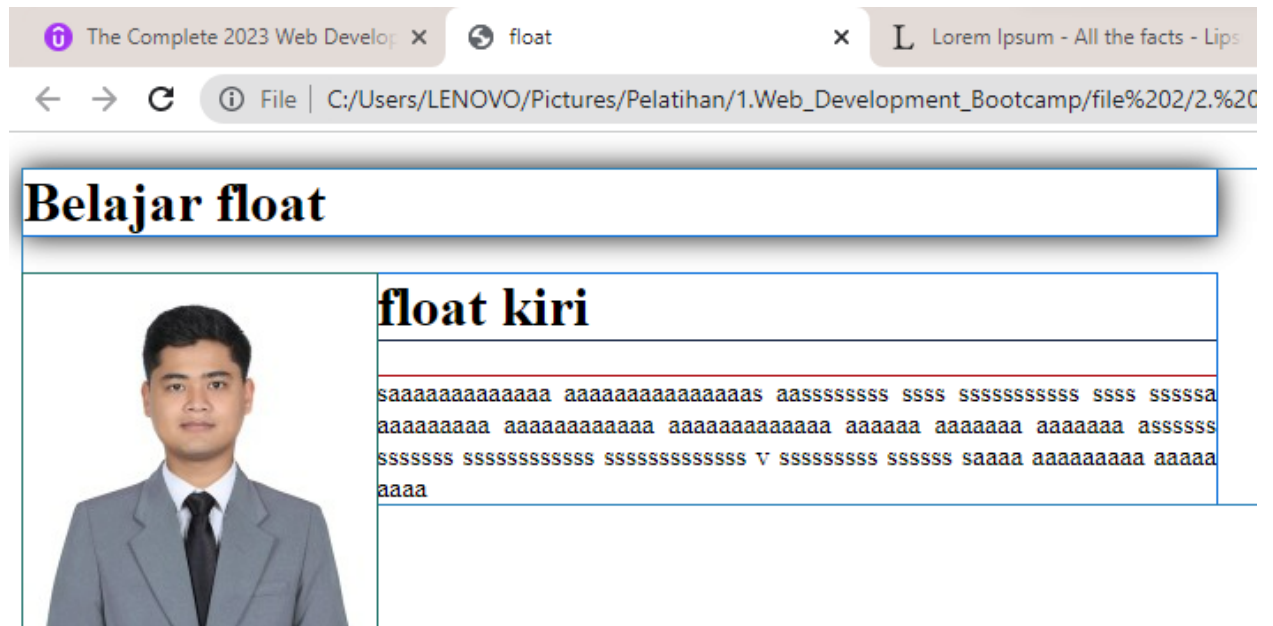
- **Float dan Clear**

Float = right dan left = agar tulisan diletakan pada kanan atau kiri gambar

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>float</title>

  <style>
    div{
      width: 50%;
    }
    .foto{
      float: left;
      /* float agar tulisan diletakan di sebelah gambar dalam 1 div */
      position: relative;
    }
    p{
      text-align: justify;
    }
  </style>
  <!-- css internal -->
</head>
<body>
  <div>
    <h1>Belajar float</h1>
  </div>
  <div>
    
    <h1>float kiri</h1>
    <p>aaaaaaaaaaaaaaaaa aaaaaaaaaaaaaaas aasssssssss ssss ssssssssssss ssss
      sssssa aaaaaaaaaa aaaaaaaaaaaaaa aaaaaaaaaaaaaa aaaaaa aaaaaaaa aaaaaaa
      assssss ssssssss ssssssssssssss ssssssssssssss V ssssssssss ssssss
      saaaa aaaaaaaaaa aaaaa aaaa
    </p>
  </div>
</body>
</html>
```

Hasil



Clear = left right = agar tulisan diletakan dibawah atau diatas gambar meski satu grup

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>float</title>

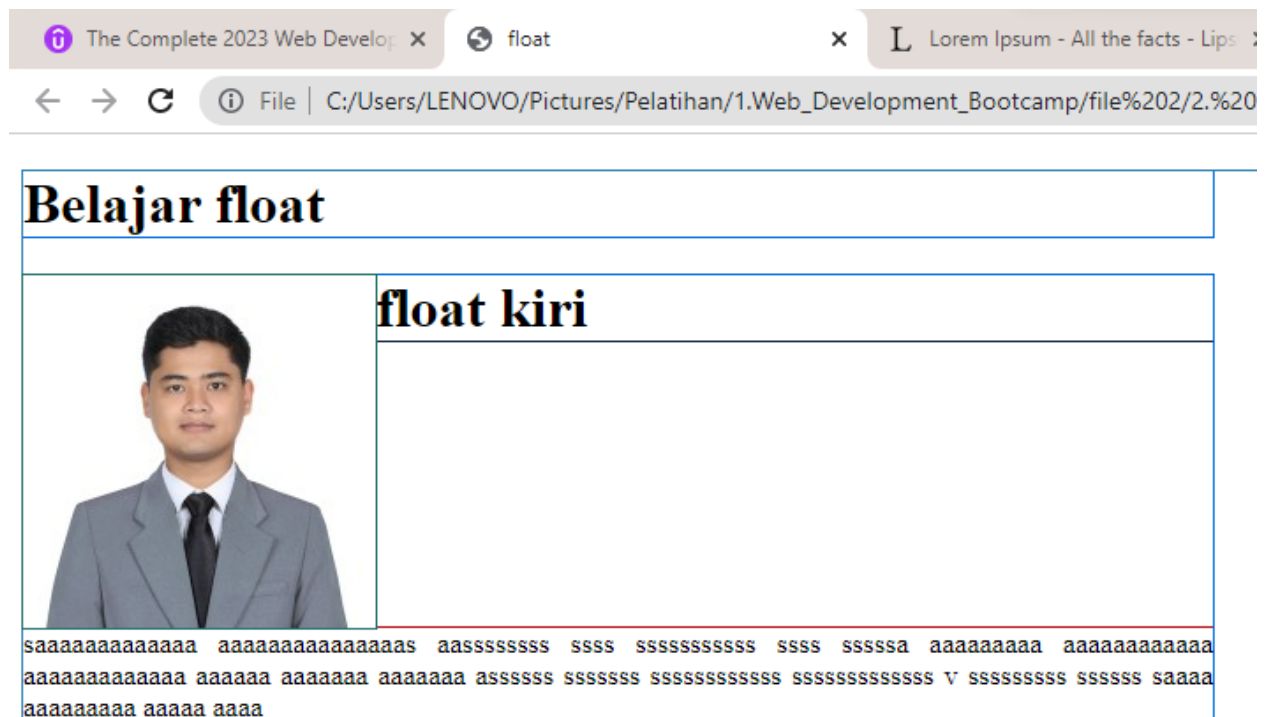
  <style>
    div{
      width: 50%;
    }
    .foto{
      float: left;
      /* float agar tulisan diletakan di sebelah gambar dalam 1 div */
      position: relative;
    }
    p{
      text-align: justify;
      clear: left; /* bagian kiri dikosongkan*/
    }
  </style>
  <!-- css internal -->
</head>
```

```

<body>
  <div>
    <h1>Belajar float</h1>
  </div>
  <div>
    
    <h1>float kiri</h1>
    <p>saiaaaaaaaaaa aaaaaaaaaaaaaaas aasssssssss ssss ssssssssssss ssss
      sssssa aaaaaaaaa aaaaaaaaaaaaaa aaaaaaaaaaaaaa aaaaaa aaaaaa aaaaaa
      asssssss sssssss ssssssssssssss ssssssssssssss v sssssssss sssssss
      saaaa aaaaaaaaa aaaaa aaaa
    </p>
  </div>
</body>
</html>

```

Hasil



BOOTSTRAP

- Instalasi bootstrap

Gunakan template bootstrap link = <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

2. **Include Bootstrap's CSS and JS.** Place the `<link>` tag in the `<head>` for our CSS, and the `<script>` tag for our JavaScript bundle (including Popper for positioning dropdowns, poppers, and tooltips) before the closing `</body>`. Learn more about our [CDN links](#).

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap demo</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css"
  </head>
  <body>
    <h1>Hello, world!</h1>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/js/bootstrap.bundle.
  </body>
</html>
```

File html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap demo</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-GLh1TQ8iRABdZLL1603oVMWSktQOp6b7In1Z13/Jr59b6EGGoI1aFkw7cmDA6j6gD" crossorigin="anonymous">
  </head>
  <body>
    <h1>Hello, world!</h1>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/js/bootstrap.bundle.min.js" integrity="sha384-w76AqPfDkMBDXo30jS1Sgez6pr3x5MlQ1ZAGC+nuZB+EYdgRZgiwxhTBTkF7CXvN" crossorigin="anonymous"></script>
  </body>
</html>
```

Web Desain

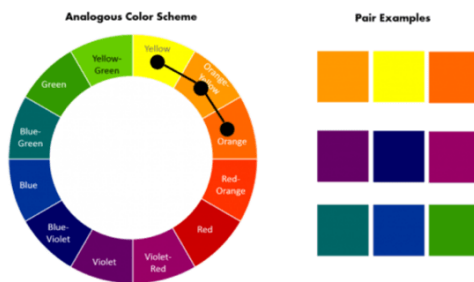
- **Teori Warna**

Ekpresi warna

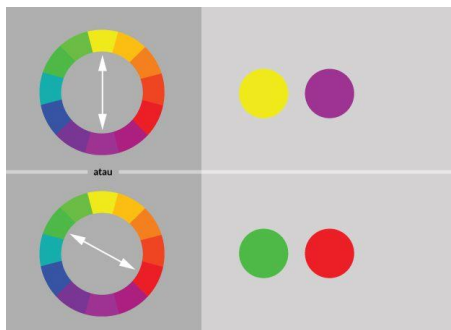
Merah	Kuning	Hijau	Biru	Ungu
Love, energic, intensity	Joy, intellect, attention	Fresh, safety, growth	Stability, trust, serenity	Royal, wealth, feminity

Teknik kombinasi warna

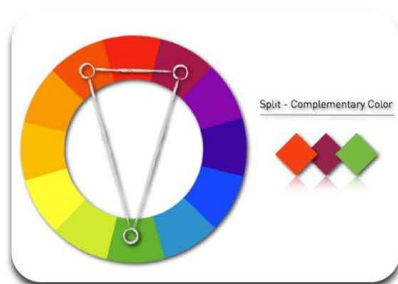
Ambil yang bersebelahan



Ambil warna yang berlawanan



Ambil warna membentuk segitiga



Referensi kombinasi warna

<https://color.adobe.com/create/color-wheel>

<https://colorhunt.co/>

- **Typografi (jenis font text)**

Pada penggunaan font text = gunakan 2 jenis font saja

Eksresi jenis font

Ekspresi font	Nama font	Jenis font
Tradisional	Minion pro	Serif
Stabil	Trajan	
Respectable	Baskerville	
Masuk akal	Helvetica	Sans - serif
Simple	Avenir	
Lurus ke depan	Din	
Personal	Freestyle script	Script
Creative	Adios script pro	
Elegant	Snell roundhand	
Friendly	Vag rounded	Display
Keras	Gin	
Menyenangkan	Thirsty rough	
Stylish	Sackers gothic	Modern
Cantik	Gotham	
Smart	Futura	

- **User interface**

Paragraf = jangan terlalu panjang / pendek, idealnya = 40 sampai 60 karakter perbaris

Align = Baiknya gunakan rata kiri

Mengurangi titik penelusuran

- **User experience**

Simple

Konsisten

Reading patterns = F – layout atau Z – layout

Support all platform (responsif)

Jangan menyulitkan / memaksa user untuk melakukan sesuatu

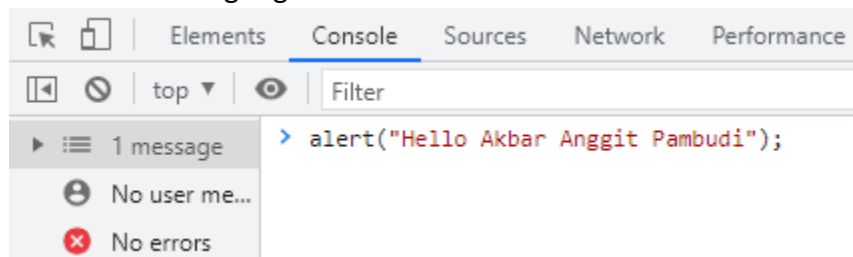
JAVASCRIPT ES6

- Javascript vs Java

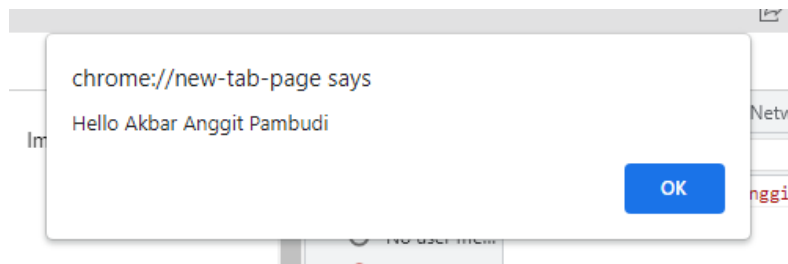
Javascript	Java
Bahasa pemograman yang ditafsirkan	Bahasa pemograman yang dikompilasi
Javascript, Python, Ruby	Java, C / C++, Swift
Fokus	
IOS	Swift
Android	Java
Website / aplikasi website	Javascript

- Alert Dan Prompt

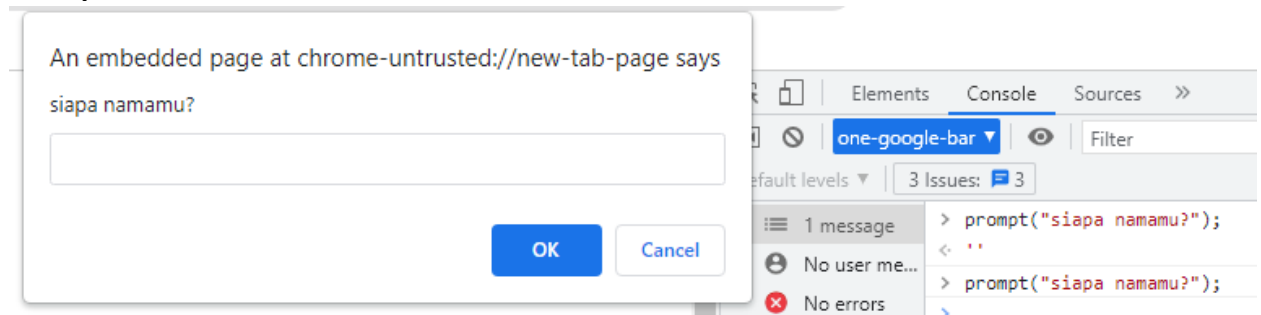
Alert Console di google



Hasil



Prompt

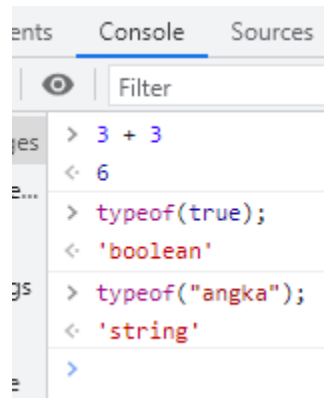


- **Tipe Data**

String = " " (menggunakan petik 2)

Number = tanpa petik 2 diatasnya

Boolean = True / false



- **Variabel**

Penamaan variabel

Nama variabel menggunakan font unta = kecil diawal kata selanjutnya besar

Tidak boleh ada spasi

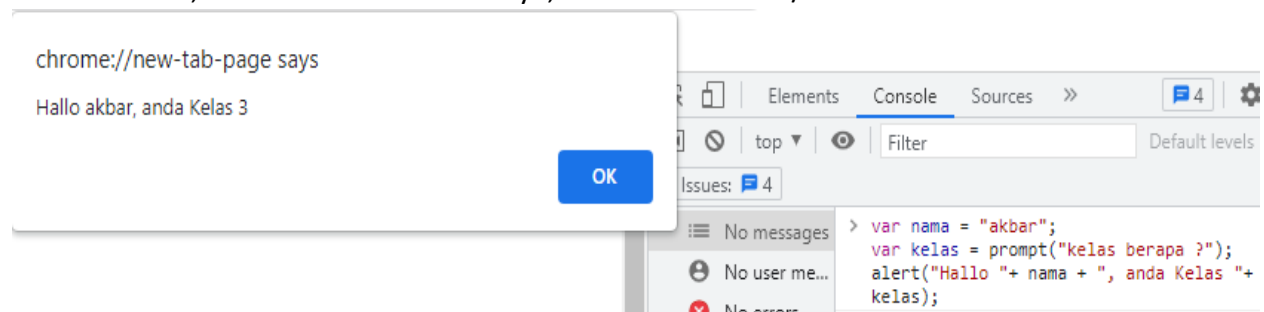
Boleh pake (_ / \$)

Syntax

Var namaAnda = "akbar";

Ket :

Var = variabel, nama = nama variabelnya, "akbar" = value / nilai dalam variabel



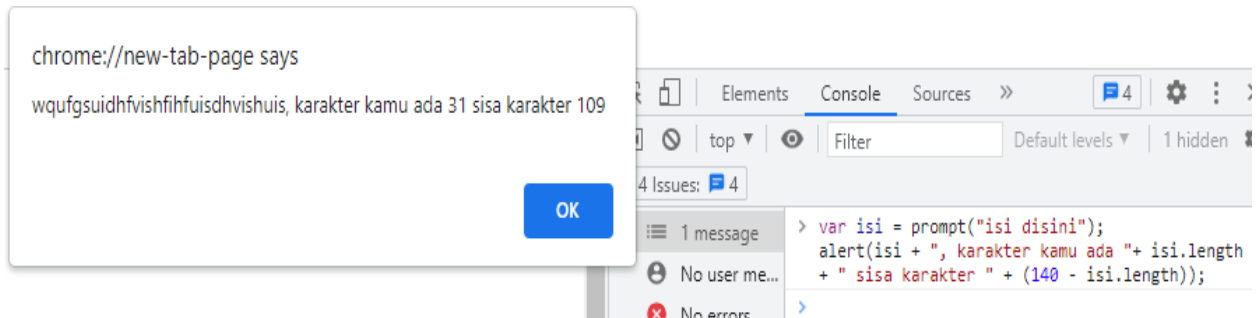
- **Length**

Menghitung jumlah karakter yang ada

Syntax

nama.length;

ket : nama = nama variabelnya



- **Slice (x, y)**

Slice = mengambil karakter tertentu pada index tertentu

Syntax :

name.slice(1, 2);

Ket : name = nama variabel, 1 = dimulai dari index 1, 2 = batas index yang diambil

Ingat:

Perhitungan index dimulai dari 0,1,2,3,4,5.....

Contoh

Var nama = "Akbar Anggit Pambudi";

Alert (nama.slice(2,3));

Hasilnya = b

Ket: 2 = mengambil karkater dimulai dari index 2

3 – 2 = 1, mengambil karakter sebanyak 1

- **toUpperCase()**

toUpperCase() = menjadikan huruf menjadi kapital

Contoh:

var nama = "akbar";

nama.toUpperCase();

Hasil = AKBAR

- **toLowerCase()**

toLowerCase() = menjadikan huruf yang kapital menjadi huruf kecil

Contoh:

var nama = "aKbAr";

nama.toLowerCase();

Hasil = akbar

- **Perhitungan Dasar**

Dalam perhitungan javascript hanya prioritaskan (tanda kurung), selain itu akan dihitung dari depan

Jenis	Syntax	Hasil	Keterangan
Penambahan	Var a = 1 + 2;	3	Symbol (+)
Pengurangan	Var a = 3 – 2;	1	Symbol (-)
Perkalian	Var a = 3 * 2;	6	Symbol (*)
Pembagian	Var a = 4 / 2;	2	Symbol (/)
Modulo (sisa pembagian)	Var a = 7 % 3;	1	Karena 7 dibagi 3 sisa 1

- **Increment (kenaikan / tambah 1) Dan Decrement (Pengurangan 1)**

Jenis	Syntax	Hasil
Increment (kenaikan 1)	Var a = 5; a++;	6
	Var a = 6; a+=2;	8
Decrement (penurunan 1)	Var a = 4; a--;	3
	Var a = 8; a-=2;	6

Yang bisa dipake disini diantaranya : +=, -=, *=, /=

- **Function (create dan call) Dan console.log**

Function	
Create (membuat function)	function buyMilk() { }
Call (memanggil function yang dibuat)	buyMilk ()

Contoh 1

Create

```
function hargaMinuman() {
    console.log("akbar harga minumannya " + 5000 );
}
```

Call

```
hargaMinuman();
```

Hasil

Akbar harga minumannya 5000

Contoh 2

```
> function beli(uang){
  var beli = uang - 5000;
  console.log(beli + " itu kembalianya ")
}
beli(10000);

5000 itu kembalianya VM729:3
< undefined
```

- **Parameter Function (Math.floor() / Pembulatan kebawah)**

Math.floor()

Math.floor() digunakan untuk membulatkan kebawah

Jika $8/3 = 2,6666$ maka jika menggunakan Math.floor() hasilnya jadi 2

INGAT : penulisan Math.floor() = M nya harus kapital

Contoh :

Jika harga roti 2300 dengan uang 3000 maka dapat berapa?

Jawab:

```
> function beliRoti(uangnya){
  var harga = Math.floor(uangnya / 2300);
  console.log(harga);
}
beliRoti(3000);

1 VM772:3
< undefined
```

- **Parameter Function (Math.round() / Pembulatan keterdekatan)**

Math.round() digunakan untuk membulatkan keterdekatan

Jika $8/3 = 2,6666$ maka jika menggunakan Math.round() hasilnya jadi 3

```
> var a = Math.round(8/3);
  console.log(a);

3 VM1436:2
```

- **Parameter Function (Math.pow() / Perhitungan pangkat)**

Math.pow() digunakan untuk menghitung pangkat

Contoh :

Hitunglah 7 pangkat 2 ?

Maka,

```
> var a = Math.pow( 7, 2 );
  console.log(a);

49 VM234:2
< undefined
```

- **Parameter Function (Math.random / Gatca Angka / Mengacak Angka)**

Math.random() = untuk melakukan gatca angka / memunculkan angka secara acak
Default

```
> var n = Math.random();  
    console.log(n);
```

0.6043936514884831

VM247:2

Ket : defaultnya Math.random, mengacak dari 0 – 0,9

Kemudian angka dibelakang koma (,), total ada 16 angka

Contoh 1 :

Jika , Math.random() * 6; maka angka yang keluar mulai dari 0, - 5,99

```
> var n = Math.random()*6;  
    console.log(n);
```

5.767595042408191

VM665:2

< undefined

Contoh 2:

Membuat presentasi kecocokan pasangan

```
> var nama1 = "Akbar";  
    var nama2 = "inda";  
    var kecocokan = Math.random()*100;  
    var hasil = Math.floor(kecocokan)+1;  
    console.log(nama1 + " dan " + nama2 + "  
    adalah " + hasil + "%");
```

Akbar dan inda adalah 87%

VM1193:5

< undefined

- Kasus (jika batas umur 90 tahun berapa sisa umur anda sekarang baik hari bulan tahun.
Cat: 1 tahun = 360 hari, 1 tahun = 12 bulan)

Jawab:

```
> function sisaUmur(umur){  
    var hari = (360 * 90) - (360 * umur);  
    var bulan = (12 * 90) - (12 * umur);  
    var tahun = 90 - umur;  
    console.log("sisa umur dalam hari "+hari+"  
dalam bulan "+ bulan + " dalam tahun "+  
tahun);  
}  
sisaUmur(26);
```

```
sisa umur dalam hari 23040 dalam          VM973:5  
bulan 768 dalam tahun 64
```

```
< undefined
```

- Return (mengembalikan nilai dalam function)

Contoh 1 :

```
> function angka(){  
    return(1 + 4);  
}  
var bilangan = angka();  
console.log(bilangan);
```

```
5          VM750:5
```

```
< undefined
```

Contoh 2 :

```
> function kali (angka1, angka2){  
    var hitung = angka1 * angka2;  
    return hitung;  
}  
  
console.log(kali(2,3));
```

```
6          VM549:6
```

```
< undefined
```

- Kasus (Menghitung Berat Badan Ideal (BMI))

Rumus:

$$\text{BMI} = \frac{\text{Berat Badan}}{(\text{Tinggi Badan})^2}$$

Maka,

```
> function beratIdeal( berat, tinggi ){
    var BMI = berat / (Math.pow(tinggi,2));
    return BMI;
}
console.log(beratIdeal(75,80));
0.01171875 VM2679:5
< undefined
```

- Statement IF Else (Jika Benar dan Jika Salah)

```
> var cek = true;
    if(cek===true){
        console.log("berarti cek benar");
    }else{
        console.log("berarti cek salah");
    }
berarti cek benar VM534:3
< undefined
```

Contoh:

```
> var acak = Math.floor(Math.random()*6);
    if(acak > 3){
        console.log("angka yang muncul lebih dari 3, dan hasilnya " + acak);
    }else{
        console.log("angka yang muncul lebih kecil sama dengan 3 dan hasilnya "+ acak);
    }
angka yang muncul lebih kecil sama dengan 3 dan hasilnya 3
< undefined
```

Contoh (IF ganda):

```
> var a = 6;
    if(a <= 5){
        console.log("angka lebih kecil sama dengan 5 dan hasilnya " + a);
    }if(a > 5 && a <10){
        console.log("angka lebih besar 5 dan kurang 10 dan hasilnya "+a);
    }else{
        console.log("angka lebih besar sama dengan 10 dan hasilnya "+ a);
    }
angka lebih besar 5 dan kurang 10 dan hasilnya 6
< undefined
```

- **Komperator (===, !==, >, <, >=, <=, &&, ||, !)**

Komperator	Keterangan
===	Sama dengan
!==	Tidak sama dengan
>	Lebih besar
<	Lebih Kecil
>=	Lebih besar sama dengan
<=	Lebih kecil sama dengan
&& / and	Dan (kedua sisi harus benar jika ingin true)
/ or	Atau (salah satu sisi harus benar biar true)
!	Tidak / Salah

- **Array (length array, push (menambahkan) , pop (memanggil))**

Array = kumpulan data yang disimpan dalam satu variabel

Var array = [0,1,2,3,4,5.....]; keterangan= array menghitung dari index 0,1,2,3.....

```
> var array = ["akbar", "anggit", "budi"];
  console.log( array[0] );
```

akbar

```
< undefined
```

```
> var array = ["akbar", "anggit", "budi"];
  console.log( array[2] );
```

budi

```
< undefined
```

Length (menghitung ada berapa index didalam array)

```
> var array = ["akbar", "anggit", "budi"];
  console.log(array.length);
```

3

Includes (mengecek)

```
> var data1 = [1,2,3,4];
  var data2 = prompt("masukan angka");
  if(data2.includes(data1)){
    alert("data yang dimasukan masuk data1");
  }else{
    alert("data yang dimasukan tidak masuk kedata1");
  }
```

chrome://new-tab-page says

data yang dimasukan tidak masuk kedata1

OK

Push (menambahkan data pada array)

namaVariabel.push();

```
> var array = [];  
    array.push("akbar");
```

```
< 1
```

```
> console.log(array[0]);
```

```
    akbar
```

```
< undefined
```

```
>
```

Pop (menghapus data paling belakang)

```
> var array= [1,2,3,4,5,3];  
    array.pop();
```

```
< 3
```

```
> console.log(array);
```

```
    ▶ (5) [1, 2, 3, 4, 5]
```

```
< undefined
```

```
.
```

- **Kasus FizzBuzz (jika habis dibagi 3 = "Fizz", Jika habis dibagi 5 ="Buzz")**

```
var array = [];  
var mulai = 1;  
function fizzBuzz(){  
    if(mulai % 3 === 0 && mulai % 5 === 0){  
        array.push("FizzBuzz");  
    }  
    else if(mulai % 3 === 0 ){  
        array.push("Fizz");  
    }else if(mulai % 5 === 0){  
        array.push("Buzz");  
    }else{  
        array.push(mulai);  
    }  
    mulai++;  
    console.log(array);  
}
```

```
> fizzBuzz();
```

```
    ▶ (15) [1, 2, 'Fizz', 4, 'Buzz', 'Fizz', 7, 8, 'Fizz', 'Buzz', 11, 'Fizz',  
    13, 14, 'FizzBuzz']
```

```
< undefined
```

[VM346:15](#)

- **Kasus Acak Nama**

```
> var nama = ["akbar", "anggit", "pambudi"];  
    function acakNama(){  
        var index = nama.length;  
        var acak = Math.floor(Math.random()*index);  
        var hasil = nama[acak];  
        return ("nama yang keluar adalah "+ hasil);  
    }  
    acakNama();  
    < 'nama yang keluar adalah anggit'
```

- **Loop (menjalankan pada batas tertentu) (while (memeriksa keadaan yang selalu benar jika salah berhenti)) (for (menjalankan terus walau salah))**

While

Menjalankan terus menerus sampai batas ditentukan tetapi jika ditengah-tengah ada keadaan yang salah akan berhenti

```
> var i=0;  
    while( i<10){  
        console.log(i)  
        i++  
    }
```

0

1

2

3

4

5

6

7

8

9

< 9

For

Menjalankan terus menerus sampai batas ditentukan meskipun keadaan ada yang salah

```
> for(var i = 0; i < 100; i++){  
    console.log(i);  
}
```

0

1

2

3

4

- **Switch (Case, Default)**

Switch merupakan loop ketika case pertama sesuai maka akan berhenti berjalan

Contoh

```
> var acak = Math.floor(Math.random()*5)+1;
  switch(acak){
    case 1:
      console.log("hasilnya 1");
      break;
    case 2:
      console.log("hasilnya 2");
      break;
    case 3:
      console.log("hasilnya 3");
      break;
    case 4:
      console.log("hasilnya 4");
      break;
    case 5:
      console.log("hasilnya 5");
      break;
    default:
      console.log("error");
  }
```

hasilnya 2

DOM (DOCUMENT OBJECT MODEL)

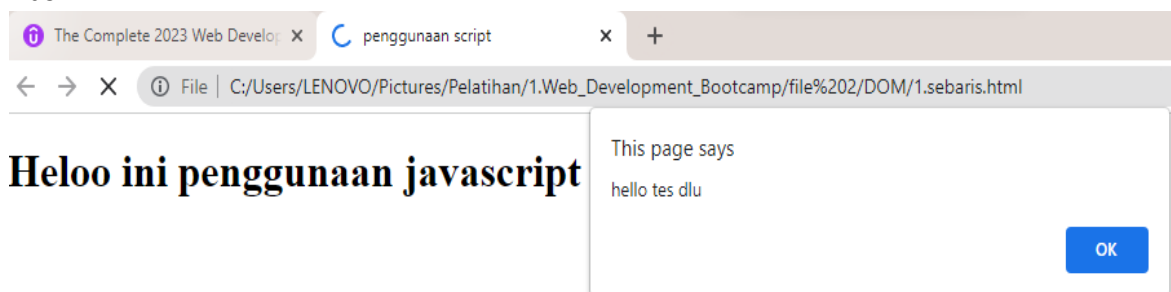
- Penggunaan javascript (sebaris, internal External)

Sebaris Javascript = penggunaan java script langsung pada body html

File html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>penggunaan script</title>
</head>
<body onload="alert('hello tes dlu') ">
  <!-- onload adalah penggunaan java script sebaris dengan body -->
  <h1>Heloo ini penggunaan javascript sebaris</h1>
</body>
</html>
```

Hasil

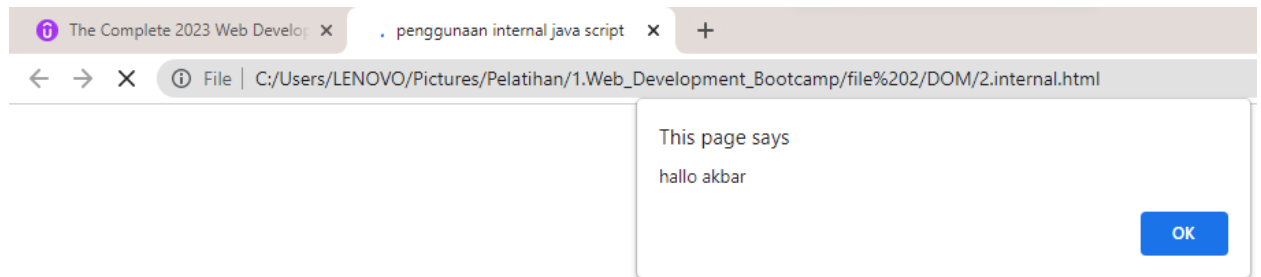


Internal Javascript = penggunaan java script terpisah dan mash dalam file html yang sama

File Html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>penggunaan internal java script</title>
</head>
<body>
  <h1>hello ini penggunaan internal javascript</h1>
  <script> // ini penggunaan internal java script
    alert("hallo akbar");
  </script>
</body>
</html>
```

Hasil



Eksternal javascript = penggunaan java script difile terpisah

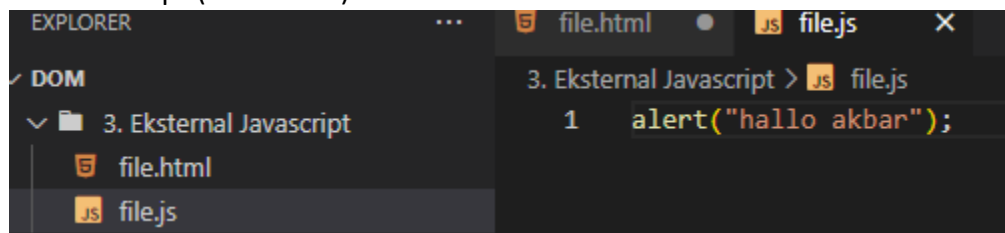
Ingat untuk penggunaan eksternal = sebagikanya ditulis didalam body paling bawah, agar script dijalankan terakhir dan tidak erorr

File HTML

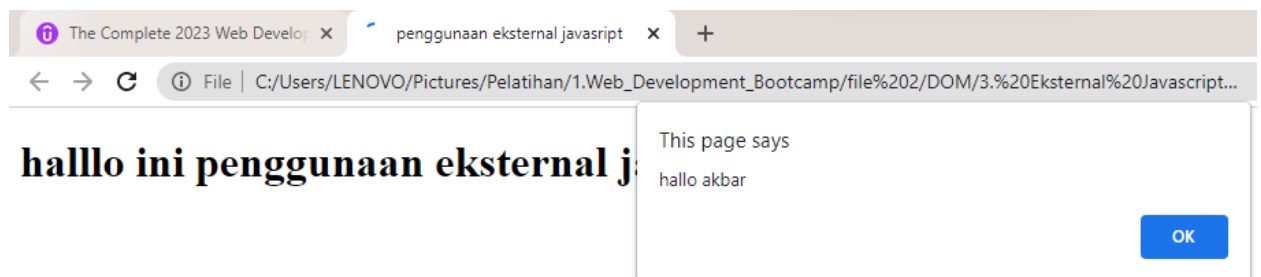
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>penggunaan eksternal javascript</title>
</head>
<body>
  <h1>hallo ini penggunaan eksternal javascript</h1>

  <script src="file.js"></script> <!-- ini link untuk panggil Javascript
eksternal -->
</body>
</html>
```

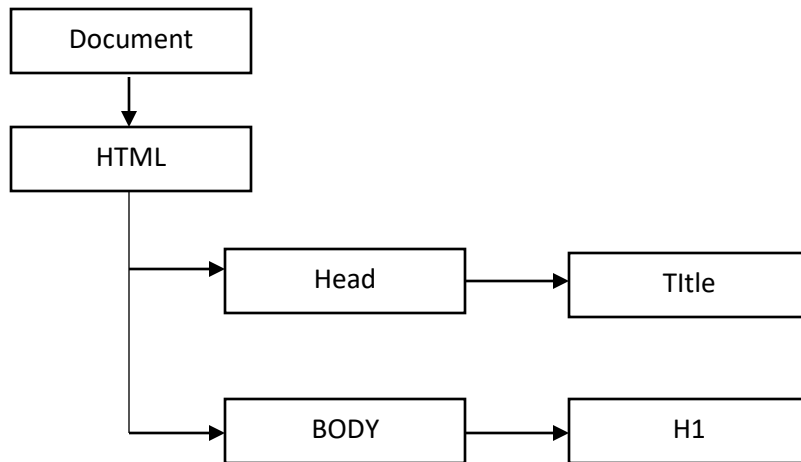
File Javascript (eksternal)



Hasil



- **Pengenalan DOM**

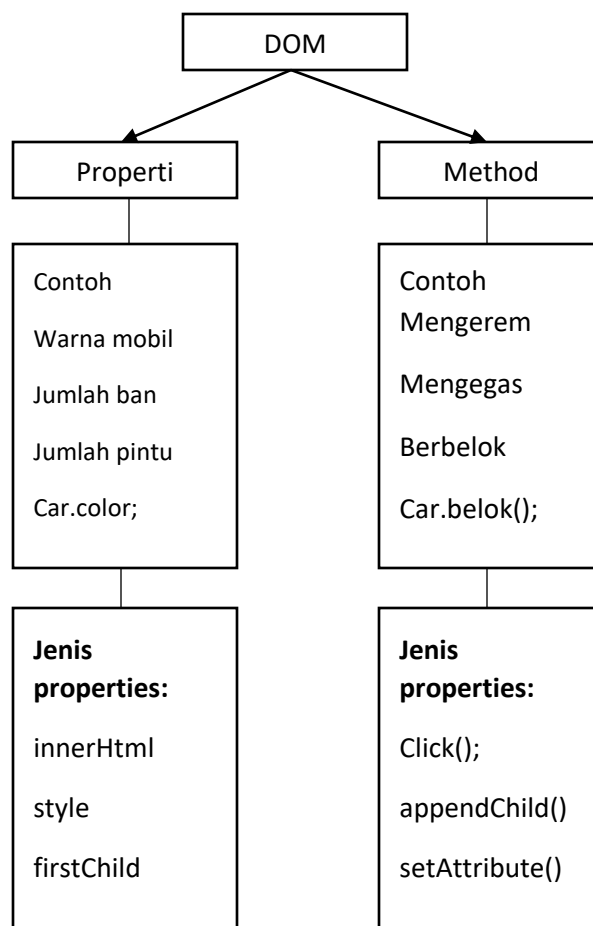


Jika ingin melihat isi html

`Document.firstChild;`

Jika ingin melihat isi body pada html

`Document.firstChild.lastElementChild`



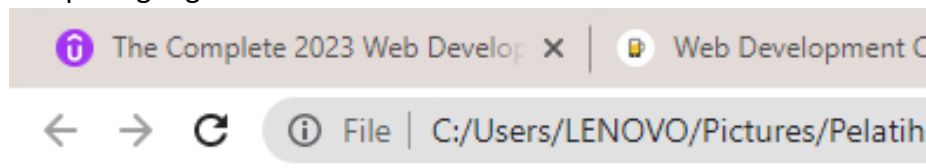
- Selector (1. Get element Dan 2. Query Selector (Rekomendasi))
File html

```
<!DOCTYPE html>
<html lang="en" dir="ltr">
  <head>
    <meta charset="utf-8">
    <title>My Website</title>
    <link rel="stylesheet" href="styles.css">

  </head>
  <body>
    <h1 id="tes">Hello</h1>
    <input type="checkbox">
    <button class="btn">Click Me</button>
    <ul id="lis">

      <li class="list">
        <a href="https://www.google.com">Google</a>
      </li>
      <li class="list">Second</li>
      <li class="list">Third</li>
    </ul>
    <script src="index.js"></script>
  </body>
</html>
```

Tampilan google



Hello

☐ Click Me

- [Google](#)
- Second
- Third

Get Elements

getElementsByName(); = mencari berdasarkan nama elemennya

Contoh 1: mencari nama element "li" diatas, maka

```
> document.getElementsByTagName("li");  
< ▼ HTMLCollection(3) [li.list, li.list, li.list] ⓘ  
  ▶ 0: li.list  
  ▶ 1: li.list  
  ▶ 2: li.list  
    length: 3  
  ▶ [[Prototype]]: HTMLCollection
```

Contoh2: rubah warna li khususnya index 0, maka

```
> document.getElementsByTagName("li")[0].style.color="gold";  
< 'gold'
```

Hasil

Hello

☐ Click Me

- [Google](#)
- Second
- Third

document.getElementsByClassName(); = mencari berdasarkan nama classnya

Contoh1: cari kelas yang Bernama btn, maka

```
> document.getElementsByClassName("btn");  
< ▼ HTMLCollection [button.btn] ⓘ  
  ▶ 0: button.btn  
    length: 1  
  ▶ [[Prototype]]: HTMLCollection
```

>

Contoh2: rubah warna pada class btn dengan warna emas, maka

```
> document.getElementsByClassName("btn")[0].style.color="gold";  
< 'gold'
```

Hasil

Hello

☐ Click Me

- [Google](#)
- Second
- Third

document.getElementById(); = mencari berdasarkan nama classnya

```
> document.getElementById("tes");  
< <h1 id="tes">Hello</h1>
```

Query Selector (Recomendasi)

Jenis	Sintax	Keterangan
Selector class	Document.querySelector("#namaClassnya");	Gunakan # didpan nama id
Selector id	Document.querySelector(".namaIdnya ");	Gunakan . (titik) didpan nama class
Selector berdasarkan nama	Document.querySelector("namanya");	Nama seperti = h1, p, li, a
Selector gabungan	Document.querySelector("gabungan");	gabungan

Contoh

Selector class

contoh : mencari nama kelas btn, maka

```
> document.querySelector(".btn");  
< <button class="btn">Click Me</button>
```

Selector Id

Contoh: mencari nama id tes, maka

```
> document.querySelector("#tes");  
< <h1 id="tes">hayyy</h1>
```

Selector berdasarkan nama (h1, p, li, a, img)

Contoh : mencari h1

```
> document.querySelector("h1");  
< <h1 id="tes">hayyy</h1>
```

Selector gabungan

Contoh 1 : mencari list yang terdapat link

```
> document.querySelector("li a");  
< <a href="https://www.google.com">Google</a>
```

Contoh 2: mencari id "tes" yang ada pada h1

```
> document.querySelector("h1#tes");  
< <h1 id="tes">hayyy</h1>
```

Contoh 3: cari a (link), yang ada nama id = "lis"

```
> document.querySelector("#lis a ")  
< <a href="https://www.google.com">Google</a>
```


#Query Selector ALL = menampilkan semua yang memiliki class sama

Contoh 1 : tampilkan semua list yang memiliki class list

```
> document.querySelectorAll(" li.list ")
< NodeList(3) [li.list, li.list, li.list]
```

Contoh2 : tampilkan list index 0 yang memiliki class list

```
> document.querySelectorAll(" li.list ")[0]
< <li class="list">
  ::marker
  <a href="https://www.google.com">Google</a>
</li>
```

Contoh3: rubah warna lis index 0 yang memiliki class list

```
> document.querySelectorAll(" li.list ")[0].style.color="yellow";
< 'yellow'
```

Hasil

Hello

☐ Click Me

- [Google](#)
- Second
- Third

- **Dom Style**

link dom style = https://www.w3schools.com/jsref/dom_obj_style.asp

File.CSS	File.Javascript
h1{ Color: red; }	Document.querySelector("h1").style.color="red";
h1{ font-size:12em; }	Document.querySelector("h1").style.fontSize="12em";

Contoh : atur padding h1 10%, maka

```
> document.querySelector("h1").style.padding="10%";
< '10%'
```

Hasil

Hello

☐ Click Me

- [Google](#)
- Second
- Third

- Dom Manipulasi (Add class, Remove Class)
#Cek Nama Class Dan Menambahkan Nama Class

File Html

```
<body>
  <h1 id="tes">Hello</h1>
  <input type="checkbox">
  <button class="btn">Click Me</button>
  <ul id="lis">
    <li class="list">
      <a href="https://www.google.com">Google</a>
    </li>
    <li class="list">Second</li>
    <li class="list">Third</li>
  </ul>
  <script src="index.js"></script>
</body>
```

Cek nama kelas input

Hasil = tidak ada nama kelas

```
> document.querySelector("input").classList
< ▼ DOMTokenList [value: ''] ⓘ
  length: 0
  value: ""
  ► [[Prototype]]: DOMTokenList
>
```

Tambahkan nama kelas input "iniClass"

Hasil

```
> document.querySelector("input").classList.add("iniClass");
< undefined
> document.querySelector("input").classList
< ▼ DOMTokenList ['iniClass', value: 'iniClass'] ⓘ
  0: "iniClass"
  length: 1
  value: "iniClass"
  ► [[Prototype]]: DOMTokenList
```

#Remove Class (Remove dan Toggle(mengaktifkan class jika ingin digunakan atau menonaktifkan kelas jika ingin)

Remove

```
> document.querySelector("input").classList.remove("iniClass");
< undefined
> document.querySelector("input").classList;
< ► DOMTokenList [value: '']
```

Toggle

```
> document.querySelector("input").classList.toggle("iniClass");  
< false  
> document.querySelector("input").classList.toggle("iniClass");  
< true
```

- **Dom Manipulasi (innerHTML dan textContent)**

Fungsi keduanya sama merubah text yang ada pada html

Perbedaan:

InnerHTML = bisa rubah jenis text

```
> document.querySelector("h1").innerHTML="<em>Hayyy</em>"  
< '<em>Hayyy</em>'
```

Hasil

Hayyy

☐ Click Me

- [Google](#)
- Second
- Third

textContent = tidak bisa rubah font text

```
> document.querySelector("h1").textContent="<em>heyyy</em>";  
< '<em>heyyy</em>'
```

Hasil

heyyy

☐ Click Me

- [Google](#)
- Second
- Third

- **Dom Attribute (Attributes, Get Attribute, Set Attribute)**

attributes = mengecek atribut pada class atau nama atau id tertentu

Contoh cek attributes yang dimiliki h1, maka

```
> document.querySelector("h1").attributes;  
< ▶ NamedNodeMap {0: id, id: id, Length: 1}
```

Get Attribute = mendapatkan isi dari attribute tertentu

Contoh : dapatkan link dari attribute a (link)

```
> document.querySelector("a").getAttribute("href");  
< 'https://www.google.com'
```

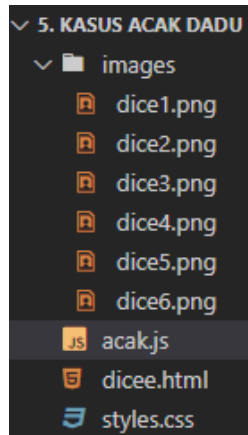
Set Attribute = mengganti isi attribute tertentu

Contoh : ganti link google dengan “www.bing.com”

```
> document.querySelector("a").setAttribute("href", "www.bing.com");  
<< undefined  
  
> document.querySelector("a").getAttribute("href");  
<< 'www.bing.com'
```

- Kasus Dadu Acak (ketika direfresh dadu akan acak)

File



File Html

```
<!DOCTYPE html>  
<html lang="en" dir="ltr">  
  <head>  
    <meta charset="utf-8">  
    <title>Dicee</title>  
    <link rel="stylesheet" href="styles.css">  
    <link href="https://fonts.googleapis.com/css?family=Indie+Flower|Lobster" rel="stylesheet">  
  </head>  
  <body>  
    <div class="container">  
      <h1>Refresh Me</h1>  
      <div class="dice">  
        <p>Player 1</p>  
          
      </div>  
      <div class="dice">  
        <p>Player 2</p>  
          
      </div>  
    </div>  
    <script src="acak.js"></script>    <!-- memanggil file javascript eksternal -->  
  </body>
```

```
<footer>
  www 🍺 App Brewery 🍺 com
</footer>
</html>
```

File Javascript eksternal

```
// Acak Dadu 1
var acak1 = Math.floor(Math.random()*6)+1; // mengacak 1-6 karena nama gambar
dice1,dice2..dice6
var gambar1 = "images/dice"+ acak1 + ".png";
document.querySelector(".img1").setAttribute("src", gambar1); //ubah atribut src
agar gambar acak

// Acak Dadu 2
var acak2 = Math.floor(Math.random()*6)+1;
var gambar2 = "images/dice" + acak2 + ".png";
document.querySelector(".img2").setAttribute("src", gambar2);

if(acak1 > acak2){ //jika angka acak 1 lebih besar angka acak 2
  document.querySelector("h1").innerHTML="Player 1 Win!"; // maka h1 rubah jadi
player 1 win!
}else if(acak1 < acak2){ //jika angka acak1 lebih kecil angka acak 2
  document.querySelector("h1").innerHTML="Player 2 Win!"; // maka h1 rubah jadi
player 2 win!
}else{ // jika tidak kedua diatas
  document.querySelector("h1").innerHTML="Draw"; // maka h1 rubah jadi draw
}
```

Hasil



- **AddEventListener (memberikan respon ketika target diklik)**

Contoh:

```
Document.querySelector(" h1 ").addEventListener( "click", responnya );
```

```
Function responnya(){
  alert("katakan hallo");
}
```

- **Kasus Kalkulator Sederhana (function, retrun)**

```
> function tambah(num1, num2){
    return (num1 + num2);
}
function kali( num1,num2 ){
    return ( num1 * num2 );
}
function kalkulator( num1, num2, operator ){
    return operator( num1, num2 );
}
< undefined
> kalkulator(1, 2, tambah);
< 3
> kalkulator(1, 2, kali);
< 2
>
```

- **Play audio javascript**

```
var audio = new Audio('audio_file.mp3');
audio.play();
```

- **Contractor Function (This)**

Contoh tanpa konstruktor function (kurang efektif)

```
> var karyawan = { nama:"akbar", umur:"16 tahun", bahasa:["indo", "arab"] };
< undefined
> karyawan
< ▶ {nama: 'akbar', umur: '16 tahun', bahasa: Array(2)}
> // jika ingin tambah maka tulis lagi
< undefined
> var karyawan2 = { nama:"akbar", umur:"16 tahun", bahasa:["indo", "arab"] };
```

Contoh dengan konstruktor function

Konstruktor function

```
> function konstruktorKaryawan (nama, umur, asal, bahasa){
  this.nama = nama;
  this.umur = umur;
  this.asal = asal;
  this.bahasa = bahasa;
}
< undefined
```

Memasukan data kedalam konstruktor

```
> var karyawan1 = new konstruktorKaryawan ("akbar", "17 tahun", "cilacap",
  ["indo","arab"]);
< undefined

> var karyawan2 = new konstruktorKaryawan ("budi", "17 tahun", "cilacap",
  ["indo","arab"]);
```

Jika dilihat data karyawannya

```
> karyawan2
< ► konstruktorKaryawan {nama: 'budi', umur: '17 tahun', asal: 'cilacap', bah
  asa: Array(2)}

> console.log(karyawan1.nama);
akbar
VM4894:1
< undefined
```

Pada data karyawan2 otomatis ada nama, umur, asal, Bahasa.

Kemudian jika dipanggil data karyawan1 berdasarkan nama juga bisa walaupun pada saat memasukan data tidak menambahkan nama itu karna dibuatkan konstruktornya

- **Function Construktor (Ada Fungsi didalam fungsi)**

Contoh1:

```
> function orang(nama, umur){
  this.nama = nama;
  this.umur = umur;
  this.kegiatan = function(){
    alert("lagi belajar");
  }
}
< undefined

> var orang1 = new orang("budi", "17 tahun");
< undefined

> orang1
< ► orang {nama: 'budi', umur: '17 tahun', kegiatan: f}

> orang1.kegiatan();
< undefined
```

Hasil

```
chrome://new-tab-page says  
lagi belajar
```

OK

Contoh 2:

Konstruktur fungsi

```
> function Audio (fileLocation){  
    this.fileLocation = fileLocation;  
    this.play= function(){  
  
    }  
}  
← undefined
```

Tambahkan variabel music kedalam audio

```
> var tom = new Audio("musik/aaa.mp3");  
← undefined
```

Jalankan fungsi music

```
> tom.play();  
← undefined
```

- **Callback Function (Memanggil fungsi)**

Contoh :

```
> function Callback(nama){  
    console.log("Hallo " + nama);  
}  
← undefined  
> Callback("Akbar");  
Hallo Akbar
```

- **Set Time Out (animasi web)**

Link : https://www.w3schools.com/js/js_timing.asp

Contoh:

```
> function tes(){  
    setTimeout(function(){ alert("hello"); }, 3000);  
}  
← undefined  
> tes();
```

Setelah dijalankan 3000 milisecond maka akan keluar

```
chrome://new-tab-page says  
hello
```

OK

JQuery

- Perbedaan Dasar JQuery dan JavaScript

Perbedaan		
Syntax JavaScript	Syntax JQuery	Keterangan
Document.querySelector("h1");	\$("h1"); Atau jQuery("h1");	jQuery / \$ = singkatan dari document.querySelector

- Instalasi CDN Script JQuery Pada Html

Copy link CDN JQuery chipset terbaru pada html =

<https://developers.google.com/speed/libraries#jquery>

File html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Instalasi CDN JQuery</title>

  <!-- link script CDN JQuery -->
  <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.3/jquery.min.js"></script>
  <!-- link javascript eksternal -->
  <script src="script.js"></script>
</head>
<body>
  <h1>Hallo</h1>
</body>
</html>
```

File javascript eksternal

```
// agar link script bisa diletakan pada bagian head
$(document).ready(function(){ //tambahkan fungsi document ready
  $("h1").css("color", "red");
  // .css menggantikan document.querySelector("h1").style.color="red";
});
```

Hasil

← → C

Hallo

- **Selector JQuery**

Jenis	Syntax	Keterangan
Nama element	<code>\$("Button");</code>	Nama elemen : p, h1, button, img, a
Class	<code>\$(".namaKelas");</code>	Pake titik didepanya (.)
Id	<code>\$("#namaid");</code>	Pake tanda pager didepan (#)
Gabungan class, id, atau nama element	<code>\$(#id.kelass)</code> <code>\$(a.kelaas)</code>	Id dan kelas Nama elemen dan kelas

- **Manipulasi (Style, tambah kelas (addClass))**

Style JQuery

Perbandingan		
JavaScript	jQuery	Keterangan
<code>Document.querySelector("h1").style.color = "red";</code>	<code>\$(" h1 ").css("color", "red");</code>	Gunakan .css

Contoh :

Jquery

```
$("h1").css("font-size", "12em");
```

JavaScript

```
document.querySelector("h1").style.fontSize="12em";
```

Menambahkan Nama Kelas (addClass)

Menambahkan nama kelas tunggal

```
$("h1").addClass("namaKelas");
```

Menambahkan nama kelas ganda (pake spasi)

```
$("h1").addClass("namaKelas1 namaKelas2");
```

Mengecek nama kelas (hasClass)

```
$("h1").hasClass("benerKelasIni");
```

- **Manipulasi text (text vs textContent Dan html vs innerHtml)**

text vs textContent

Hanya bisa merubah text tidak merubah font text

Contoh: "hallo"

```
$("h1").text("hallo");
```

html vs innerHtml

bisa merubah font text

contoh: "hello"

```
$("h1").html("<em>hello</em>");
```

- **Attribute (attr)**

Cek Attribute

```
$("#a").attr("href");
// contoh 2
$("#img").attr("src");
```

```
$("#h1").attr("class"); // cek nama kelas yang dimiliki h1
```

Edit isi Attribute

```
$("#a").attr("href", "www.google.com");
```

- **Event / addEventListener**

```
$("#button").click(function(){ // event ketika diklik
    $("#h1").css("color","gold"); // maka warna h1 akan berubah warna gold
});
$("#input").keypress(function(tes){ // event ketika keyboard ditekan
    $("#h1").html(tes.key); // setelah ditekan maka h1 akan ganti text berdasarkan
yang ditekan
});
// dalam event jQuery bisa gunakan on
$("#button").on("click", function(){
    $("#h1").css("color", "gold");
});
```

- **Menambahkan element (p, h1, button, dst) (before(), after(), prepend(), append())**

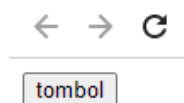
Cat: element baru yang ditambahkan tidak merubah kodingan html hanya pada console

before() = menambahkan element sebelum

Contoh: tambahkan element button sebelum/diatas elemen h1 maka,

```
$("#h1").before("<button>tombol</button>");
```

Hasil



Hallo

after() = menambahkan element setelah

Contoh: tambahkan element button sesudah/dibawah elemen h1 maka,

```
$("#h1").after("<button>tombol</button>");
```

Hasil



Hallo

append() = menambahkan element sebelah kanan

Contoh: tambahkan element button sebelah kanan elemen h1 maka,

```
$("#h1").append("<button>tombol</button>");
```

Hasil



Hallo tombol

prepend() = menambahkan element sebelah kiri

Contoh: tambahkan element button sebelah kiri elemen h1 maka,

```
$("#h1").prepend("<button>tombol</button>");
```

Hasil



tombol **Hallo**

- Animasi JQuery (slideup, toggle, fadeout, fadeIn, dst)

Link animasi : <https://api.jquery.com/category/effects/>

Contoh 1 : toggle() = ketika diklik maka h2 ilang dan muncul lagi setelah diklik lagi, maka

```
$("#button.tes").click(function(){ // event ketika diklik
    $("#h2").toggle(); // maka warna h2 ilang
});
```

Contoh2 : animate() = animasi yang bisa custom perubahanya

Ketika diklik h2 margi jadi 20%

```
$("#button.tes").click(function(){ // event ketika diklik
    $("#h2").animate({margin: "20%"}); // maka margi h2 jadi 20%
});
```

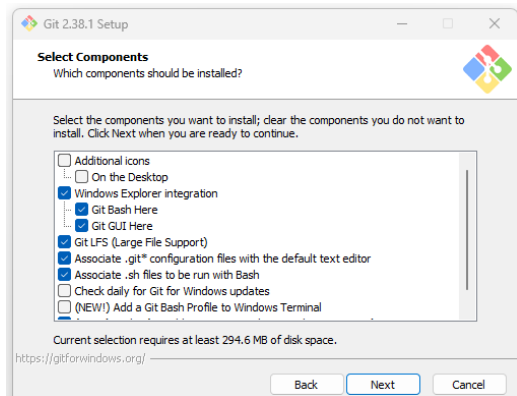
Command Terminal (Hyper)

- **Instalasi**

Download dan install hyper, link = <https://hyper.is/#installation>

Download Git Bash , link = <https://git-scm.com/downloads>

Pada install git = pada bagian Select Component = pastikan git bash here tercentang, selain itu next aja



- **Konfigurasi Hyper**

Copy file konfigurasi, link = <https://gist.github.com/coco-napky/404220405435b3d0373e37ec43e54a23>

Atau file dibawah ini

```
module.exports
```

```
= {
```

```
  config: {
    // default font size in pixels for all tabs
    fontSize: 12,

    // font family with optional fallbacks
    fontFamily: 'Menlo, "DejaVu Sans Mono", Consolas, "Lucida Console",
    monospace',

    // terminal cursor background color and opacity (hex, rgb, hsl, hsv, hwb
    or cmyk)
    cursorColor: 'rgba(248,28,229,0.8)',

    // `BEAM` for |, `UNDERLINE` for _, `BLOCK` for █
    cursorShape: 'BLOCK',

    // color of the text
    foregroundColor: '#fff',

    // terminal background color
```

```

backgroundColor: '#000',

// border color (window, tabs)
borderColor: '#333',

// custom css to embed in the main window
css: '',

// custom css to embed in the terminal window
termCSS: '',

// set to `true` (without backticks) if you're using a Linux setup that
doesn't show native menus
// default: `false` on Linux, `true` on Windows (ignored on macOS)
showHamburgerMenu: '',

// set to `false` if you want to hide the minimize, maximize and close
buttons
// additionally, set to `left` if you want them on the left, like in
Ubuntu
// default: `true` on windows and Linux (ignored on macOS)
showWindowControls: '',

// custom padding (css format, i.e.: `top right bottom left`)
padding: '12px 14px',

// the full list. if you're going to provide the full color palette,
// including the 6 x 6 color cubes and the grayscale map, just provide
// an array here instead of a color map object
colors: {
  black: '#000000',
  red: '#ff0000',
  green: '#33ff00',
  yellow: '#ffff00',
  blue: '#0066ff',
  magenta: '#cc00ff',
  cyan: '#00ffff',
  white: '#d0d0d0',
  lightBlack: '#808080',
  lightRed: '#ff0000',
  lightGreen: '#33ff00',
  lightYellow: '#ffff00',
  lightBlue: '#0066ff',

```

```

    lightMagenta: '#cc00ff',
    lightCyan: '#00ffff',
    lightWhite: '#ffffff'
  },

  // the shell to run when spawning a new session (i.e.
  /usr/local/bin/fish)
  // if left empty, your system's login shell will be used by default
  // make sure to use a full path if the binary name doesn't work
  // (e.g `C:\\Windows\\System32\\bash.exe` instad of just `bash.exe`)
  // if you're using powershell, make sure to remove the `--login` below
  shell: 'C:\\Program Files\\Git\\git-cmd.exe',

  // for setting shell arguments (i.e. for using interactive shellArgs: ['-i'])
  // by default ['--login'] will be used
  shellArgs: ['--command=usr/bin/bash.exe', '-l', '-i'],

  // for environment variables
  env: { TERM: 'cygwin' },

  // set to false for no bell
  bell: 'SOUND',

  // if true, selected text will automatically be copied to the clipboard
  copyOnSelect: false

  // if true, on right click selected text will be copied or pasted if no
  // selection is present (true by default on Windows)
  // quickEdit: true

  // URL to custom bell
  // bellSoundURL: 'http://example.com/bell.mp3',

  // for advanced config flags please refer to https://hyper.is/#cfg
},

// a list of plugins to fetch and install from npm
// format: [@org/]project[#version]
// examples:
//   `hyperpower`
//   `@company/project`
//   `project#1.0.1`

```

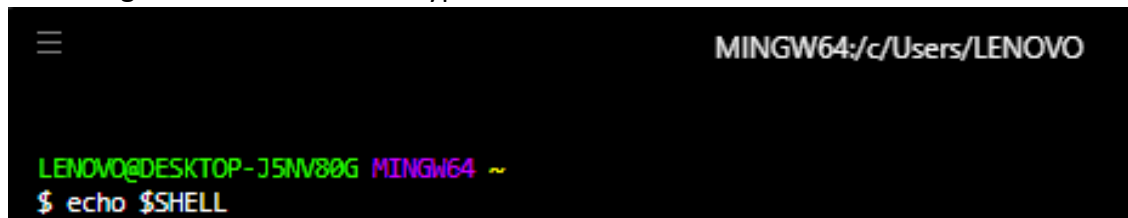
```
plugins: [],

// in development, you can create a directory under
// `~/.hyper_plugins/local/` and include it here
// to load it and avoid it being `npm install`ed
localPlugins: []
};
```

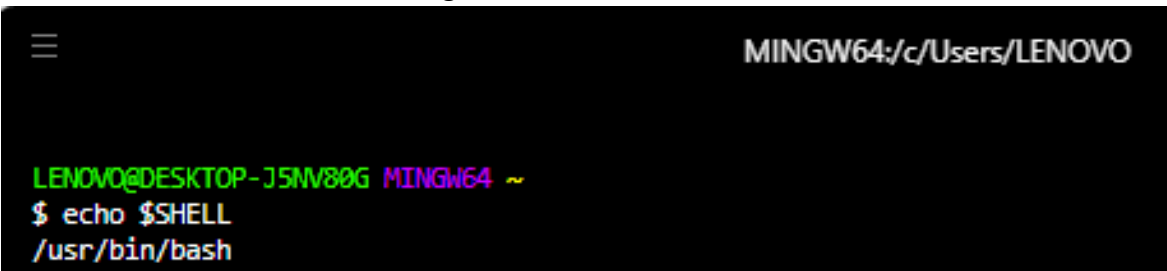
Buka hyper > Edit > preference

Copikan file configarusi kedalam note terebut, kemudian save dan tutup hyper.

Cek konfigurasi berhasil = buka hyper kemudian ketikan echo \$SHELL

A terminal window with a black background and white text. The title bar at the top reads 'MINGW64:/c/Users/LENOVO'. The prompt is 'LENOVO@DESKTOP-J5NV80G MINGW64 ~'. The command '\$ echo \$SHELL' has been entered and is highlighted in green.

Jika muncul bin/bash berarti konfigurasi berhasil

A terminal window with a black background and white text. The title bar at the top reads 'MINGW64:/c/Users/LENOVO'. The prompt is 'LENOVO@DESKTOP-J5NV80G MINGW64 ~'. The command '\$ echo \$SHELL' has been entered, and the output '/usr/bin/bash' is displayed below it.

- **Baris Perintah Command**

mkdir = membuat folder baru

Contoh : mkdir namaFolder

mkdir .Secrets = membuat folder tersembunyi

Contoh : mkdir .Secrets

touch : membuat file

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/7. Commad Line
$ touch file.txt
```

start = membuka file

Contoh: start file.txt

ls (list) = menampilkan semua isi file pada folder

Contoh : ls

rm = menghapus file dan folder

Contoh : rm file.txt

rm -r = menghapus folder

Contoh: rm -r namaFolder/

#**rm *** = menghapus semua file dalam folder sama

cd = berpindah folder

Contoh :

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~
$ cd Pictures/

LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures
```

Tips : cd dow lalu tekan tabs = makan akan langsung cd download/

cd ~ = untuk langsung kembali ke folder awal

cd .. = untuk mundur kefolder sebelumnya (satu tingkat)

#**pwd** = cek alur lokasi

cara cepat = ketik cd lalu drop folder yang ingin dibuka

alt + klik = untuk mengganti command di tengah agar tdk kelamaan main anak panah kiri

ctrl + A = untuk berpindah kursor langsung pada bagian depan

ctrl + E = untuk berpindah kursor langsung pada bagian belakang

ctrl + U = untuk menghapus semua command yang sedang diketik (berlangsung)

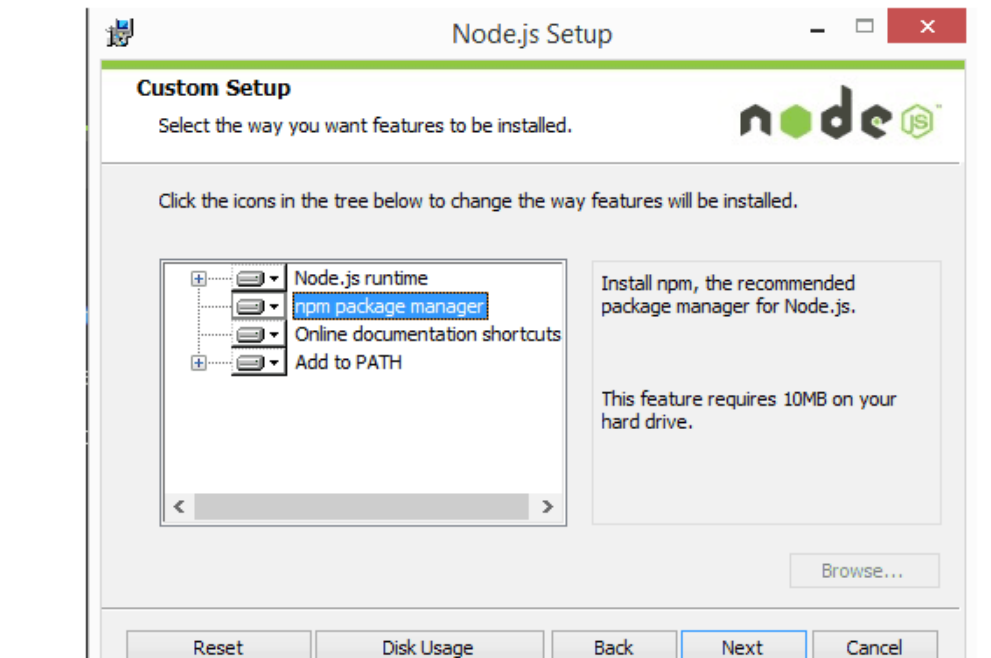
NODE.JS

- **Instalasi Node.Js**

Download Node JS, link = <https://nodejs.org/en/>

Langkah Instalasi

1. **Unduh penginstal Windows** dari [situs web Nodes.js®](https://nodejs.org/en/).
2. **Pilih versi LTS** yang ditampilkan di sebelah kiri.
3. **Jalankan penginstal** (file .msi yang Anda unduh di langkah sebelumnya.)
4. **Ikuti petunjuk di penginstal** (Terima perjanjian lisensi, klik tombol BERIKUTNYA berkali-kali dan terima pengaturan penginstalan default).



5. **Nyalakan kembali komputer Anda.** Anda tidak akan dapat menjalankan Node.js® sampai Anda me-restart komputer Anda.

6. Konfirmasikan bahwa Node telah berhasil diinstal di komputer Anda dengan membuka terminal Hyper dan mengetikkan perintah `node --version`

Anda akan melihat versi node yang baru saja Anda instal.

- **Menjalankan file js dengan node di terminal**

Buat file js yang berisi `console.log("hello world");`

Pada folder yang menyimpan tersebut jalan kan node filejsnya.js

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/8. Node.js/1. menjalankan file js dengan node
$ ls
index.js

LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/8. Node.js/1. menjalankan file js dengan node
$ node index.js
```

Maka hasilnya akan muncul hello world

```
hello world
```

- **Node REPL (Read, Evaluation, Print, Loop) (menggunakan perintah node untuk menjalankan hyper seperti console yang ada di chrome)**

Untuk menjalankan ketika = node

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~
$ node
Welcome to Node.js v18.12.1.
Type ".help" for more information.
> 
```

Setelah itu, lakukan yang diinginkan sama seperti console chrome

Contoh

```
> 1 + 2;
3
> console.log("Hello Akbar");
Hello Akbar
undefined
> 
```

Cara keluar dari mode console node

Bisa ketikan = `.exit`

Atau tekan `Ctrl + C` sebanyak 2X

- Menerapkan modul atau paket (require()) dan menyalin file menjadi 2
require (menerapkan modul atau paket)

```
//jshint esversion:6
// agar tidak keluar peringatan ketikkan diatas

const fs = require("fs");
```

Copy file

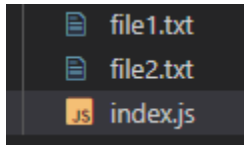
Contoh : buat file1.txt, salin isi file tersebut ke dalam file2.txt

```
1  ✓ //jshint esversion:6
2  // agar tidak keluar peringatan ketikkan diatas
3
4  const fs = require("fs");
5
6  //salin file1 menjadi 2 file yaitu file1.txt dan file2.txt
7  fs.copyFileSync("file1.txt","file2.txt"); //gunakan copyFileSync
```

Jalankan pada terminal file javascriptnya

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/8. Node.js/2. menerapkan
an paket (require)
$ node index.js
```

Hasilnya otomatis file akan bertambah sendiri



- **Instalasi NPM (Node Package Manager) Pada Folder**

NPM = kumpulan kode terbesar didunia, sehingga npm merupakan potongan-potongan kode yang ditulis orang lain untuk digunakan oleh kita

Install NPM = agar dapat menggunakan code npm orang lain

Ketik **npm init** pada folder proyek kita

Contoh:

```
LENOVO@DESKTOP-J5NW80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/
8. Node.js/3. Penggunaan NPM
$ npm init
```

Maka akan keluar

```
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.
```

```
See `npm help init` for definitive documentation on these fields
and exactly what they do.
```

```
Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.
```

```
Press ^C at any time to quit.
```

```
package name: (3.-penggunaan-npm)
```

```
version: (1.0.0)
```

```
description: haaiii belajar npm dasar nih
```

```
entry point: (index.js)
```

```
test command:
```

```
git repository:
```

```
keywords:
```

```
author: Akbar Anggit Pambudi
```

```
license: (ISC)
```

```
About to write to C:\Users\LENOVO\Pictures\Pelatihan\1.Web_Development_Bootcamp\file 2
\8. Node.js\3. Penggunaan NPM\package.json:
```

```
{
  "name": "3.-penggunaan-npm",
  "version": "1.0.0",
  "description": "haaiii belajar npm dasar nih",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Akbar Anggit Pambudi",
  "license": "ISC"
}
```

```
Is this OK? (yes)
```

Setelah terinstal maka dalam folder akan ada file json artinya package npm telah terinstall di folder dan bisa menggunakan code npm orang

```
package.json
1  {
2    "name": "3.-penggunaan-npm",
3    "version": "1.0.0",
4    "description": "haaiii belajar npm dasar nih",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "author": "Akbar Anggit Pambudi",
10   "license": "ISC"
11 }
```

- Penggunaan Code NPM orang (Contoh Npm = Superheroes)

Buat folder dan install NPM pada folder yang akan digunakan seperti cara diatas

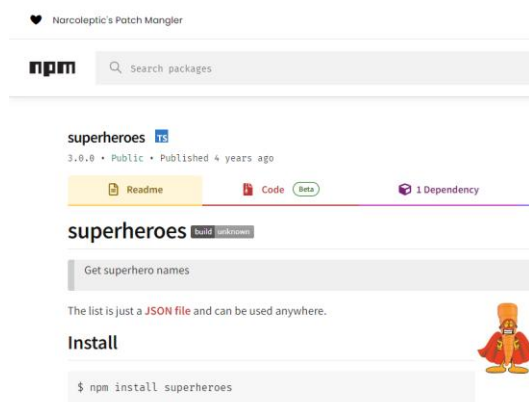
Hasil

```

1  {
2    "name": "4.-penerapan-code-npm-orang-lain",
3    "version": "1.0.0",
4    "description": "belajar penggunaan code npm orang lain",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \\"Error: no test specified\\" && exit 1"
8    },
9    "author": "Akbar Anggit Pambudi",
10   "license": "ISC"
11  }

```

Cari nama NPM yang ingin digunakan



install package NPM orang yang ingin digunakan

Ketik = npm install namaNPMorang

superheroes **TS**

3.0.0 • Public • Published 4 years ago

Pada npm diatas Namanya yang paling atas superheroes

```

LENOVO@DESKTOP-J5MV806 MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/
8. Node.js/4. penerapan code Npm orang lain
$ npm install superheroes

added 3 packages, and audited 4 packages in 7s

found 0 vulnerabilities

```

Hasilnya akan ada penambahan package npm

```

14  }
15  },
16  "dependencies": {
17    "superheroes": "^3.0.0"
18  },
19  "devDependencies": {
20    "typescript": "^4.0.0"
21  },
22  "scripts": {
23    "test": "echo \\"Error: no test specified\\" && exit 1"
24  },
25  "author": "Akbar Anggit Pambudi",
26  "license": "ISC"
27  }

```

Buat file.js untuk menggunakannya

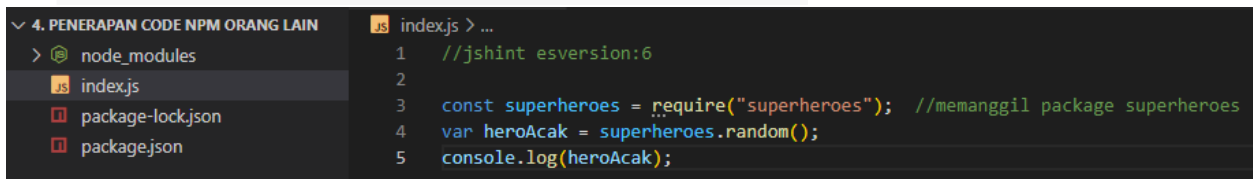
 npmjs.com/package/superheroes

Usage

```
const superheroes = require('superheroes');

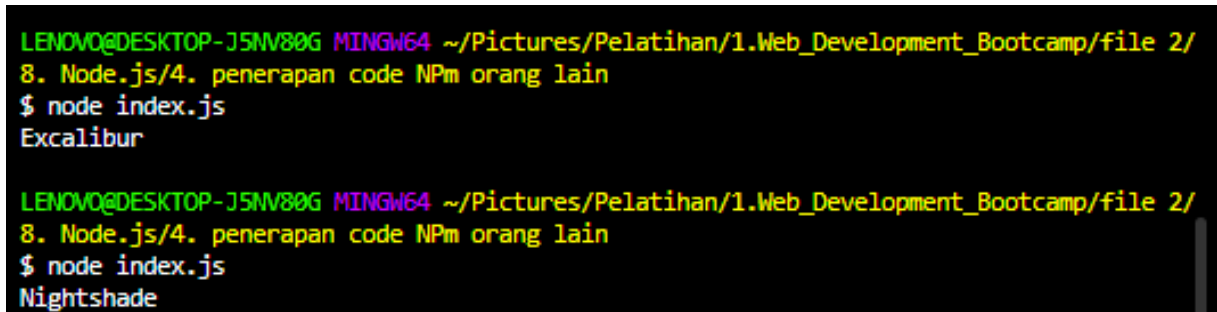
superheroes.all;
//=> ['3-D Man', 'A-Bomb', ...]

superheroes.random();
//=> 'Spider-Ham'
```



```
1 //jshint esversion:6
2
3 const superheroes = require("superheroes"); //memanggil package superheroes
4 var heroAcak = superheroes.random();
5 console.log(heroAcak);
```

jalankan menggunakan hyper (node namaFile.js)



```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/
8. Node.js/4. penerapan code Npm orang lain
$ node index.js
Excalibur

LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/
8. Node.js/4. penerapan code Npm orang lain
$ node index.js
Nightshade
```

EXPRESS.JS

Express.js merupakan library untuk node.js seperti bootstrap untuk css

- **Instalasi Express.js**

Pertama buat folder dan didalamnya buat file.js

Install package npm = npm init

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/  
9. Express.js dg node.js/1. instalasi express.js  
$ npm init
```

Install package express = npm install express

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/  
9. Express.js dg node.js/1. instalasi express.js  
$ npm install express  
  
added 57 packages, and audited 58 packages in 2s  
  
7 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities
```

Membuat require express pada file.js

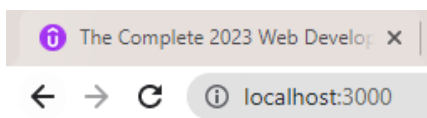
```
//jshint esversion:6  
  
const express = require("express"); // memanggil package express  
const app = express();  
  
app.get("/", function(req, res){ // membuat request dan response  
  res.send("<h1> hello world </h1>");  
});  
  
app.listen(3000, function(){ //membuat port server  
  console.log("server berhasil bejalan diport 3000") //jika dijalankan  
  tampilkan ini  
});
```

Jalankan file.js di hyper

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/  
9. Express.js dg node.js/1. instalasi express.js  
$ node index.js  
server berhasil bejalan diport 3000
```

Pada browser jalankan localhost dengan port 3000 = <http://localhost:3000/>

Hasil



hello world

- **Instalasi Nodemon (agar ketika jalankan file js di node restartnya otomatis)**

Bebas install dilokasi mana karena akan semua proyek terinstall

Pada hyper ketikan

```
npm install -g nodemon
```

Npm install – g nodemon

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/  
9. Express.js dg node.js/1. instalasi express.js  
$ npm install -g nodemon  
  
added 32 packages, and audited 33 packages in 7s  
  
3 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities
```

Cara menjalankan file.js = nodemon namaFile.js

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/  
9. Express.js dg node.js/1. instalasi express.js  
$ nodemon index.js  
[nodemon] 2.0.20  
[nodemon] to restart at any time, enter `rs`  
[nodemon] watching path(s): *.*  
[nodemon] watching extensions: js,mjs,json  
[nodemon] starting `node index.js`  
server berhasil bejalan diport 3000
```

Tambahan : Tekan rs untuk restart

- Route (localhost:3000/nama, localhost:3000/alamat)
Template express.js

```
//jshint esversion:6

const express = require("express"); // memanggil package express
const app = express();

app.get("/", function(req, res){ // membuat request dan response
  res.send("<h1> hello world </h1>");
});

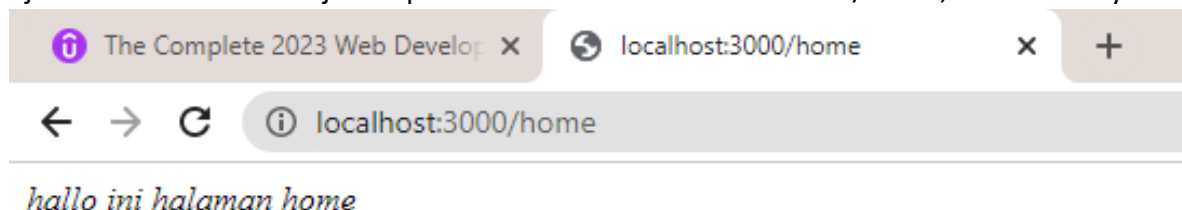
app.listen(3000, function(){ //membuat port server
  console.log("server berhasil bejalan diport 3000") //jika dijalankan
  tampilkan ini
});
```

Membuat rute baru

Rute = localhost:3000/home

```
JS index.js > app.get("/Home") callback
1 //jshint esversion:6
2
3 const express = require("express"); // memanggil package express
4 const app = express();
5
6 v app.get("/", function(req, res){ // membuat request dan response
7   |   res.send("<h1> hello world </h1>");
8   | };
9
10 // membuat rute localhost:3000/home
11 v app.get("/Home", function(req, res){ /// pada "/home" itu rutenya
12   |   res.send("<em> hallo ini halaman home <em>");
13   | };
14
15 v app.listen(3000, function(){ //membuat port server
16   |   console.log("server berhasil bejalan diport 3000") //jika dijalankan tampilkan ini
17   | };
```

#jalankan nodemon file.js dan pada browser ketik localhost:3000/home, maka hasilnya



- **Post, Get Dan Body-Parser (dalam kasus kalkulator)**

Buat folder, file.js, file.html, install package npm, install package express.

Copy template express.js

File html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Calculator</title>
</head>
<body>
  <form action="/" method="post">
    <input type="text" name="angka1" placeholder="masukan angka pertama">
    <input type="text" name="angka2" placeholder="masukan angka kedua">
    <button type="submit" name="tombol" >Hitung</button>
  </form>
</body>
</html>
```

File javascript

```
//jshint esversion:6

const express = require("express"); // gunakan package express
const bodyParser = require("body-parser");
// body-parser = modul nodejs yang digunakan untuk mengambil data dari form pada
framework Express

const app = express();

app.use(bodyParser.urlencoded({extended:true})); // gunakan body-parser

app.get("/", function(req,res){ // "/" mengacu pada action file html
  res.sendFile(__dirname + "/index.html" ); //mengambil file untuk ditampilkan
  //__dirname = lokasi folder
  // /index.html = nama filenya
});
```

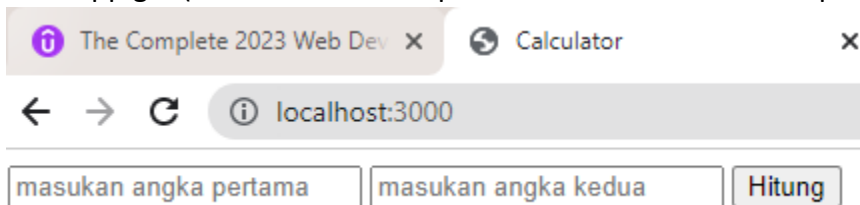
```

app.post("/",function(req, res){  /// ketika diklik tombol hitung maka akan
merespon
    // "/" mengacu pada action file html
    //disini fungsi bodyparser, mengambil data dari form html (req.body.nama)
    var num1 = Number(req.body.angka1);
    // number berfungsi agar memastikan data yang diambil berbentuk angka
    //req.body.angka1 =mengambil angka yang dimasukan dlm input yang mempunyai
nama angka1 di bagian body html
    var num2 = Number(req.body.angka2);
    var hitung = num1 + num2;
    res.send("hasil tambnya adalah " + hitung); // merespon hasilnya perhitungan
});

app.listen(3000, function(){  //port server bebas bisa 4000, 3983
    console.log("jika server berjalan katakan yes");
});

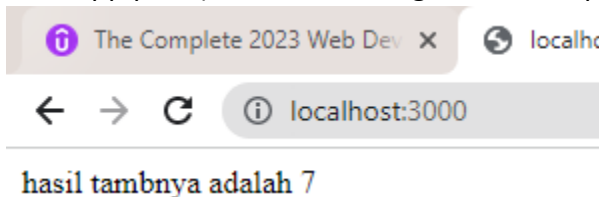
```

Hasil app.get (maka akan menampilkan file html form method post)



The screenshot shows a web browser window with two tabs: 'The Complete 2023 Web Dev' and 'Calculator'. The address bar shows 'localhost:3000'. The page content consists of two text input fields labeled 'masukan angka pertama' and 'masukan angka kedua', followed by a button labeled 'Hitung'.

Hasil app.post (ketika klik hitung akan merespon)



The screenshot shows the same web browser window as before, but the page content now displays the text 'hasil tambnya adalah 7' in a monospace font.

- **Latihan Membuat Kalkulator BMI**

Buat folder, file.js, file.html, install package npm, install package express.

Copy template express.js

File html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Kalkulator BMI</title>
</head>
<body>
  <form action="/kalkulatorBMI" method="post">
    <h1>Kalkulator Berat Badan Ideal</h1>
    <br>
    <input type="number" name="berat" placeholder="Masukan berat badan anda">
    <input type="number" name="tinggi" placeholder="Masukan tinggi badan
anda">
    <button type="submit">Hitung</button>
  </form>
</body>
</html>
```

File Javascript

```
//jshint esversion:6

const express = require("express");

const bodyParser = require("body-parser");
const app = express();
app.use(bodyParser.urlencoded({extended:true}));

app.get("/kalkulatorBMI", function(req, res){
  res.sendFile( __dirname + "/index.html" );
});

app.post("/kalkulatorBMI", function(req,res){
  var angka1 = req.body.berat;
  var angka2 = req.body.tinggi;
  var hasil = angka1/(Math.pow(angka2,2)) ;
  res.send("hasilnya adalah " + hasil);
})
```

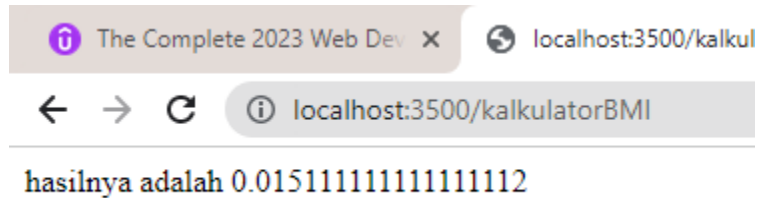
```
app.listen(3500, function(){  
  console.log("server 3500 berjalan jawan yes");  
});
```

Hasil app.get (akan menampilkan file html form post



The screenshot shows a web browser with two tabs: "The Complete 2023 Web Dev" and "Kalkulator BMI". The address bar shows "localhost:3500/kalkulatorBMI". The page title is "Kalkulator Berat Badan Ideal". Below the title, there is a form with three input fields: "Masukan berat badan and", "Masukan tinggi badan anc", and a "Hitung" button.

App.post (ketika klik hitung maka akan menampilkan)



The screenshot shows the same web browser as before, but the address bar now shows "localhost:3500/kalkul". The page content displays the result: "hasilnya adalah 0.01511111111111112".

API (Application Programming Interface)

API = Sekumpulan command, fungsi, protocol dan objek yang dapat digunakan programmer untuk membuat perangkat lunak yang berinteraksi dengan sistem eksternal

Contoh API :

<https://v2.jokeapi.dev/joke/Programming?contains=debugger>
<https://v2.jokeapi.dev/joke/Programming?contains=debugger>

Ket :

Merah : bagian warna merah adalah endpoint

Biru : bagian biru adalah parameter

Ungu : bagian ungu adalah path/detail (ditandai dengan tanda tanya(?))

- **Contoh menerapkan Api cuaca**

Link = <https://openweathermap.org/current>

Cara pemanggilan API

Built-in API request by city name

You can call by city name or city name, state code and country code. Please note that searching by states available only for the USA locations.

API call

```
https://api.openweathermap.org/data/2.5/weather?q={city  
name}&appid={API key}
```

```
https://api.openweathermap.org/data/2.5/weather?q={city  
name},{country code}&appid={API key}
```

```
https://api.openweathermap.org/data/2.5/weather?q={city  
name},{state code},{country code}&appid={API key}
```

API key pribadi

Key	Name	Status	Actions	Create key
d9bb052530df9fa47ff6c77ce50e62e9	Default	Active	 	API key name <button>Generate</button>

Maka, jadinya

<https://api.openweathermap.org/data/2.5/weather?q=Paris&appid=d9bb052530df9fa47ff6c77ce50e62e9#>

Hasil

```
{  
  "coord": {  
    "lon": 2.3488,  
    "lat": 48.8534  
  },  
  "weather": [  
    {  
      "id": 701,  
      "main": "Mist",  
      "description": "mist",  
      "icon": "50n"  
    }  
  ]  
}
```

- **Penggunaan API Dengan Postman**

Copy endpoint API wether

<https://api.openweathermap.org/data/2.5/weather>

GET `https://api.openweathermap.org/data/2.5/weather` Save Send

Params • Authorization Headers (8) Body • Pre-request Script Tests Settings Cookies

Query Params

	KEY	VALUE	DESCRIPTION	...	Bulk Edit
--	-----	-------	-------------	-----	-----------

Masukan parameter dan API key

GET `https://api.openweathermap.org/data/2.5/weather?q=Indonesia&appid=d9bb052530df9fa47ff6c77ce50e62e9` Save Send

Params • Authorization Headers (8) Body • Pre-request Script Tests Settings Cookies

Query Params

	KEY	VALUE	DESCRIPTION	...	Bulk Edit
☒	q	Indonesia			
☒	appid	d9bb052530df9fa47ff6c77ce50e62e9			

Maka otomatis link api dengan api key kita langsung jadi, dan ketika diklik send langsung muncul

Body Cookies Headers (9) Test Results Status: 200 OK Time: 78 ms Size: 840 B Save Response

Pretty Raw Preview Visualize JSON Copy Search

```

1  {
2    "coord": {
3      "lon": 120,
4      "lat": -5
5    },
6    "weather": [
7      {
8        "id": 804,
9        "main": "Clouds",

```


- **Praktek API didalam node.js html**

Buat folder, file.js, file.html, install package npm, install package express.

Copy tamplate express.js

File.js

```
const express = require("express");
const https = require("https"); //memanggil link external
const bodyParser = require("body-parser");

const app = express();

app.use(bodyParser.urlencoded({extended:true}));

app.get("/", function(req, res){
  res.sendFile(__dirname + "/index.html");
});

app.post("/", function(req, res){
  const daerah = req.body.kota;
  //pengunaan body-parser untuk mengambil input dengan nama "kota"
  const kunci = "d9bb052530df9fa47ff6c77ce50e62e9";
  const link = "https://api.openweathermap.org/data/2.5/weather?q=" + daerah +
"&appid=" + kunci + "&units=metric";
  https.get(link, function(response){ // memanggil link api
    console.log(response.statusCode);
    //mengecek status kode apakah sudah baik

    response.on("data", function(data){ // memanggil data
      const cuaca = JSON.parse(data);
      // JSON.parse mengubah data binary jadi bentuk javascript
      const temperatur = cuaca.main.temp;
      // mengambil data temperatur saja
      const keteranganCuaca = cuaca.weather[0].description;
      //mengambil data deskripsi saja
      const icon = cuaca.weather[0].icon;
      const img = "http://openweathermap.org/img/wn/" + icon + "@2x.png";

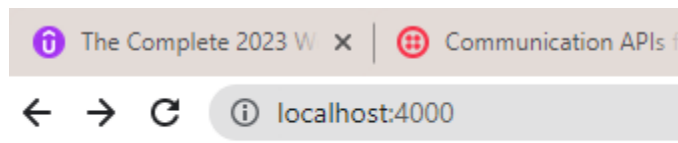
      res.write("<p>keadaan awan : " + keteranganCuaca+"</p>");
      res.write("<h1>cuaca hari ini di " + daerah + " adalah " + temperatur
+ " celcius</h1>");
      res.write("");
      res.send();
    });
  });
});
```

```
});
app.listen(4000, function(){
  console.log("server berjalan pada port 4000 dan berhasil");
});
```

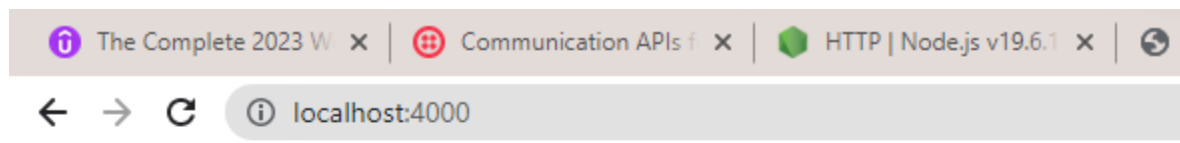
File html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cuaca</title>
</head>
<body>
  <form action="/" method="post">
    <label for="namaKota">Nama Kota : </label>
    <input id="kotaInput" type="text" name="kota">
    <button type="submit">Submit</button>
  </form>
</body>
</html>
```

Hasil



Nama Kota :



keadaan awan : overcast clouds

cuaca hari ini di bali adalah 30.13 celcius



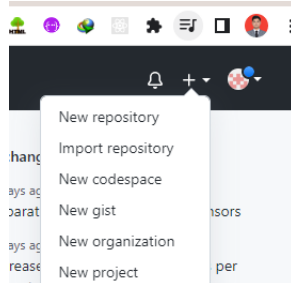
GIT & GITHUB

- Command terminal Git**

Command	Keterangan
git --help	Referensi kumpulan command
open file1.txt	Membuka file di notepad
vim file1.txt	Membuka file di terminal
git init	Melacak folder berada (rute) dan masuk branch master
git status	Mengecek staging area (kalau merah berarti file hanya pada folder). Jika (hijau berarti file siap dipost)
git add namafile (git add file1.txt) git add .	Menambahkan ke staging area. git add . = menambahkan semua file dalam folder tersebut kedalam staging area
git rm --cached -r .	Membatalkan semua file yang sudah masuk git add
git commit -m "deskripsi terserah"	digunakan untuk menyimpan perubahan yang sudah dilakukan, namun tidak ada perubahan yang terjadi pada remote repository
git log	untuk melihat catatan log perubahan pada repository.
git diff namaFile (git diff file1.txt)	Mengecek perubahan yang dilakukan dengan sebelumnya
git checkout namaFile (git checkout file1.txt)	Membatalkan perubahan
git remote add origin linkRepository	Membuat remote github
git push -u origin master	Origin = nama remote master = nama branch
Git clone linkRepository	Untuk download file git hub (bisa langsung download juga)

- Membuat Repository GitHub**

1. pertama tombol + , kemudian new repository



2. Berikan nama repository dan kemudian klik create

3. Selesai

- **Remote Git Dan GitHub**

0. Git init dulu

1. membuat remote git = git remote add origin linkRepository

Contoh linkRepository

Quick setup — if you've done this kind of thing before

 Set up in Desktop or **HTTPS** **SSH** <https://github.com/Akbaranggit/membuat-folder-repository.git>

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/
11. git dan github/1. dasar (master)
$ git remote add origin https://github.com/Akbaranggit/membuat-folder-repository.git
```

2. push kerepository github sebagai branch master

```
11. git dan github/1. dasar (master)
$ git push -u origin master
```

- **Membuat file .gitignore (untuk menyembunyikan file yang diinginkan)**

Membuat file file1, file2, rahasia, .gitignore

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/
11. git dan github/2. gitignore
$ touch file1.txt file2.txt rahasia.txt .gitignore
```

Cek = Git status

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/
11. git dan github/2. gitignore (master)
$ git status
On branch master

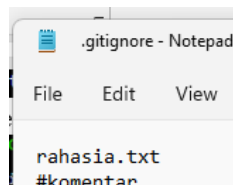
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitignore
    file1.txt
    file2.txt
    rahasia.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Sembunyikan file rahasia.txt, dengan cara memasukan ke .gitignore

Tuliskan nama file rahasia.txt didalam .ignore



Cek = git status

```
11. git dan github/2. gitignore (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitignore
    file1.txt
    file2.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Maka ketika dicek file rahasia.txt telah disembunyikan

- **Branching Dan Merging**

branching = membuat branch cabang

1. membuat branch baru = git branch namaBranch

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/
11. git dan github/1. dasar (master)
$ git branch cabang
```

2. mengecek branch yang dibuat = git branch

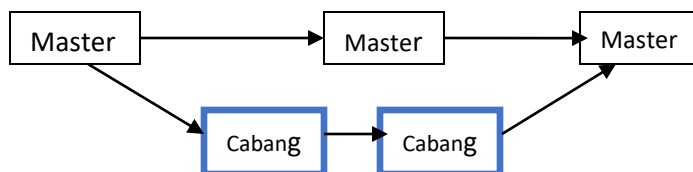
```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/
11. git dan github/1. dasar (master)
$ git branch
  cabang
* master
```

3. berpindah branch = git checkout namaBranch

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/
11. git dan github/1. dasar (master)
$ git checkout cabang
Switched to branch 'cabang'

LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/
11. git dan github/1. dasar (cabang)
$ git branch
* cabang
  master
```

Merging



Ket

Membuat repository digithub

Git remote add origin linkRepository

Git touch file1.txt file2.txt = membuat file1 dan file2

Git add .

Git commit -m "upload "

Git push origin master -u

Kemudian buat cabang branch

Git branch cabang

Git checkout cabang = untuk switch ke branch cabang

Lalu pada folder edit isi file

Git add .

Git commit -m "edit fil dari cabang"

Kemudian pindah ke branch master lagi

Git checkout master

Git touch file3.txt

Git add .

Git commit -m "upload file 3"

Git push origin master -u

Git merge cabang = menggabungkan apa yang diedit oleh branch cabang

Tekan :q = untuk keluar

Lalu git push origin master jadilah yang diatas

EJS (Embedded JavaScript Templating)

- **Latihan jika hari ini libur dan kerja**

Buat folder, buat file js, html, install npm init, npm install express, npm install body-parser
File javascript

```
const express = require("express");
const bodyParser = require("body-parser");

const app = express();
app.use(bodyParser.urlencoded({extended:true}));

app.get("/", function(req, res){
  const hari = new Date();
  const hitung = hari.getDate();

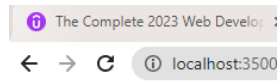
  if( hitung === 6 || hitung === 0){
    res.write("hari ini libur");
    res.write("horeee");
    res.send();
  }else{
    res.sendFile(__dirname + "/index.html");
  }
});

app.listen(3500, function(){
  console.log("server beehasil berjalan diport 3500");
});
```

File html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>libur</title>
</head>
<body>
  <h1>Belajarrrrrrrrrr</h1>
  <p>horeeeeeeeeeee</p>
  <br>
  <h1>Belajarrrrrrrrrr</h1>
  <p>horeeeeeeeeeee</p>
</body>
</html>
```

Hasil



Belajarrrrrrrrrr

horeeeeeeeeeee

Belajarrrrrrrrrr

horeeeeeeeeeee

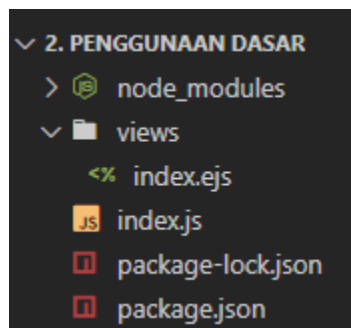
- **Install EJS Package Dan Penggunaan Ejs (npm install ejs)**

1. file ejs sama dengan file html

2. file ejs wajib disimpan di folder "views"

Buat folder dan di dalamnya buat folder views, buat file js, file ejs pada folder viewa, install npm init, npm install express, npm install body-parser, npm install ejs

Folder views



File js

```
const express = require("express");
const bodyParser = require("body-parser");

const app = express();

app.set("view engine", "ejs"); // mengatur ejs

app.get("/", function(req, res){
  const hari = new Date();
  const sekarang = hari.getDay();
  var ubah = "";
  if(sekarang === 6 || sekarang === 0){
    ubah= "liburan";
  }else{
    ubah = "belajar";
  }
  res.render("index", {dina: ubah});
});
```



```

    // index = file ejs
    //dina itu nama variabel di ejs
    //ubah itu nama varibel di js
  });

app.listen(3000, function(){
  console.log("server berjalan diport 3000");
});

```

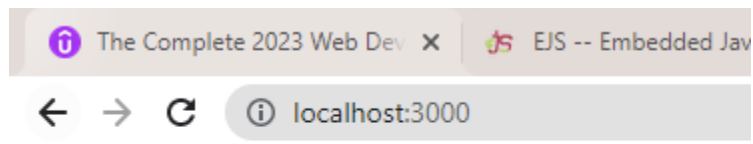
File ejs

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>belajar dasar ejs</title>
</head>
<body>
  <% if(dina === "belajar"){ %>
    <h1 style="color:green"> Hari ini waktunya <%= dina %> </h1>
  <!-- dina = nama variabel -->
  <% }else{ %>
    <h1 style="color:red"> Hari ini waktunya <%= dina %> </h1>
  <!-- dina = nama variabel -->
  <% } %>
  <!-- code selain html / code javascript wajib dikasih tanda diatas pada file ejs
  -->
</body>
</html>

```

Hasil

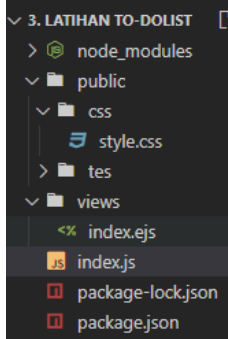


Hari ini waktunya belajar

- **Latihan membuat to do list**

1. file ejs sama dengan file html
2. file ejs wajib disimpan di folder "views"
3. file css wajib disimpan di folder "public"

Buat folder dan didalamnya buat folder views, buat file js, file ejs pada folder viewa, install npm init, npm install express, npm install body-parser, npm install ejs



File ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>too do list</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1><%= tanggal %></h1>
  <br>
  <h2>makanan favorit</h2>
  <ul>
    <li>mie ayam</li>
    <li>bakso</li>
    <!-- melakukan looping agar list bertambah kebawah, bukan dalam 1 list -->
    <% for(var i = 0; i<tambahlist.length; i++){ %>
      <li><%= tambahlist[i] %></li>
    <%} %>
  </ul>
  <form action="/" method="post">
    <label for="namaList">Nama List Baru</label>
    <input type="text" name="listBaru">
    <button type="submit">Tambah</button>
  </form>
</body>
</html>
```

File js

```
const express = require("express");
const bodyParser = require("body-parser");

const app = express();
app.use(bodyParser.urlencoded({extended:true}));

app.use(express.static("public")); //untuk membaca css
//css wajib dimasukan folder "public" agar kebaca

app.set("view engine", "ejs"); // mengatur penggunaan ejs

var itemList=[]; //variabel untuk menampung data list

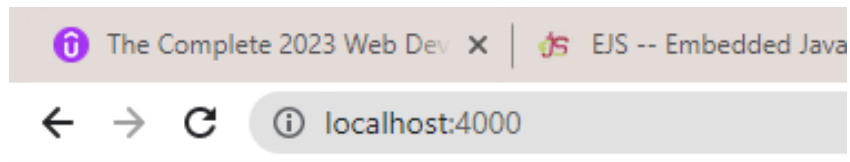
app.get("/", function(req, res){
  var waktu = new Date();
  var atur = {
    weekday: "long",
    day: "numeric",
    month: "long"
  }
  var jadi = waktu.toLocaleDateString("en-US", atur);

  res.render("index",{tanggal: jadi, tambahlist: itemList});
  // tanggal:jadi = "tanggal" mengambil nama variabel ejs
  //"jadi" mengambil nama variabel di js
  //tambahlist: itemList = "tambahlist" mengambli nama variabel list di ejs
  //"itemList" mengambil nama variabel penampung array di js
});

app.post("/", function(req, res){
  var item = req.body.listBaru; //mengambil data input di body dengan nama
listBaru diejs
  itemList.push(item); //data yang diambil dipush ke itemList
  res.redirect("/"); // merespon kembali ke rute "/"
});

app.listen(4000, function(){
  console.log("server berjalan di port 4000");
});
```

Hasil



Tuesday, February 21

makanan favorit

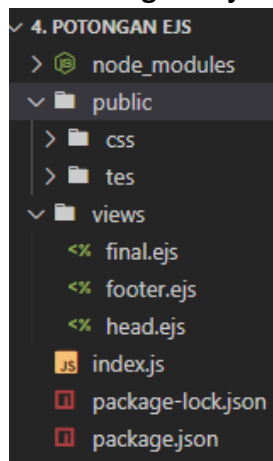
- mie ayam
- bakso
- ayam goreng
- spaghetti

Nama List Baru

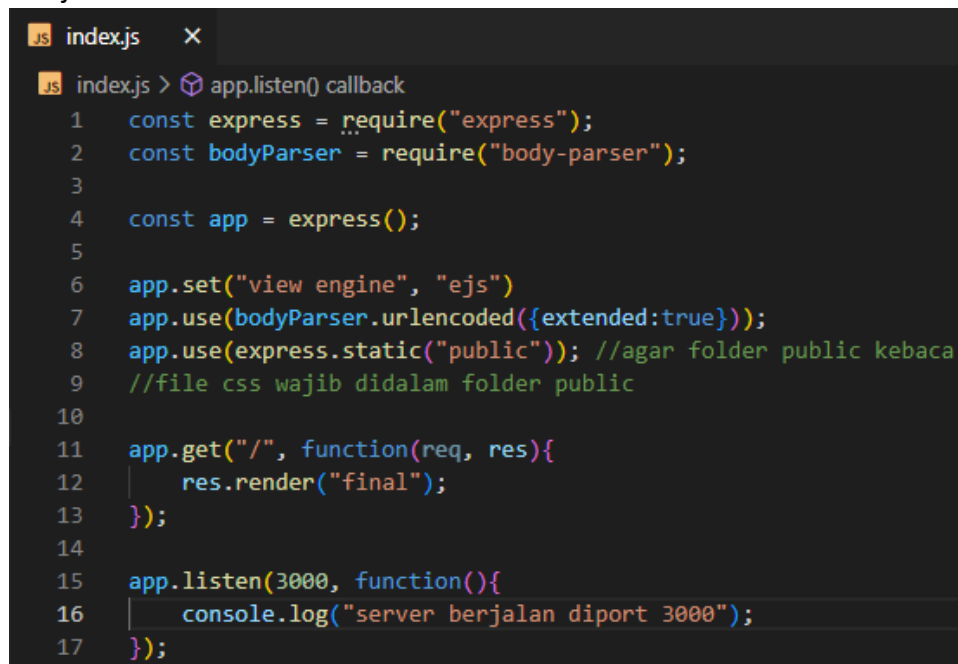
- Perbedaan var, let, const

Perbedaan				
Jenis	var	let	const	keterangan
nilai	Dapat dirubah	Dapat dirubah	Tidak dapat dirubah	Jika const a = 1; maka tidak akan bisa dirubah
Lingkup for, while	Global	local	local	Global = bisa di panggil diluar lingkup local = tidak bisa dipanggil diluar lingkup

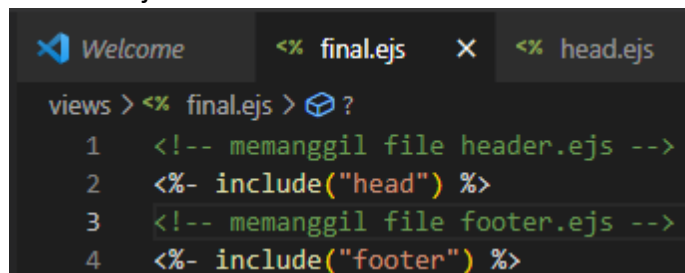
- Memotong file Ejs menjadi beberapa bagian <%- include(namaFileEjs) -%>



File js



File final.ejs



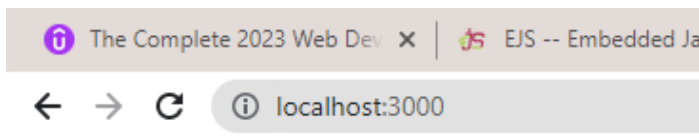
File head.ejs

```
<% head.ejs X
views > <% head.ejs > html > body
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta http-equiv="X-UA-Compatible" content="IE=edge">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Document</title>
8 </head>
9 <body>
10  <h1>halooooooooo ini badan</h1>
11 </body>
```

File footer.ejs

```
<% footer.ejs X
views > <% footer.ejs > ...
1 
2 <footer>
3   <p>hallo ini footer</p>
4 </footer>
5 </html>
```

Hasil



halooooooooo ini badan

hallo ini footer

- **Lodash (low case / upper case)**

Install:

npm install lodash

Const _ = require("lodash");

Contoh penggunaan

_.lowerCase(req.params.title);

- **Potong text (substring(1,3))**

Var sub = "helloword";

Console.log(substring(1,3));

Hasil = ell

Karena mengambil karakter index 1 sampe 3

DATABASE SQL (Structured Query Language)

- Pengenalan database (SQL VS NON SQL)

Perbedaan		
Jenis	SQL	NON SQL
Kepanjangan	Structured Query Language	Non Structued Query Language
Contoh Database	MySQL, PostgreSQL	MongoDB, Redis
Struktur	(Tabel) SQL mengelompokan databasenya pada tiap table. misal: table pembeli, table produk	(Dokumen) Non SQL menggunakan sintax seperti JSon. Misal: {id:123, nama: "budi", umur: 15} {id: 345, nama: "anggit", umur:26}
Fleksibilitas	Kurang flesibel, karena dalam satu table isinya sama Misal : id, nama, umur alamat	Lebih fleksibel, karna setiap database bisa beda-beda isinya. Misal: {id: 243, nama: "adi", umur:15, status: "jumbo"} Yang kedua bisa beda {id:235, nama: "Jo", umur:23}
Rasional (hubungan antar data)	Lebih mudah dihubungkan dengan data lain	Lebih sulit dihubungkan dengan data lain
Kekuatan untuk data yang besar	Tidak baik untuk data yang sangat besar	Baik untuk data yang sangat besar

- Membuat Database, Table, Primary key (CREATE)

Membuat database

```
CREATE DATABASE databasename;
```

Membuat table dan primary key

```
1 CREATE TABLE pelanggan(  
2 id int NOT NULL,  
3 nama string,  
4 modal money,  
5 PRIMARY KEY (id)  
6 );
```

Keterangan :

Pelanggan = nama table

Id, nama, modal = nama kolom table

Int, string, money = tipe data

Not null = id tidak boleh kosong

Primary key (id) = id jadikan untuk menghubungkan dan tidak boleh sama

- Menambahkan data kedalam table yang telah dibuat (INSERT INTO)

Versi 1

```
1 INSERT INTO pelanggan
2 VALUES (1, "budi", 1.20);
```

Versi 2

```
1 INSERT INTO pelanggan(id, nama)
2 VALUES (2, "anggres");
```

- Menampilkan Database (SELECT)

Tampilkan semua isi table

```
1 SELECT * FROM pelanggan;
```

Hasil

id	nama	modal
1	budi	1.2
2	anggres	NULL

Tampilkan kolom tertentu pada table

```
1 SELECT nama FROM pelanggan;
```

Hasil

nama
budi
anggres

```
1 SELECT nama, modal FROM pelanggan;
```

Hasil

nama	modal
budi	1.2
anggres	NULL

- Menampilkan Database dengan Kondisi (Select dan Where)

```
1 SELECT * FROM pelanggan WHERE id = 1;
```

Hasil

id	nama	modal
1	budi	1.2

- Menyisipkan data tertentu pada table (Update)

```
1 UPDATE pelanggan
2 SET modal = 5.30
3 WHERE id = 2;
```

Hasil

id	nama	modal
1	budi	1.2
2	anggres	5.3

- Menambahkan nama kolom pada table yang telah dibuat (ALTER TABLE)

```
1 ALTER TABLE pelanggan ADD alamat varchar;
```

Keterangan:

Pelanggan = nama table

Alamat = nama kolom yang ingin ditambahkan

Varchar = tipe data yang ingin digunakan pada kolom alamat

Hasil

id	nama	modal	alamat
1	budi	1.2	NULL
2	anggres	5.3	NULL

- Menghapus data pada table (DELETE)

Hapus data yang mempunyai id = 2;

```
1 DELETE FROM pelanggan WHERE id = 2;
```

Hasil

id	nama	modal	alamat
1	budi	1.2	NULL

- **Relationship foreign key (hubungan antar tabel) dan inner join**

Foreign Key

Table pelanggan

Id_pelanggan (pk)	Nama	alamat
id_pelanggan	nama	alamat
1	ardi	cilacap
2	anggit	cilacap
3	inda	bandung

Table produk

Id_produk (PK)	Nama_produk	stok
id_produk	nama_produk	stok
12	sendal	34
23	sepatu	54
33	buku	100

Tabel pesen

Id_pesen (PK)	Id_pelanggan (foreign key)	Id_produk (foreign key)
---------------	----------------------------	-------------------------

Buat table order dan hubunganya

```

1 CREATE TABLE pesen (
2   id_pesen int NOT NULL,
3   id_pelanggan int,
4   id_produk int,
5   PRIMARY KEY (id_pesen),
6   FOREIGN KEY (id_pelanggan) REFERENCES pelanggan(id_pelanggan),
7   FOREIGN KEY (id_produk) REFERENCES produk(id_produk)
8 );

```

Foreign key (id_pelanggan dan id_produk) = ambil dari kolom yang dimasukan di table pesen

Reference pelanggan dan produk = nama table yang dihubungkan

(id_pelanggan dan id_produk yang belakang) = nama primary key dari masing2 produk

Buat isi order id_pesen = 55, id_pelanggan = 2, id_produk = 33

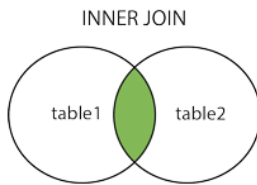
```

1 INSERT INTO pesen
2 VALUES (55, 2, 33)

```

id_pesen	id_pelanggan	id_produk
55	2	33

Inner join (antar table pelanggan dan table pesen)



Menampilkan id_pesen, nama pelanggan, alamat pelanggan yang ada di table pesen

```
1 SELECT pesen.id_pesen, pelanggan.nama, pelanggan.alamat
2 FROM pesen
3 INNER JOIN pelanggan ON pesen.id_pelanggan = pelanggan.id_pelanggan;
```

Keterangan :

Pesen.id_pesen = id_pesen dalam table pesen

Pelanggan.nama dan pelanggan.alamat = nama dan alamat dalam table pelanggan

From pesen = dari table pesen (hanya nama dan alamat pelanggan yang idnya ada di table pesen)

Inner jon pelanggan = join dengan table pelanggan

On pesen.id_pelanggan = pelanggan.id_pelanggan ==== dimana id_pelanggan pada table pesen sama dengan id_pelanggan di table pelanggan

id_pesen	nama	alamat
55	anggit	cilacap

DATABASE MONGODB (NON SQL)

- **CRUD MongoDB**

Doc MongoDB CRUD = <https://www.mongodb.com/docs/manual/crud/>

Jalankan mongod

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~  
$ mongod
```

Buka tab baru jalankan mongo

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~  
$ mongo
```

Kumpulan command mongo (help)

```
> help  
db.help()           help on db methods  
db.mycoll.help()    help on collection methods  
sh.help()           sharding helpers  
rs.help()           replica set helpers  
help admin          administrative help  
help connect        connecting to a db help  
help keys           key shortcuts  
help misc           misc things to know  
help mr             mapreduce
```

Menampilkan database yang ada (show dbs)

```
> show dbs  
admin      0.000GB  
blogDB     0.000GB  
buatdatabase 0.000GB  
config     0.000GB  
indexDB    0.000GB  
local      0.000GB
```

Melihat sekarang diposisi mana (db)

```
> db  
proyedDB
```

Melihat nama-nama kolom database (show collections)

```
> show collections  
artikels
```

Membuat Database dan switch database (use namaDatabase)

```
> use namaAja  
switched to db namaAja
```

Cat = jika database belum ada isi maka di show collections tidak akan muncul

Insert db.namaKolom.insertOne() Dan db.namaKolom.insertMany())

Insert = Membuat nama kolom database dan menambahkan isi kolom tersebut

Contoh insertOne

```
> db.produk.insertOne({id:12, nama:"buku", stok:32})
{
  "acknowledged" : true,
  "insertedId" : ObjectId("63f88d86e31396e0976e0082")
}
```

Contoh insertMany

```
db.produk.insertMany(
  [
    {id:235, nama:"buku", stok:23},
    {id:234, nama:"pulpen", stok:25}
  ]
)
```

Read 1 (db.namaKolomYangAda.find())

Contoh 1

```
> db
namaAja
> show collections
produk
> db.produk.find()
{ "_id" : ObjectId("63f88d86e31396e0976e0082"), "id" : 12, "nama" : "buku", "stok" : 32 }
2 }
```

Contoh 2

```
> db.produk.find({nama:"buku"})
{ "_id" : ObjectId("63f88d86e31396e0976e0082"), "id" : 12, "nama" : "buku", "stok" : 32 }
2 }
```

Read 2 (db.namaKolomYang Ada.find(query))

Doc query (\$gt, \$gte..) =

<https://www.mongodb.com/docs/manual/reference/operator/query/>

Contoh query \$gt = paling besar

```
> db.produk.find({stok: {$gt:20}})
{ "_id" : ObjectId("63f88d86e31396e0976e0082"), "id" : 12, "nama" : "buku", "stok" : 32 }
```

Update (db.namaKolom.updateOne() dan db.namaKolom.updateMany())

Update merubah data pada table

```
> db.produk.updateOne({id:12}, {$set: {stok:50}})
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
```

Delete (db.namaKolom.deleteOne() Dan db.namaKolom.deleteMany())

Delete = menghapus data tertentu

```
> db.produk.deleteOne({id:12})
{ "acknowledged" : true, "deletedCount" : 1 }
```

Drop hapus database (db.dropDatabase ())

1.use namaDatabase

2. db.dropDatabase()

```
> use wikiAnggit
switched to db wikiAnggit
> db.dropDatabase()
{ "ok" : 1 }
```

- **Relationship mongoDB**

```
db.produk.insert(
  {
    Id:123,
    nama:"buku",
    stok:34,
    review: [
      {
        pelanggan: "akbar",
        skor:5
      },
      {
        Pelanggan:"budi",
        Skor:4
      }
    ]
  }
)
```

- **MongoDB with Node js (versi manual)**

Link = <https://www.mongodb.com/docs/drivers/>

Masih masalah

PENGUNAAN MONGODB DENGAN MOONGOSE DALAM NODE JS

- Instalasi Mongoose Dan Tamplate

Pada hyper

1. Install package npm = npm init
2. Install mongodb = npm install mongodb
3. Install mongoose = npm install mongoose

File js

```
//jshint esversion:6
const mongoose = require("mongoose"); //memanggil package mongoose

//mengubungkan mongoose dengan mongodb
mongoose.connect("mongodb://127.0.0.1:27017/tokoKasur", {useNewUrlParser:true});
// tokoKasur = nama database yang akan dibuat

//membuat skema kolom dari database tokoKasur
const pelangganSkema = new mongoose.Schema(
  {
    nama: String,
    umur: Number,
    alamat: String
  }
);

//menghubungkan skema dan membuat nama kolom
const buatkolom1 = mongoose.model("pelanggan", pelangganSkema);
// pelanggan = nama kolom yang akan dibuat

//menambahkan data isi kolom yang dibuat dengan format skema yang dibuat
const isi1 = new buatkolom1(
  {
    nama: "akbar",
    umur: 26,
    alamat: "jalan cinyawang no 1"
  }
);
```

```

//simpan data isi kolom ketika dijalankans

// isi1.save(function(err){
//     if(err) {
//         console.log("gagal menyimpan");
//     }else{
//         console.log("berhasil menyimpan");
//     }
// });

// jika sudah jalankan sekali, jangan jalankan save lagi data akan double
//tanpa function juga bisa contoh isi1.save();

//***** (Insert (menambahkan data pada kolom yang sudah dibuat)) *****/
const isi2 = new buatkolom1(
    {
        nama: "kasun",
        umur: 45,
        alamat: "patimuan"
    }
);
//data 2
const isi3 = new buatkolom1(
    {
        nama: "kasar",
        umur: 35,
        alamat: "cilacap"
    }
);
//data3
const isi4 = new buatkolom1(
    {
        nama: "anyar",
        umur: 20,
        alamat: "ciled"
    }
);
//melakukan insert many karena data yang ditambahkan lebih dari 1
buatkolom1.insertMany([isi2, isi3, isi4], function(err){
    if (err){
        console.log("gagal menambahkan");
    }else{
        console.log("berhasil menambahkan");
    }
});

```


Hasil

```
> db.pelanggans.find()
{ "_id" : ObjectId("63f9cf051e77fd86272f8bd4"), "nama" : "akbar", "umur" : 26, "alamat" : "jalan cinyawang no 1", "__v" : 0 }
{ "_id" : ObjectId("63f9d3f12415a52d94837302"), "nama" : "kasun", "umur" : 45, "alamat" : "patimuan", "__v" : 0 }
{ "_id" : ObjectId("63f9d3f12415a52d94837303"), "nama" : "kasar", "umur" : 35, "alamat" : "cilacap", "__v" : 0 }
{ "_id" : ObjectId("63f9d3f12415a52d94837304"), "nama" : "anyar", "umur" : 20, "alamat" : "ciled", "__v" : 0 }
```

- Read mongoose di node js (find)

```
//***** READ *****/
// buatkolom1 ambil dari const buatkolom1 = mongoose.model("pelanggan", pelangganSkema);
buatkolom1.find(function(err, tampilkanHasil){
  if(err){
    console.log(err);
  }else{
    console.log(tampilkanHasil);
  }
});
```

Hasil

```
[
  {
    _id: new ObjectId("63f9cf051e77fd86272f8bd4"),
    nama: 'akbar',
    umur: 26,
    alamat: 'jalan cinyawang no 1',
    __v: 0
  },
  {
    _id: new ObjectId("63f9d3f12415a52d94837302"),
    nama: 'kasun',
    umur: 45,
    alamat: 'patimuan',
    __v: 0
  },
  {
    _id: new ObjectId("63f9d3f12415a52d94837303"),
```

```
// find berdasarkan nama
// buatkolom1 ambil dari const buatkolom1 = mongoose.model("pelanggan", pelangganSkema);
buatkolom1.find(function(err, tampilkanHasil){
  if(err){
    console.log(err);
  }else{
    tampilkanHasil.forEach(function(buatkolom1){ //foreach agar tampil kebawah
      console.log(buatkolom1.nama); //tampilkan hanya nama dari buat kolom1
    });
  }
});
```

Hasil

```
akbar
kasun
kasar
anyar
```

- Validasi Skema

```
const pelangganSkema = new mongoose.Schema(
  {
    nama: { //validasi
      type: String, //tipe = string
      require: [true,"masukan nama"]
      //true / bisa ketik 1
      //"masukan nama" = jika yang diinputkan tidak membeikan nama ma akan keluar peringatan ini
    },
    umur: {
      type: Number, // tipe = number
      min: 1, //minimal umur = 1
      max: 60 // maksimal umur = 60
    },
    alamat: String
  }
);
```

- Update (Merubah Data Tertentu)

```
//***** Update *****/
buatkolom1.updateOne({id:"63f9cf051e77fd86272f8bd4"}, {nama:"anggit"}, function (err){
  //update data yang memiliki id:"63f9cf051e77fd86272f8bd4" dan rubah namanya jadi anggit
  if(!err){ // jika tidak error
    console.log("berhasil update");
  }else{
    console.log(err);
  }
});
```

- Delete (Menghapus Data Tertentu)

Delete one (hapus 1 data dengan nama kasur)

```
//delete one
buatkolom1.deleteOne({nama:"kasur"}, function(err){
  if(err){
    console.log(err);
  }else{
    console.log("berhasil hapus 1 data yang namanya kasur");
  }
});
```

Delete many

```
//delete many
buatkolom1.deleteMany({nama:"kasur"}, function(err){
  if(err){
    console.log(err);
  }else{
    console.log("berhasil hapus semua data yang namanya sama = kasur");
  }
});
```

- Relationship

```
//jshint esversion:6
const mongoose = require("mongoose"); //memanggil package mongoose

//mengubungkan mongoose dengan mongodb
mongoose.connect("mongodb://127.0.0.1:27017/tokped", {useNewUrlParser:true});
// tokped = nama database yang akan dibuat

//membuat skema kolom produk dari database tokped
const produkSkema = new mongoose.Schema(
  {
    nama_produk : String,
    rating: {
      //validasi
      type:Number,
      min:1,
      max:5
    }
  }
);

//menghubungkan skema dan membuat nama kolom
const kolom1 = mongoose.model("produk", produkSkema);
// produk = nama kolom yang akan dibuat

//menambahkan data isi kolom yang dibuat dengan format skema yang dibuat
const tas = new kolom1(
  {
    nama_produk: "tas",
    rating: 5
  }
);
// tas.save();
// kalo udah dijalankan jangan jlnkan lgi nnti double
//membuat skema kolom order dari database tokped
const orderSkema = new mongoose.Schema(
  {
    jumlah : Number,
    pelayan : String,
    detail_produk : produkSkema
  }
);
// ini relationship skema produk dipanggil disini
);
```

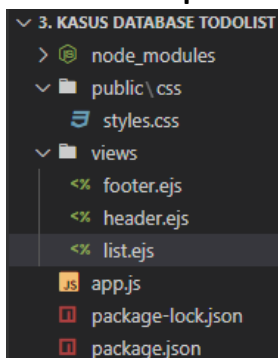
```
//menghubungkan skema dan membuat nama kolom
const kolom2 = mongoose.model("order", orderSkema);
// order = nama kolom yang akan dibuat

//menambahkan data isi kolom yang dibuat dengan format skema yang dibuat
const beli = new kolom2(
  {
    jumlah: 30,
    pelayan: "hartono",
    detail_produk: tas
  }
);
beli.save();
```

Hasil

```
> show collections
orders
produk
> db.orders.find()
{ "_id" : ObjectId("63f9e7ce19eb86f8c559448d"), "jumlah" : 30, "pelayan" : "hartono", "detail_produk" : { "nama_produk" : "tas",
"rating" : 5, "_id" : ObjectId("63f9e7ce19eb86f8c559448c") }, "__v" : 0 }
> 
```

- Kasus Penerapan Database MongoDB, Express Route Parameter, Lodash



File list.ejs

```
<%- include("header") -%>

<div class="box" id="heading">
  <h1> <%= listTitle %> </h1>
</div>
```

```

<div class="box">

  <% newListItems.forEach(function(item){ %>
    <form action="/delete" method="post">
      <!-- untuk app.post("/delete") dijs -->
      <div class="item">
        <input type="checkbox" name="cekbok" value=" <%= item.nama %> "
onchange="this.form.submit()">
        <!-- item.nama agar nilai dari input ini ketika dipanggil nama berupa
item berdasarkan nama -->
        <!-- onchange = untuk melakukan fungsi submit di type chcheckbox -->
        <p> <%= item.nama %> </p>
      </div>
    </form>
  <% }) %>

  <form class="item" action="/" method="post">
    <!-- untuk app.post("/") dijs -->
    <input type="text" name="newItem" placeholder="New Item"
autocomplete="off">
    <button type="submit" name="button"></button>
  </form>
</div>

<%- include("footer") -%>

```

File js

```

//jshint esversion:6

const express = require("express");
const bodyParser = require("body-parser");
const mongoose = require("mongoose");

const app = express();

app.set('view engine', 'ejs');

app.use(bodyParser.urlencoded({extended: true}));
app.use(express.static("public"));
// agar folder public kebaca karena css wajib disipen dipublic

//membuat database baru dan menghubungkan mongoose dengan mongodb
mongoose.connect("mongodb://127.0.0.1:27017/listDB", {useNewUrlParser:true});
// listDB = nama database yagn akan dibuat

```

```
//mengatur isi kolom yang akan dibuat
const itemSkema = new mongoose.Schema(
  {
    nama: String
  }
);

//menghubungkan skema dan membuat nama kolom
const itembaru = mongoose.model("item", itemSkema);
//item = nama kolom yang akan dibuat

//menambahkan data kedalam kolom yang dibuat
const item1 = new itembaru(
  {
    nama: "ini item pertama"
  }
);
const item2 = new itembaru(
  {
    nama: "ini item pertama"
  }
);
const item3 = new itembaru(
  {
    nama: "ini item pertama"
  }
);

//const untuk memanggil data yang dimasukan
const penampungItem = [item1, item2, item3];

app.get("/", function(req, res) {
  itembaru.find({}, function(err, tampil){ // itembaru ambil dari const skema
    if(tampil.length === 0){ //jika tampil tidak terdapat array
      itembaru.insertMany(penampungItem, function(err){
        //insert data dari const penampungitem
        if(err){
          console.log(err);
        }else{
          console.log("berhasil tambah database");
        }
      });
    }
    res.redirect("/");
    //jalankan Kembali app.get "/"
  }else{ //jika yang ditampilkan terdapat array
```

```

    res.render("list", {listTitle:"Sekarang", newListItems:tampil});
    //maka jalankan file "list.ejs"
    //listTitle dan newListItems = ambil dari list.ejs
  }
});
});

app.post("/", function(req, res){ //post ambil dari form action="/"

  const item = req.body.newItem; //newItem nama input didalam form action="/"
  const inputUpdate = new itembaru( //menambahkan data list baru
    {
      nama: item //nama diisi dengan const item
    }
  );
  inputUpdate.save(); //menyimpan didatabase
  res.redirect("/"); //setelah selesai kembali jalankan app.get "/"
});

app.post("/delete", function(req, res){ //post ambil dari form action="/delete"
  const yangDiCeklis = req.body.cekbok; //cekbok nama didalam form action
  ="/delete"
  itembaru.findOneAndRemove(yangDiCeklis, function(err){ // mencari dan menghapus
    if(err){
      console.log(err);
    }else{
      console.log("berhasil dihapus");
      res.redirect("/"); //setelah selesai kembali jalankan app.get "/"
    }
  });
});

app.listen(3000, function() {
  console.log("Server started on port 3000");
});

```

Hasil



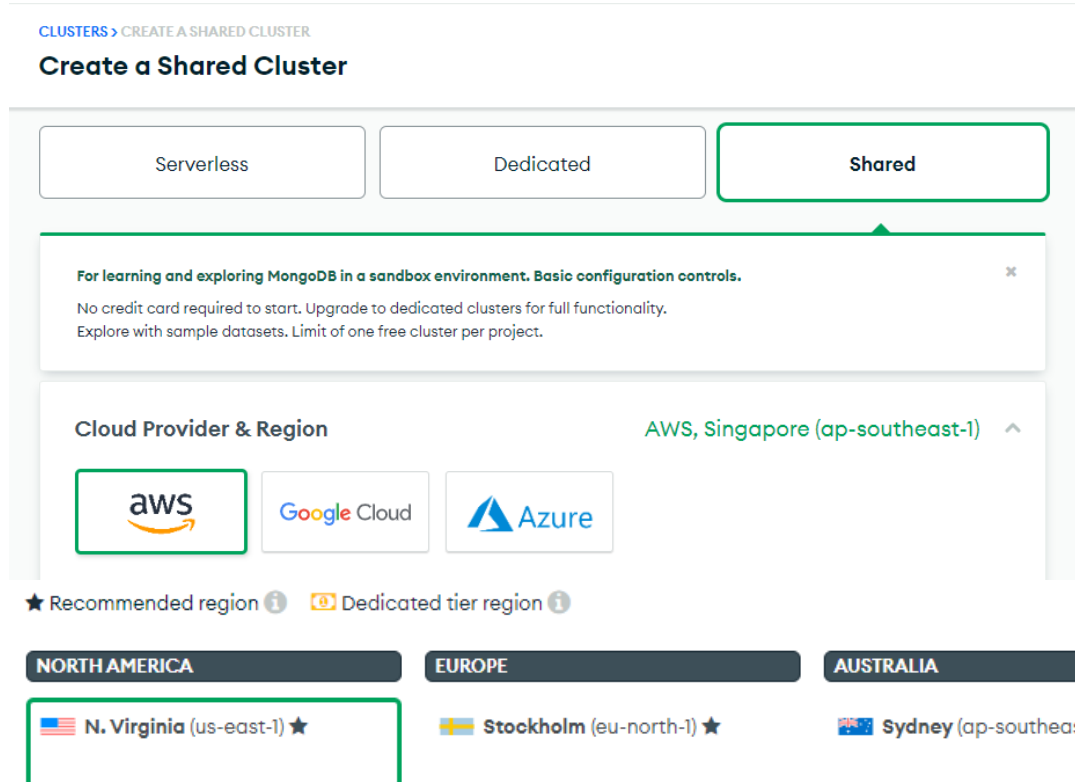
Penggunaan MongoDB Dengan MongoDB Atlas

- Create Cluster MongoDB Atlas

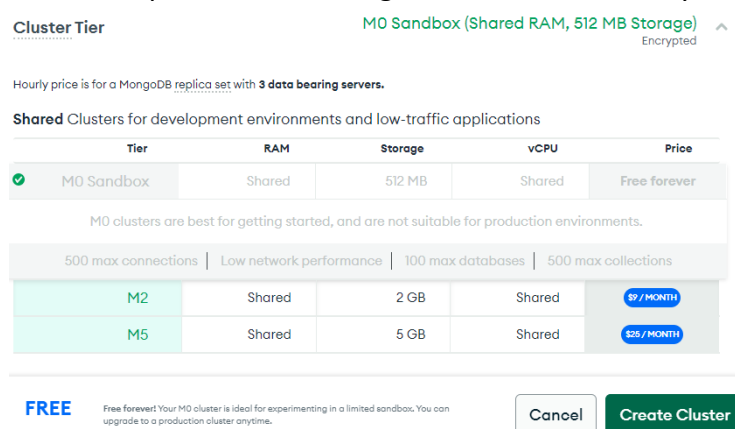
Klik create



Pilih shared, awa, dan negara virginia

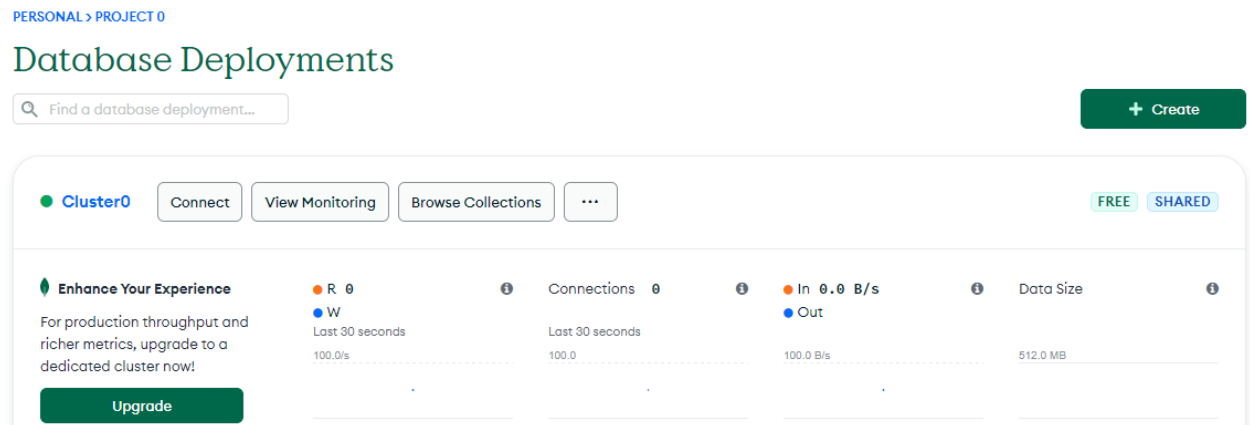


Pastikan pilih cluster m0, agar tidak dikenakan biaya



Terakhir klik create cluster

Hasilnya tunggu sampe seperti ini



- Mengatur akun cluster dan menghubungkan Mongo db Atlas dengan hyper

Klik connect pada cluster0

Masukan username dan password, kemudian klik create database user

2. Create a database user

This first user will have [atlasAdmin](#) permissions for this project.
Keep your credentials handy, you'll need them for the next step.

Username

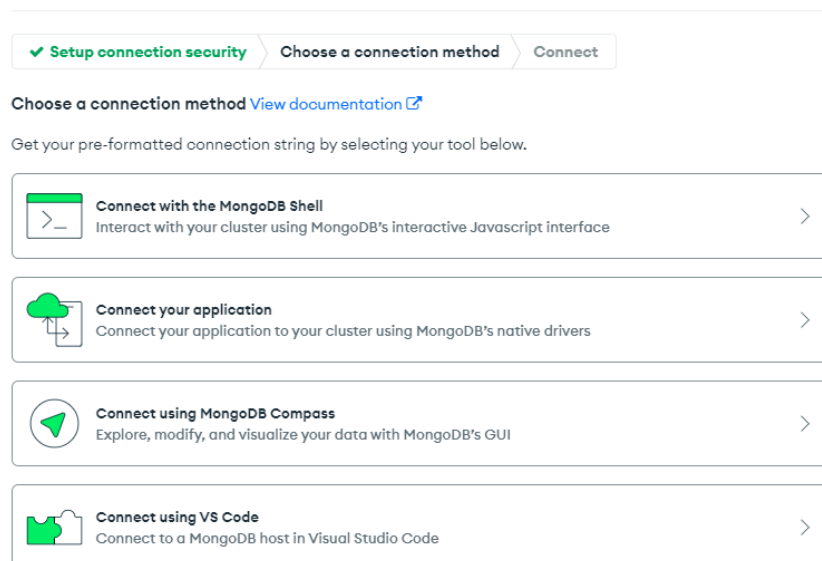
Password
 HIDE [Autogenerate Secure Password](#) [Copy](#)

[Create Database User](#)

Langsung klik choose method connection

Lalu pilih connect with mongoddb shell

Connect to Cluster0



Cek versi mongo dihiper = mongo --version

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~  
$ mongo --version  
MongoDB shell version v5.0.13
```

Pilih version mongo shell sesuai punya kita

Connect to Cluster0

✓ Setup connection security

✓ Choose a connection method

Connect

I do not have the MongoDB Shell installed

I have the MongoDB Shell installed

1

Select your mongo shell version

4.4

(To check your shell version, run mongosh --version OR mongo --version)

2

Run your connection string in your command line

Use this connection string in your application:

mongo "mongodb+srv://cluster0.ybw6bt6.mongodb.net/myFirstDatabase" --username Akbar

Copy connection command lline diatas jalankan pada hyper

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~  
$ mongo "mongodb+srv://cluster0.ybw6bt6.mongodb.net/myFirstDatabase" --username Akbar  
MongoDB shell version v5.0.13  
Enter password:  
connecting to: mongodb://ac-ts2rbbz-shard-00-01.ybw6bt6.mongodb.net:27017,ac-ts2rbbz-shard-00-00.ybw6bt6.mongodb.net:27017,ac-ts2rbbz-shard-00-02.ybw6bt6.mongodb.net:27017/myFirstDatabase?authSource=admin&compressors=disabled&gssapiServiceName=mongodb&replicaSet=atlas-p5oqkz-shard-0&ssl=true  
Implicit session: session { "id" : UUID("8865c776-d939-4efd-a70b-14eb7b67a920") }  
MongoDB server version: 5.0.14  
=====  
Warning: the "mongo" shell has been superseded by "mongosh",  
which delivers improved usability and compatibility. The "mongo" shell has been deprecated and will be removed in  
an upcoming release.  
For installation instructions, see  
https://docs.mongodb.com/mongodb-shell/install/  
=====  
MongoDB Enterprise atlas-p5oqkz-shard-0:PRIMARY> 
```

buka tab baru dan jalankan mongod

```
MINGW64:/c/Users/LENOVO  
LENOVO@DESKTOP-J5NV80G MINGW64 ~  
$ mongod
```

Ngetiknya di tab yagn pertama

```
MongoDB Enterprise atlas-p5oqkz-shard-0:PRIMARY> show dbs  
Proyek_website 0.000GB  
admin           0.000GB  
indexDB         0.000GB  
local           2.661GB  
MongoDB Enterprise atlas-p5oqkz-shard-0:PRIMARY> 
```

- Membuat database dan kolom

Klik browse collections

The screenshot shows the MongoDB Atlas interface. At the top, there are buttons for 'Cluster0', 'Connect', 'View Monitoring', 'Browse Collections', and a menu icon. Below this, there's a section for 'Enhance Your Experience' with a 'Connections' link. The main navigation bar includes 'Overview', 'Real Time', 'Metrics', 'Collections' (which is highlighted), 'Search', 'Profiler', 'Performance Advisor', and 'Onl'. Below the navigation bar, it says 'DATABASES: 2 COLLECTIONS: 3'. On the right, there's a 'VISUAL' button. The main content area shows a '+ Create Database' button and a 'Search Namespaces' input field. To the right, the details for the 'Proyek_website.Pekerja' collection are shown, including 'STORAGE SIZE: 20KB', 'LOGICAL DATA SIZE: 45B', 'TOTAL DOCUMENTS: 1', and 'INDEXES TOTAL SIZE: 20KB'.

Isi nama database dan nama kolom lalu create

The screenshot shows the 'Create Database' dialog box. It has a title bar with a close button. The form contains two input fields: 'Database name' with a placeholder 'Enter database name' and 'Collection name' with a placeholder 'Enter collection name'. Below these fields, there are two checkboxes under 'Additional Preferences': 'Capped Collection' and 'Time Series Collection', both of which are currently unchecked. At the bottom of the dialog, there are two buttons: 'Cancel' and 'Create'.

Hasil

- ▶ Proyek_website
- ▶ indexDB
- ▼ tes
 - ▶ pakaian

Ket : tes = nama database, pakaian = nama kolom

Klik insert Document untuk menambahkan data pada kolom

The screenshot shows the MongoDB Atlas interface for the 'tes.pakaian' collection. It displays 'STORAGE SIZE: 4KB', 'LOGICAL DATA SIZE: 0B', 'TOTAL DOCUMENTS: 0', and 'INDEXES TOTAL SIZE: 4KB'. Below this, there are tabs for 'Find', 'Indexes', 'Schema Anti-Patterns', 'Aggregation', and 'Search Indexes'. The 'Find' tab is selected. At the bottom, there's a 'FILTER' input field with the placeholder '{ field: 'value' }', an 'OPTIONS' button, and 'Apply' and 'Reset' buttons. An 'INSERT DOCUMENT' button is located at the top right of the main content area.

Isi kan yang diinginkan lalu klik insert

×

Insert to Collection pakaian

VIEW {} ≡

1	_id: 63fa50184c56365ba0552a3e	ObjectId
2	jenis: "ini pakaian jenis kaos"	String

Cancel

Insert

Hasilnya

QUERY RESULTS: 1-1 OF 1

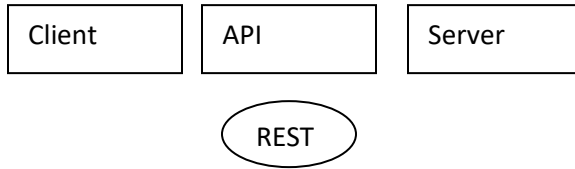
```
{
  "_id": ObjectId('63fa50184c56365ba0552a3e'),
  "jenis": "ini pakaian jenis kaos"
}
```

Cek di hyper

```
MongoDB Enterprise atlas-p5oqkz-shard-0:PRIMARY> show dbs
Proyek_website 0.000GB
admin           0.000GB
indexDB         0.000GB
local           2.661GB
MongoDB Enterprise atlas-p5oqkz-shard-0:PRIMARY> show dbs
Proyek_website 0.000GB
admin           0.000GB
indexDB         0.000GB
local           2.661GB
tes             0.000GB
MongoDB Enterprise atlas-p5oqkz-shard-0:PRIMARY> use tes
switched to db tes
MongoDB Enterprise atlas-p5oqkz-shard-0:PRIMARY> show collections
pakaian
MongoDB Enterprise atlas-p5oqkz-shard-0:PRIMARY> db.pakaian.find()
{ "_id" : ObjectId("63fa50184c56365ba0552a3e"), "jenis" : "ini pakaian jenis kaos" }
MongoDB Enterprise atlas-p5oqkz-shard-0:PRIMARY>
```

REST (Representational State Transfer)

- **Pengenalan**



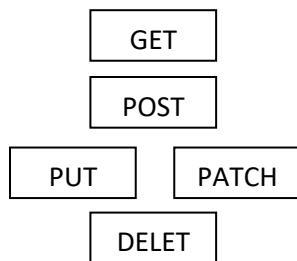
Rest = Standar salah satu gaya arsitektur untuk mendesain API

Contoh gaya arsitektur API lain adalah SOAP, GraphQL, FALCOR dan lainnya

Dua perintah yang penting dalam REST:

1. Use HTTP Request Verb

Yang wajib digunakan dalam rest



Keterangan:

Get (Seperti Read dalam DB) = `app.get( rute, function( req, res ){ } );`

Post (Seperti Create dalam DB) = `app.post( rute, function( req, res ){ } );`

Put & Patch (Seperti Update dalam DB)

Put = Memperbarui isi semua

`app.put( rute, function( req, res ){ } );`

Patch = Memperbarui isi tertentu saja

`app.patch( rute, function( req, res ){ } );`

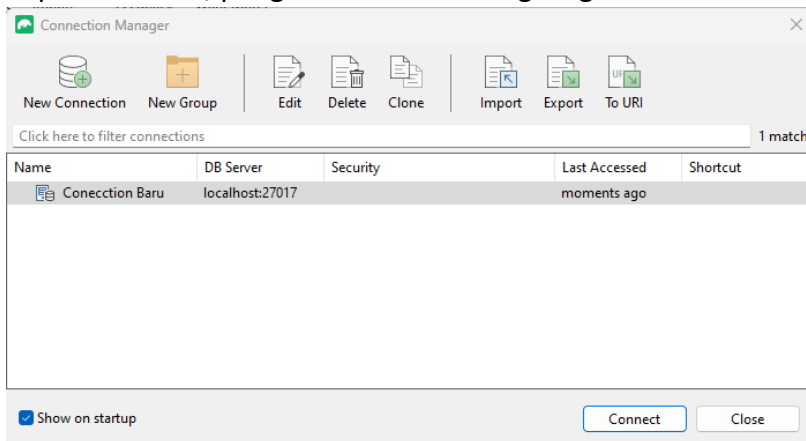
Delete (Seperti Delete dalam DB) = `app.delete( rute, function( req, res ){ } );`

2. Penggunaan Spesifik rute / titik akhir URL

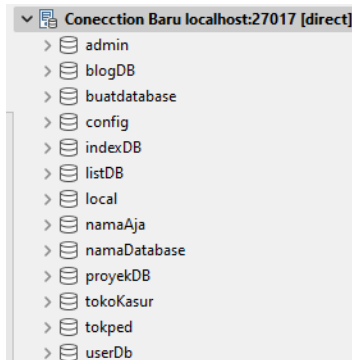
HTTP VERB	/artikel	/artikel/Gajah
Get	Mengambil semua artikel	Mengambil artikel gajah
Post	Create satu artikel baru	-
Put	-	Update artikel gajah
Patch	-	Update artikel gajah
Delete	Delete semua artikel	Delete artikel gajah

- **Penggunaan ROBO 3T (menggunakan aplikasi untuk MongoDB)**

1. Donwload robo 3t = <https://robomongo.org/>
2. Install seperti biasa
3. pada hyper jalankan mongod
4. pada robo 3t, pengaturan default langsung connect

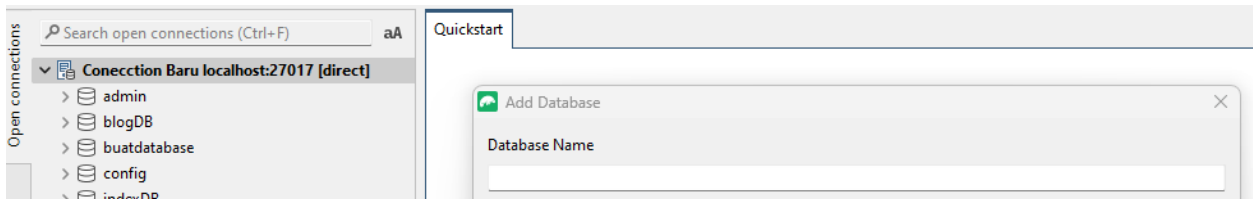


Kalo dah connect keluar database mongodb kita



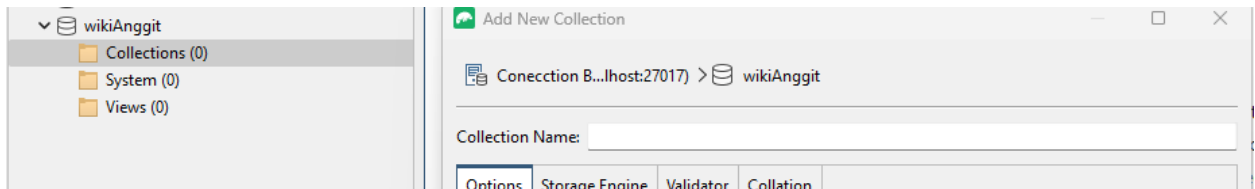
Membuat database mongoDB dengan Robo 3t

Pada connection baru klik klan > pilih add database > isi nama database baru > oke

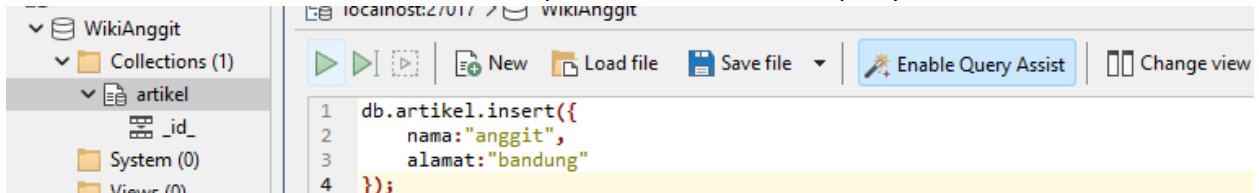


Membuat Kolom (Collections)

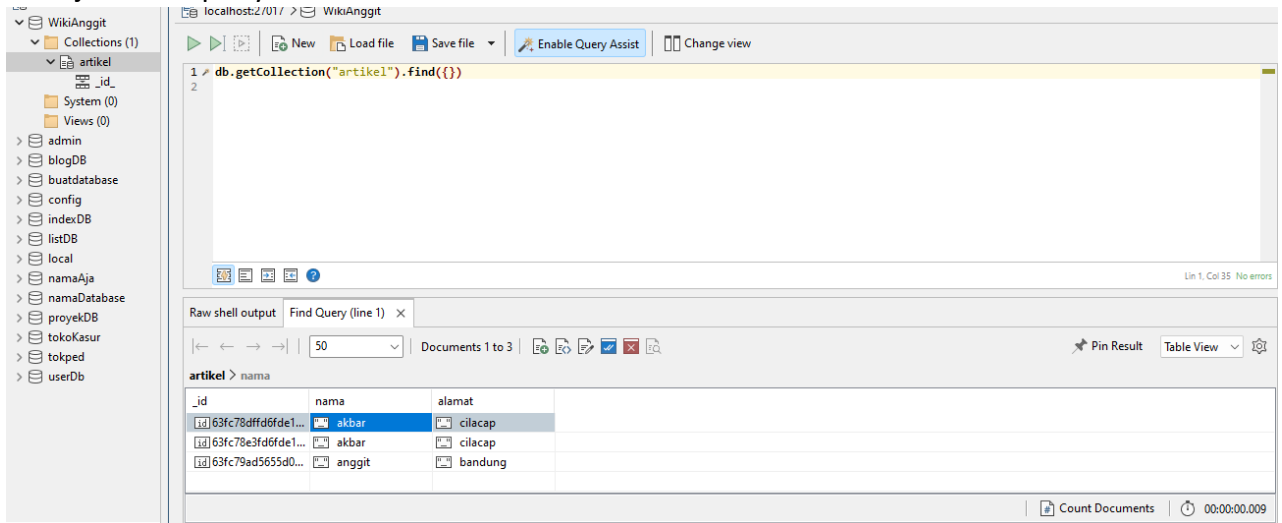
Pada database di collections > klik kanan > add collection > isi namakolom > oke



Insert data kedalam kolom > klik kanan pada kolom > ketikan query save



Hasil jalankan query untuk cek

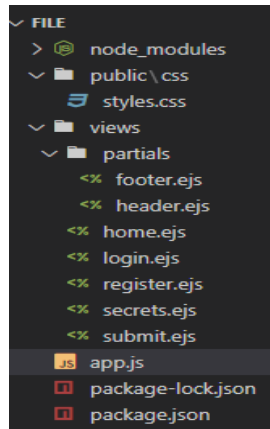


Authentication & Security

- Menerapkan Template node js

Link Download =

https://drive.google.com/uc?export=download&id=1U6tcos_A2rCGuMLdR0RZsDo_KI2_pf_t



File js

```
//jshint esversion:6
const express = require("express");
const bodyParser = require("body-parser");
const ejs = require("ejs");

const app = express();
app.use(express.static("public")); // agar folder public kebaca untuk css
app.set("view engine", "ejs"); // penerapan package ejs
app.use(bodyParser.urlencoded({extended:true})); //penerapan package bodyparser

app.get("/", function(req,res){
    res.render("home"); //jalankan file home.ejss
});

app.get("/login", function(req,res){
    res.render("login"); //jalankan file login.ejs
});

app.get("/register", function(req,res){
    res.render("register"); //jalankan file register.ejs
});

app.get("/secrets", function(req,res){
    res.render("secrets"); //jalankan file secrets.ejs
});
```



```

app.get("/submit", function(req,res){
    res.render("submit"); //jalankan file submit.ejs
});

app.listen(3500, function(){
    console.log("server brjalan diport 3500");
});

```

- **Tingkat 1**

login dengan email dan password

simpan dalam database

File Register.ejs

```

<%- include('partials/header') %>
<div class="container mt-5">
    <h1>Register</h1>
    <div class="row">
        <div class="col-sm-8">
            <div class="card">
                <div class="card-body">

                    <!-- Makes POST request to /register route -->
                    <form action="/register" method="POST">
                        <div class="form-group">
                            <label for="email">Email</label>
                            <input type="email" class="form-control" name="username">
                        </div>
                        <div class="form-group">
                            <label for="password">Password</label>
                            <input type="password" class="form-control" name="password">
                        </div>
                        <button type="submit" class="btn btn-dark">Register</button>
                    </form>

                </div>
            </div>
        </div>
    </div>
</div>

<%- include('partials/header') %>

```

File login.ejs

```
<%- include('partials/header') %>

<div class="container mt-5">
  <h1>Login</h1>
  <div class="row">
    <div class="col-sm-8">
      <div class="card">
        <div class="card-body">

          <!-- Makes POST request to /login route -->
          <form action="/login" method="POST">
            <div class="form-group">
              <label for="email">Email</label>
              <input type="email" class="form-control" name="username">
            </div>
            <div class="form-group">
              <label for="password">Password</label>
              <input type="password" class="form-control" name="password">
            </div>
            <button type="submit" class="btn btn-dark">Login</button>
          </form>

        </div>
      </div>
    </div>
  </div>
</div>

<%- include('partials/footer') %>
```

File app.js

```
//jshint esversion:6
const express = require("express");
const bodyParser = require("body-parser");
const ejs = require("ejs");
const mongoose = require("mongoose");

const app = express();
app.use(express.static("public")); // agar folder public kebaca untuk css
app.set("view engine", "ejs"); // penerapan package ejs
app.use(bodyParser.urlencoded({extended:true})); //penerapan package bodyparser
```

```
//menghubungkan mongoose dengan mongodb
mongoose.connect("mongodb://127.0.0.1:27017/AppSC", {useNewUrlParser:true});
// AppSC = nama database baru yang akan dibuat
const userSkema = { //membuat skema kolom
  email: String,
  password: String
};

//menghubungkan skema dengan kolom
// users = nama kolom yang akan kita buat
const hubung1 = new mongoose.model("users", userSkema);

app.get("/", function(req,res){
  res.render("home"); //jalankan file home.ejs
});

app.get("/register", function(req,res){
  res.render("register"); //jalankan file register.ejs
});

app.post("/register", function(req, res){
  const user1 = new hubung1 ({
    email: req.body.username,
    //username & password ambil dari file register.ejs
    password: req.body.password
    //password & username: nama input di file tersebut
  });
  user1.save(function(err){
    if(err){
      console.log(err);
    }else{
      res.render("secrets"); //jalankan file secrets.ejs
    }
  });
});

app.get("/login", function(req,res){
  res.render("login"); //jalankan file login.ejs
});
```

```

app.post("/login", function(req, res){
  const nama_pengguna = req.body.username;
  //username & password ambil dari file login.ejs
  const password = req.body.password;
  //password & username: nama input di file tersebut

  hubung1.findOne({email: nama_pengguna}, function(err, ditemukan){
    //hubung1 ambil dari const hubungin1 (const yg menghubungkan skema dengan kolom)
    // email = ambil dari skema kolom
    if(err){
      console.log(err);
    }else{
      if(ditemukan){
        if(ditemukan.password === password){
          res.render("secrets"); // jalankan file secrets.ejs
        }
      }
    }
  });
});

app.get("/secrets", function(req,res){
  res.render("secrets"); //jalankan file secrets.ejs
});

app.get("/submit", function(req,res){
  res.render("submit"); //jalankan file submit.ejs
});

app.listen(3500, function(){
  console.log("server brjalan diport 3500");
});

```

Hasil

1. Register = isi email dan password lalu submit maka akan disimpan di database
2. login = masukan email dan password kalo sesuai maka akan login

- **Tingkat 2 (Encripsi database) (Teknik Caesar)**

Penggunaan jumlah huruf pada sandi ceasar

Jika digunakan jumlah geser = 3, maka missal "AB" jadi "DE"

VIEW Text ▼	+	ENCODE DECODE Caesar cipher ▼	+	VIEW Text ▼
aanggit		SHIFT - 3 a→d +		ddqjjlw

Implementasi

1. Install package encryption = npm mongoose-encryption
2. Panggi package = const encrypt = require(mongoose-encryption);
3. pada skema rubah

```
const userSkema = { //membuat skema kolom
  email: String,
  password: String
};
```

Rubah jadi

```
const userSkema = new mongoose.Schema ({ //membuat skema kolom
  email: String,
  password: String
});
```

4. Buat const dan hubungkan skema dengan plugin

```
const userSkema = new mongoose.Schema ({ //membuat skema kolom
  email: String,
  password: String
});

const secret = "IniRahasiaKitaYa";
userSkema.plugin(encrypt, {secret:secret, encryptedFields: ["password"] });
//encryptedFields = menentukan mana yang akan diencrypsi
```

Full file.js

```
//jshint esversion:6
const express = require("express");
const bodyParser = require("body-parser");
const ejs = require("ejs");
const mongoose = require("mongoose");
const encrypt = require("mongoose-encryption");

const app = express();
app.use(express.static("public")); // agar folder public kebaca untuk css
app.set("view engine", "ejs"); // penerapan package ejs
app.use(bodyParser.urlencoded({extended:true})); //penerapan package bodyparser
```

```

//menghubungkan mongoose dengan mongodb
mongoose.connect("mongodb://127.0.0.1:27017/AppSC", {useNewUrlParser:true});
// AppSC = nama database baru yang akan dibuat
const userSkema = new mongoose.Schema ({ //membuat skema kolom
    email: String,
    password: String
});

const secret = "IniRahasiaKitaYa";
userSkema.plugin(encrypt, {secret:secret, encryptedFields: ["password"] });
//encryptedFields = menentukan mana yang akan diencrypsi

//menghubungkan skema dengan kolom
// users = nama kolom yang akan kita buat
const hubung1 = new mongoose.model("users", userSkema);

app.get("/", function(req,res){
    res.render("home"); //jalankan file home.ejss
});

app.get("/register", function(req,res){
    res.render("register"); //jalankan file register.ejs
});
app.post("/register", function(req, res){
    const user1 = new hubung1 ({
        email: req.body.username, //username & password ambil dari file
        password: req.body.password //password & username: nama input di file
    });
    user1.save(function(err){
        if(err){
            console.log(err);
        }else{
            res.render("secrets"); //jalankan file secrets.ejs
        }
    });
});

app.get("/login", function(req,res){
    res.render("login"); //jalankan file login.ejs
});

```

```

app.post("/login", function(req, res){
    const nama_pengguna = req.body.username; //username & password ambil dari
file login.ejs
    const password = req.body.password; //password & username: nama input di
file tersebut

    hubung1.findOne({email: nama_pengguna}, function(err, ditemukan){
        //hubung1 ambil dari const hubungin1 (const yg menghubungkan skema dengan
kolom)
        // email = ambil dari skema kolom
        if(err){
            console.log(err);
        }else{
            if(ditemukan){
                if(ditemukan.password === password){
                    res.render("secrets"); // jalankan file secrets.ejs
                }
            }
        }
    });
});

app.get("/secrets", function(req,res){
    res.render("secrets"); //jalankan file secrets.ejs
});

app.get("/submit", function(req,res){
    res.render("submit"); //jalankan file submit.ejs
});

app.listen(3500, function(){
    console.log("server brjalan diport 3500");
});

```

Hasil password acak

```

> db.users.find()
{ "_id" : ObjectId("63fcaa5b65337f6f313b0b9a"), "email" : "123@4.com", "password" : "123", "__v" : 0 }
{ "_id" : ObjectId("63fcad0ecfb906023a7b8e1e"), "email" : "123@4.com", "password" : "123", "__v" : 0 }
{ "_id" : ObjectId("63fcb88aadfbabd8604e825b"), "email" : "a@B.com", "password" : "tes", "_ct" : BinData(0,"YbIjFPXAw9goEC/ha0Ix
NJMyHAeRvj/zKyqrP630w5r"), "_ac" : BinData(0,"YwjIp7T7tk+HLEyZ3V+6bDuiwObEtNvQBnpeACqr/XNOwYJfawQiLCJfY3QiXQ=="), "__v" : 0 }
{ "_id" : ObjectId("63fcb8d714d20b4c19d2ddae"), "email" : "123@1.com", "password" : "tes123", "_ct" : BinData(0,"YZSK1/SN+11vIbPa
9x20opq+kFZGcaVR/gDRTMyjTX0e"), "_ac" : BinData(0,"YVYvpRYdY2QdopT1CJXsPEID6rg3QNCuWNszTvUpzk1bWYJfawQiLCJfY3QiXQ=="), "__v" : 0
}
{ "_id" : ObjectId("63fcb968208cdf3ba09c89a4"), "email" : "abc@a.com", "_ct" : BinData(0,"YXLiw5dHawk20YPXQ5JyYDpVR82nUXswRyX+N0v
dhw/ohrcrJIRwXpnTH2adg0sFsQ=="), "_ac" : BinData(0,"YQEzCeuUxs3KSU5bpmg1vxckhMZxfEZ5riTix/wrqQMyWYJfawQiLCJfY3QiXQ=="), "__v" : 0
}

```

- Dotenv (untuk menyimpan hal rahasia seperti api_key / const secret)

1. npm install dotenv

2. require("dotenv").config()

```
const express = require("express");
const bodyParser = require("body-parser");
const ejs = require("ejs");
const mongoose = require("mongoose");
const encrypt = require("mongoose-encryption");
require("dotenv").config();
```

3. Buat file = .env (format file didalamnya kapital)

4. pindahkan const secret dalam .env, karena kalo ketahuan password bisa didesripsi orang

```
const secret = "tess";
userSchema.plugin(encrypt, {secret:secret, encryptedFields: ["password"] });
//encryptedFields = menentukan mana yang akan diencrypsi
```

Jadi

```
.env
1 SECRET = IniRahasiaKitaYa
```

5. cara panggil didalam file .env (process.env.namaVariabel)

```
const userSchema = new mongoose.Schema ({ //membuat skema kolom
  email: String,
  password: String
});

userSchema.plugin(encrypt, {secret:process.env.SECRET, encryptedFields: ["password"] });
//encryptedFields = menentukan mana yang akan diencrypsi
```

Full file js

```
//jshint esversion:6
const express = require("express");
const bodyParser = require("body-parser");
const ejs = require("ejs");
const mongoose = require("mongoose");
const encrypt = require("mongoose-encryption");
require("dotenv").config();

const app = express();
app.use(express.static("public")); // agar folder public kebaca untuk css
app.set("view engine", "ejs"); // penerapan package ejs
app.use(bodyParser.urlencoded({extended:true})); //penerapan package bodyparser

//menghubungkan mongoose dengan mongodb
mongoose.connect("mongodb://127.0.0.1:27017/AppSC", {useNewUrlParser:true});
// AppSC = nama database baru yang akan dibuat
```



```
const userSkema = new mongoose.Schema ({ //membuat skema kolom
  email: String,
  password: String
});

userSkema.plugin(encrypt, {secret:process.env.SECRET, encryptedFields:
["password"] });
//encryptedFields = menentukan mana yang akan diencrypsi

//menghubungkan skema dengan kolom
// users = nama kolom yang akan kita buat
const hubung1 = new mongoose.model("users", userSkema);

app.get("/", function(req,res){
  res.render("home"); //jalankan file home.ejss
});

app.get("/register", function(req,res){
  res.render("register"); //jalankan file register.ejs
});
app.post("/register", function(req, res){
  const user1 = new hubung1 ({
    email: req.body.username, //username & password ambil dari file
    password: req.body.password //password & username: nama input di file
    register.ejs
    tersebut
  });
  user1.save(function(err){
    if(err){
      console.log(err);
    }else{
      res.render("secrets"); //jalankan file secrets.ejs
    }
  });
});

app.get("/login", function(req,res){
  res.render("login"); //jalankan file login.ejs
});
```

```

app.post("/login", function(req, res){
    const nama_pengguna = req.body.username; //username & password ambil dari
file login.ejs
    const password = req.body.password; //password & username: nama input di
file tersebut

    hubung1.findOne({email: nama_pengguna}, function(err, ditemukan){
        //hubung1 ambil dari const hubungin1 (const yg menghubungkan skema dengan
kolom)
        // email = ambil dari skema kolom
        if(err){
            console.log(err);
        }else{
            if(ditemukan){
                if(ditemukan.password === password){
                    res.render("secrets"); // jalankan file secrets.ejs
                }
            }
        }
    });
});

app.get("/secrets", function(req,res){
    res.render("secrets"); //jalankan file secrets.ejs
});

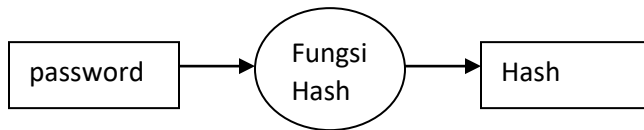
app.get("/submit", function(req,res){
    res.render("submit"); //jalankan file submit.ejs
});

app.listen(3500, function(){
    console.log("server brjalan diport 3500");
});

```

- **Tingkat 3 (HASH)**

- # Pengenalan**



Hash = Ketika menggunakan hash maka password yang udah dihashing tidak bisa di description (tidak seperti encryption) dan tidak bisa dicek passwordnya di database

Catatan : Tidak bisa pake encryption dan hash bersamaan (harus pilih salah satu)

Tidak perlu (file .env, const SECRET, dan Skema.plugin)

- # Implementasi**

1.install package = npm install md5

2. panggil package = const md5 = require("md5");

3. pada app.post("/register") jadi seperti ini (tambahkan md5 depan password)

```
app.post("/register", function(req, res){
  const user1 = new hubung1 ({
    email: req.body.username, //username & password ambil dari file register.ejs
    password: md5(req.body.password) //password & username: nama input di file tersebut
  });
  user1.save(function(err){
    if(err){
      console.log(err);
    }else{
      res.render("secrets"); //jalankan file secrets.ejs
    }
  });
});
```

4. pada app.post("/login") seperti ini (tambahkan md5 depan password)

```
app.post("/login", function(req, res){
  const nama_pengguna = req.body.username; //username & password ambil dari file login.ejs
  const password = md5(req.body.password); //password & username: nama input di file tersebut

  hubung1.findOne({email: nama_pengguna}, function(err, ditemukan){
    //hubung11 ambil dari const hubungin1 (const yg menghubungkan skema dengan kolom)
    // email = ambil dari skema kolom
    if(err){
      console.log(err);
    }else{
      if(ditemukan){
        if(ditemukan.password === password){
          res.render("secrets"); // jalankan file secrets.ejs
        }
      }
    }
  });
});
```

Full file js

```
//jshint esversion:6
const express = require("express");
const bodyParser = require("body-parser");
const ejs = require("ejs");
const mongoose = require("mongoose");
require("dotenv").config(); // untuk menyimpan kunci encrypsi atau api key
const md5 = require("md5");

const app = express();
app.use(express.static("public")); // agar folder public kebaca untuk css
app.set("view engine", "ejs"); // penerapan package ejs
app.use(bodyParser.urlencoded({extended:true})); //penerapan package bodyparser

//menghubungkan mongoose dengan mongodb
mongoose.connect("mongodb://127.0.0.1:27017/AppSC", {useNewUrlParser:true});
// AppSC = nama database baru yang akan dibuat
const userSkema = new mongoose.Schema ({ //membuat skema kolom
  email: String,
  password: String
});

//menghubungkan sekema dengan kolom
// users = nama kolom yang akan kita buat
const hubung1 = new mongoose.model("users", userSkema);

app.get("/", function(req,res){
  res.render("home"); //jalankan file home.ejss
});

app.get("/register", function(req,res){
  res.render("register"); //jalankan file register.ejs
});
```

```

app.post("/register", function(req, res){
  const user1 = new hubung1 ({
    email: req.body.username,
    //username & password ambil dari file register.ejs
    password: md5(req.body.password)
    //password & username: nama input di file tersebut
  });
  user1.save(function(err){
    if(err){
      console.log(err);
    }else{
      res.render("secrets"); //jalankan file secrets.ejs
    }
  });
});

app.get("/login", function(req,res){
  res.render("login"); //jalankan file login.ejs
});
app.post("/login", function(req, res){
  const nama_pengguna = req.body.username; //username & password ambil dari
file login.ejs
  const password = md5(req.body.password); //password & username: nama input
di file tersebut

  hubung1.findOne({email: nama_pengguna}, function(err, ditemukan){
    //hubung1 ambil dari const hubungin1 (const yg menghubungkan skema dengan
kolom)
    // email = ambil dari skema kolom
    if(err){
      console.log(err);
    }else{
      if(ditemukan){
        if(ditemukan.password === password){
          res.render("secrets"); // jalankan file secrets.ejs
        }
      }
    }
  });
});

app.get("/secrets", function(req,res){
  res.render("secrets"); //jalankan file secrets.ejs
});

```

```
app.get("/submit", function(req,res){
  res.render("submit"); //jalankan file submit.ejs
});
app.listen(3500, function(){
  console.log("server brjalan diport 3500");
});
```

- **Tingkat 4 (Hash + salting / bcrypt) password lebih susah ditebak**

Cara Update node

1. Cek versi node = node -version
2. lebih baik install node versi node LTS

Security releases now available

Download for Windows (x64)

18.14.2 LTS

Recommended For Most Users

19.7.0 Current

Latest Features

3. Untuk update install NVM, copykan ini
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.3/install.sh | bash
link = <https://github.com/nvm-sh/nvm>
4. ketikan nvm -version
5. ketikan nvm install v18.14.12 (sesuai dengan node lts terbaru)

Implementasi Hash + bcrypt (salting)

Link npmjs bcrypt = <https://www.npmjs.com/package/bcrypt>

1. Install package = npm install bcrypt
2. panggil package
const bcrypt = require("bcrypt");
const saltRounds = 10;

Catatan : rounds jumlah berapa kali diputar, baca A Note on Rounds pada link

```
//jshint esversion:6
const express = require("express");
const bodyParser = require("body-parser");
const ejs = require("ejs");
const mongoose = require("mongoose");
require("dotenv").config(); // untuk menyimpan kunci encrypsi atau api key
const md5 = require("md5"); // memanggil package hash
const bcrypt = require("bcrypt");
const saltRounds = 10;
```

3. pada app.post("/register")

```
app.post("/register", function(req, res){
  const user1 = new hubung1 ({
    email: req.body.username, //username & password ambil dari file register.ejs
    password: md5(req.body.password) //password & username: nama input di file tersebut
  });
  user1.save(function(err){
    if(err){
      console.log(err);
    }else{
      res.render("secrets"); //jalankan file secrets.ejs
    }
  });
});
```

jadi

```
app.post("/register", function(req, res){

  bcrypt.hash( req.body.password, saltRounds, function(err, hash) {
    // Store hash in your password DB.
    const user1 = new hubung1 ({
      email: req.body.username, //username ambil dari file register.ejs
      password: hash //jadi hash untuk hash+salting
    });
    user1.save(function(err){
      if(err){
        console.log(err);
      }else{
        res.render("secrets"); //jalankan file secrets.ejs
      }
    });
  });
});
```

4. Pada app.post("/login")

```
app.post("/login", function(req, res){
  const nama_pengguna = req.body.username; //username & password ambil dari file login.ejs
  const password = md5(req.body.password); //password & username: nama input di file tersebut

  hubung1.findOne({email: nama_pengguna}, function(err, ditemukan){
    //hubung11 ambil dari const hubungin1 (const yg menghubungkan skema dengan kolom)
    // email = ambil dari skema kolom
    if(err){
      console.log(err);
    }else{
      if(ditemukan){
        if(ditemukan.password === password){
          res.render("secrets"); // jalankan file secrets.ejs
        }
      }
    }
  });
});
```

Jadi

```
app.post("/login", function(req, res){
  const nama_pengguna = req.body.username; //username & password ambil dari file login.ejs
  const password = req.body.password; //password & username: nama input di file tersebut

  hubung1.findOne({email: nama_pengguna}, function(err, ditemukan){
    //hubung11 ambil dari const hubungin1 (const yg menghubungkan skema dengan kolom)
    // email = ambil dari skema kolom
    if(err){
      console.log(err);
    }else{
      if(ditemukan){
        bcrypt.compare( password, ditemukan.password, function(err, result) { //ini bicrypt
          if(result === true) {
            res.render("secrets"); // jalankan file secrets.ejs
          }
        });
      }
    }
  });
});
```

Full file js

```
//jshint esversion:6
const express = require("express");
const bodyParser = require("body-parser");
const ejs = require("ejs");
const mongoose = require("mongoose");
require("dotenv").config(); // untuk menyimpan kunci encrypsi atau api key
const md5 = require("md5"); // memanggil package hash
const bcrypt = require("bcrypt");
const saltRounds = 10;

const app = express();
app.use(express.static("public")); // agar folder public kebaca untuk css
app.set("view engine", "ejs"); // penerapan package ejs
app.use(bodyParser.urlencoded({extended:true})); //penerapan package bodyparser

//menghubungkan mongoose dengan mongodb
mongoose.connect("mongodb://127.0.0.1:27017/AppSC", {useNewUrlParser:true});
// AppSC = nama database baru yang akan dibuat
const userSkema = new mongoose.Schema ({ //membuat skema kolom
  email: String,
  password: String
});

//menghubungkan sekema dengan kolom
// users = nama kolom yang akan kita buat
const hubung1 = new mongoose.model("users", userSkema);
```



```
app.get("/", function(req,res){
    res.render("home"); //jalankan file home.ejss
});

app.get("/register", function(req,res){
    res.render("register"); //jalankan file register.ejs
});
app.post("/register", function(req, res){

    bcrypt.hash( req.body.password, saltRounds, function(err, hash) {
        // Store hash in your password DB.
        const user1 = new hubung1 ({
            email: req.body.username, //username ambil dari file register.ejs
            password: hash //jadi hash untuk hash+salting
        });
        user1.save(function(err){
            if(err){
                console.log(err);
            }else{
                res.render("secrets"); //jalankan file secrets.ejs
            }
        });
    });
});

app.get("/login", function(req,res){
    res.render("login"); //jalankan file login.ejs
});
```

```

app.post("/login", function(req, res){
    const nama_pengguna = req.body.username;
    //username & password ambil dari file login.ejs
    const password = req.body.password;
    //password & username: nama input di file tersebut

    hubung1.findOne({email: nama_pengguna}, function(err, ditemukan){
//hubung1 ambil dari const hubungin1 (const yg menghubungkan skema dengan kolom)
// email = ambil dari skema kolom
        if(err){
            console.log(err);
        }else{
            if(ditemukan){
                bcrypt.compare( password, ditemukan.password, function(err,
result) { //ini bcrypt
                    if(result === true) {
                        res.render("secrets"); // jalankan file secrets.ejs
                    }
                });
            }
        }
    });
});

app.get("/secrets", function(req,res){
    res.render("secrets"); //jalankan file secrets.ejs
});
app.get("/submit", function(req,res){
    res.render("submit"); //jalankan file submit.ejs
});
app.listen(3500, function(){
    console.log("server brjalan diport 3500");
});

```

REACT JS

- **Pengenalan React**

React = Perpustakaan javascript untuk membangun antar muka pengguna

Cat: disarankan update node terbaru (LTS)

- **Implementasi React (Dasar)**

1. Membuat project react, pada hyper = `npx create-react-app namaFolderBebas`

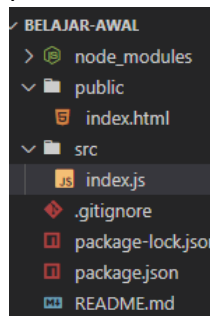
```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/17. React/1. Membuat react awal
$ npx create-react-app belajar-awal
```

2. pada hyper buka cd foldenya, lalu = `npm start`

```
LENOVO@DESKTOP-J5NV80G MINGW64 ~/Pictures/Pelatihan/1.Web_Development_Bootcamp/file 2/17. React/1. Membuat react awal/belajar-awal (master)
$ npm start
```

Tunggu sampe keluar diweb seperti ini

4. Buka folder pada VScode, pada folder public hapus semua file sisakan file "Index.html", pada folder src hapus semua file sisa kan file "Index.js"



5. Template file index.js

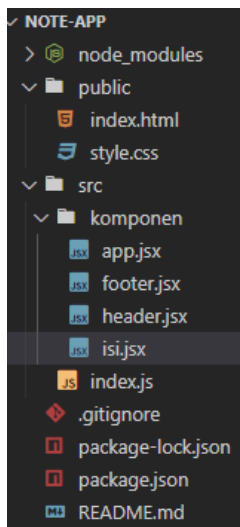
```
import React from 'react';
import ReactDOM from 'react-dom';

ReactDOM.render( <h1>Hello Word</h1>, document.getElementById('root'));
```

6. Template file index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>React App</title>
  </head>
  <body>
    <script src="./src/index.js" type="text/jsx"></script>
    <div id="root"></div>
  </body>
</html>
```

- **Latihan Membuat Note Sederhana**



File index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>React App</title>
    <link
href="https://fonts.googleapis.com/css?family=McLaren|Montserrat&display=swap"
rel="stylesheet"
/>    <!-- untuk menggunakan font google -->
    <link rel="stylesheet" href="style.css"> <!-- memanggil file css -->
  </head>
  <body>
    <script src="./src/index.js" type="text/jsx"></script>
    <!-- memanggil file index.js -->
    <div id="root"></div>
  </body>
</html>
```

File index.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import APP from './komponen/app'; // untuk memanggil yang diexport di file
app.jsx

ReactDOM.render( <APP />,document.getElementById('root') ); //jalankan file
app.jsx
// < APP /> = nama fungsi bukan nama div (cara manggingya emg gtu)
```

File app.jsx (untuk mengumpulkan file jsx lainnya)

```
import React from "react";
import Header from "../header"; //ambil fungsi dari header.jsx
import FOOTER from "../footer"; // ambil fungsi dari footer
import ISI from "../isi"; // ambil fungsi dari isi.jsx

//pake return <div>, tanpa tanda kurung juga bisa
function APP (){
  //bebas return mau pke () atau engga contoh return <h1></h1>
  // <Header />, <FOOTER />, <ISI /> bukan nama div, tp emg cari manggil fungsi gtu
  return (
    <div>
      <Header />
      <ISI />
      <FOOTER />
    </div>
  );
};
export default APP; //APP itu nama fungsi
```

File Header.jsx

```
import React from "react";

function Header (){
  //ini contoh return tanpa tanda kurung
  return <header>
    <h1>Catatan</h1>
  </header>
}
export default Header; // Header nama fungsi
```

File isi.jsx

```
import React from "react";

function ISI (){
  // classname pada div untuk menerepakkan css dengan .note{}
  return(
    <div className="note">
      <h1>Title</h1>
      <p>hello word</p>
    </div>
  );
}
export default ISI;
```

File footer.jsx

```
import React from "react";

function FOOTER(){
  const tahun = new Date().getFullYear();
  return (
    <footer>
      <p>Copyright © {tahun} </p>
    </footer>
  );
}

export default FOOTER;
```

- **Props (contoh memanggil nama)**

```
import React from "react";
import ReactDOM from "react-dom";

function Card(props) { // propt disini isi dari card dibawah ada name, img, tel, email
  //props.name = memanggil isi nama dari card dibawah
  return (
    <div>
      <h2>{props.name}</h2>
      <img src={props.img} alt="avatar_img" />
      <p>{props.tel}</p>
      <p>{props.email}</p>
    </div>
  );
}

ReactDOM.render(
  <div>
    <h1>My Contacts</h1>
    <Card
      name="Beyonce"
      img="https://blackhistorywall.files.wordpress.com/2010/02/picture-device-independent-bitmap-119.jpg"
      tel="+123 456 789"
      email="b@beyonce.com"
    />
  >
```

```

<Card
  name="Jack Bauer"

img="https://pbs.twimg.com/profile_images/625247595825246208/X3XLea04_400x400.jpg"
"

  tel="+7387384587"
  email="jack@nowhere.com"
/>
</div>,
document.getElementById("root")
);

```

Hasil



My Contacts

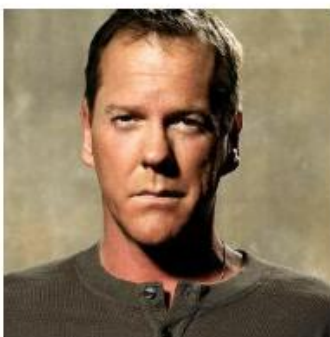
Beyonce



+123 456 789

b@beyonce.com

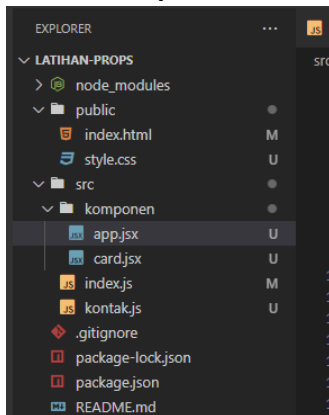
Jack Bauer



+7387384587

jack@nowhere.com

- **Latihan Props**



file html

```
public > index.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <title>React App</title>
5     <link rel="stylesheet" href="style.css" />
6   </head>
7
8   <body>
9     <div id="root"></div>
10    <script src="../src/index.js" type="text/jsx"></script>
11    <!-- memanggil file index.js -->
12  </body>
13 </html>
14
```

File index.js

```
src > index.js
1 import React from "react";
2 import ReactDOM from "react-dom";
3 import APP from "../komponen/app"; //mengambil fungsi APP dari app.jsx
4
5 ReactDOM.render(<APP />, document.getElementById("root")); //menjalankan fungsi APP dari app.jsx
6
```

File contacts.js (berisi const yang menyimpan data array)

```
kontakjs > ...
const contacts = [
  {
    name: "Beyonce",
    imgUrl:
      "https://blackhistorywall.files.wordpress.com/2010/02/picture-device-independent-bitmap-119.jpg",
    phone: "+123 456 789",
    email: "b@beyonce.com"
  },
  {
    name: "Jack Bauer",
    imgUrl:
      "https://pbs.twimg.com/profile_images/625247595825246208/X3XLea04_400x400.jpg",
    phone: "+987 654 321",
    email: "jack@nowhere.com"
  }
];
export default contacts;
//export const contacts untuk digunakan difile app.jsx
```


File App.jsx (yang dijalankan di index.js)

```
import React from "react";
import CARD from "../card"; //memanggil fungsi card dari card.jsx
import contacts from "../kontak"; // memanggil const yg bri array dari kontak.jsx

function APP() { //membuat fungsi APP yang akan dpanggil di index.js
  return (
    <div>
      <h1 className="heading">My Contacts</h1>
      <CARD
        nama= {contacts[0].name}
        gambar = {contacts[0].imgURL}
        tlfn = {contacts[0].phone}
        email= {contacts[0].email}
      />

      <CARD
        nama= {contacts[1].name}
        gambar = {contacts[1].imgURL}
        tlfn = {contacts[1].phone}
        email= {contacts[1].email}
      />
    </div>
  );
  /*
  nama, gambar, tlfn, email = attribute card
  contacs = const yang berisi array
  makanya contacs[0], karena dimulai dari index 0
  contacs[0].name = isi array dari const contacs pada index 0 dan bernama name
  */
}
export default APP; //export fungsi APP agar bisa digunakan di index.js
```

file card (yang dijalankan di app.jsx)

```
import React from "react";

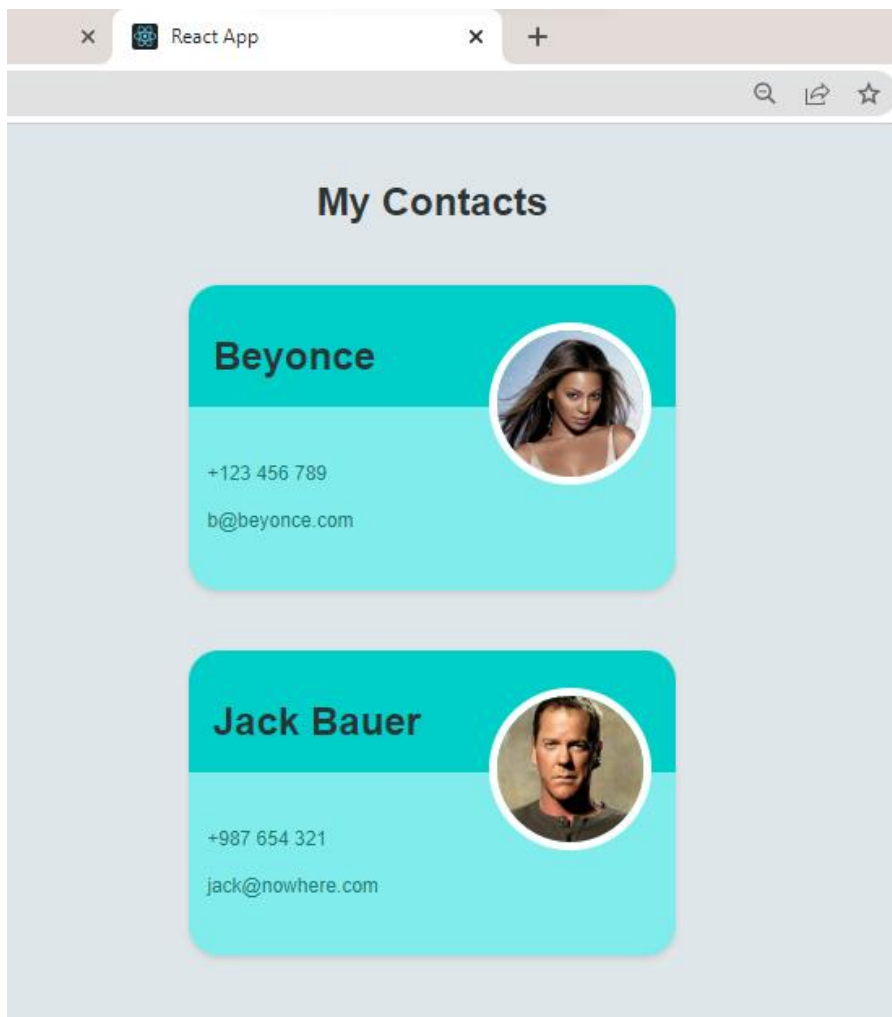
function CARD(props){
  /*
  props = untuk memanggil attribute dari fungsi
  {props.nama} = attribute Card yang bernama nama
  {props.gambar} = attribute Card yang bernama nama
  {props.tlfn} = attribute Card yang bernama nama
  {props.email} = attribute Card yang bernama nama
  className = untuk melakukan desain css
  */
}
```

```

return(
  <div className="card">
    <div className="top">
      <h2 className="name">{props.nama}</h2>
      <img className="circle-img"
        src={props.gambar}
        alt="avatar_img" />
    </div>
    <div className="bottom">
      <p className="info">{props.tlfn}</p>
      <p className="info">{props.email}</p>
    </div>
  </div>
)
}
export default CARD;
//agar fungsi bisa digunakan difile app.jsx

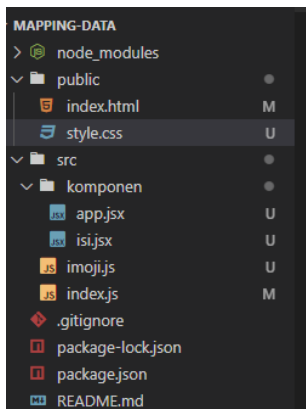
```

Hasil



- Mapping Data Array (Pemanggilan data array di app.jsx Mirip forEach)

Isi folder



File index.html

```
public > index.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <title>React App</title>
5     <link
6       href="https://fonts.googleapis.com/css?family=Montserrat&display=swap"
7       rel="stylesheet"
8     />
9     <link rel="stylesheet" href="style.css" />
10  </head>
11
12  <body>
13    <div id="root"></div>
14    <script src="../src/index.js" type="text/javascript"></script>
15  </body>
16 </html>
```

File index.js

```
src > index.js
1 import React from 'react';
2 import ReactDOM from 'react-dom';
3 import APP from '../komponen/app'; //panggil fungsi APP dari file app.jsx
4
5 ReactDOM.render( <APP />, document.getElementById('root'));
6 // jalankan fungsi APP dari file app.jsx
```

File imoji.js (berisi conts yang berisi data array)

```
src >  imoji.js > ...
1  const emojiopedia = [
2    {
3      id: 1,
4      emoji: "💪",
5      name: "Tense Biceps",
6      meaning:
7        "“You can do that!” or “I feel strong!” Arm with tense biceps. Also used in connection with doing sports
8    },
9    {
10     id: 2,
11     emoji: "🙏",
12     name: "Person With Folded Hands",
13     meaning:
14       "Two hands pressed together. Is currently very introverted, saying a prayer, or hoping for enlightenment
15   },
16   {
17     id: 3,
18     emoji: "😂",
19     name: "Rolling On The Floor, Laughing",
20     meaning:
21       "This is funny! A smiley face, rolling on the floor, laughing. The face is laughing boundlessly. The emo
22   }
23 ];
24 export default emojiopedia;
25 //export const yang berisi data arrat yaitu emojiopedia untuk digunakan di app.jsx
```

File app.jsx

```
import React from "react";
import ISI from "../isi"; //memanggil fungsi isi dari isi.jsx
import emojiopedia from "../imoji";
//memanggil const emojiopedia yang berisi data array dari file imoji.js

function createMapYa(imojiii){
  //imoji disini bebas dikasih nama apa aja
  // id,key,emot,judul,detail = memberikan nama attribut ISI
  //imojiii.name dan lainnya itu ambil dari const emojiopedia di file imoji.js,
  // karena kan dimap biar semua data array tampil
  return(
    <ISI
      id = {imojiii.id}
      key = {imojiii.id}
      emot = {imojiii.emoji}
      judul = {imojiii.name}
      detail = {imojiii.maening}
    />
  );
}
```

```
function APP() { //fungsi yang akan dijalankan di index.js
  return (
    <div>
      <h1>
        <span>emojipedia</span>
      </h1>
      <dl className="dictionary">
        {emojipedia.map(createMapYa)}
      </dl>
    </div>
  );
  //emojipedia.map = pemanggilan data array agar tidak berulang-ulang
  // createMapYa membuat fungsi untuk emojipedia.map
}
export default APP;
//export fungsi APP untuk dijalankan di index.js
```

File isi.jsx

```
src > komponen > jsx isi.jsx > ...
1  import React from "react";
2
3  function ISI(props){
4    //props = untuk memanggil semua attribut yang dimiliki
5    //emot, judul,detail nama attribute fungsi isi yang diambil dari app.jsx
6    //{props.judul} = tampilkan attribut yang memiliki nama judul
7    return(
8      <div className="term">
9        <dt>
10         <span className="emoji" role="img" aria-label="Tense Biceps">
11           {props.emot}
12         </span>
13         <span>{props.judul}</span>
14       </dt>
15       <dd>{props.detail}</dd>
16     </div>
17   )
18 }
19 export default ISI; //export ISI untuk diajalankan di app.jsx
```

Hasil



- Perbedaan Map, Filter, Reduce, Find, FindIndex

Ref = https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/filter

Map = Melakukan perulangan seperti forEach

Versi 1

```
var numbers = [3, 56, 2, 48, 5];
//Map
function kali(x){
  return x * 2;
}
var hasil = numbers.map(kali);
console.log(hasil);
```

Versi 2

```
var numbers = [3, 56, 2, 48, 5];
// jika pake foreach
var hasil = numbers.map(function(x){
  return x * 2;
});
console.log(hasil);
```

Hasilnya sama

```
► (5) [9, 168, 6, 144, 15]
```

Jika pake forEach

Versi 1

```
var numbers = [3, 56, 2, 48, 5];
// jika pake foreach
var hasil = [];
function kali (x){
  hasil.push(x * 5);
}
numbers.forEach(kali);
console.log(hasil);
```

Versi 2

```
var numbers = [3, 56, 2, 48, 5];
// jika pake foreach
var hasil = [];
numbers.forEach(function(x){
  hasil.push(x * 5);
});
console.log(hasil);
```

Hasilnya sama

```
► (5) [15, 280, 10, 240, 25]
```

Filter = Menyaring data isi array

Contoh Tampilkan data array yang nilainya lebih besar dari 10

Pada Filter

```
var numbers = [3, 56, 2, 48, 5];  
//Filter  
var hasil = numbers.filter(function (angka){  
    return angka > 10;  
});  
console.log(hasil)
```

Jika pake forEach jadinya seperti ini

```
var numbers = [3, 56, 2, 48, 5];  
// forEach  
var penampung = []  
numbers.forEach(function (angka){  
    if(angka > 10){  
        penampung.push(angka);  
    }  
});  
console.log(penampung);
```

Hasinya sama

```
► (2) [56, 48]
```

Find = Mirip dengan filter hanya saja jika sudah ketemu akan berhenti

```
var numbers = [3, 56, 2, 48, 5];  
//Find  
var hasil = numbers.find(function(angka){  
    return angka > 10;  
});  
console.log(hasil)
```

Hasil

```
56
```

FindIndex = sama seperti find hanya mencari urutan index seberapa

```
var numbers = [3, 56, 2, 48, 5];  
//FindIndex  
var hasil = numbers.findIndex(function(angka){  
    return angka > 10;  
});  
console.log(hasil)
```

Hasil

```
1
```

Reduce = menjumlahkan semua isi array

Versi reduce

```
var numbers = [3, 56, 2, 48, 5];  
//Reduce  
var hasil = numbers.reduce(function(akulator, isiarray){  
    return akulator + isiarray;  
});  
console.log(hasil);
```

Keterangan:

Akulator = 3,

Isiarray = 56

Akulator ke 2 = hasil 3 + 56 = 59

Isiarray ke 2 = 2

Begitu seterusnya hingga mendapatkan penjumlahan semua isi variabel

Hasil

```
2 114
```

Versi Foreach

```
var numbers = [3, 56, 2, 48, 5];  
//forEach  
var angka = 0;  
numbers.forEach(function(isiArray){  
    angka += isiArray;  
});  
console.log(angka);
```

Hasil

```
114
```


- **Arrow Function (=>) (Mempersingkat Kodingan function())**

Contoh

Versi function normal (tanpa singkat)

```
var numbers = [3, 56, 2, 48, 5];

var hasil = numbers.map(function(angka){
  return angka * 10;
});
console.log(hasil);
```

Versi function (singkat tingkat 1) (rekomendasi ringak membingungkan yang lain)

```
var numbers = [3, 56, 2, 48, 5];

var hasil = numbers.map( (angka) =>{
  return angka * 10;
});
console.log(hasil);
```

Contoh parameter ganda

```
var hasil = numbers.map( (x, y) =>{
  return x * y;
});
```

Versi Function (Singkat versi 2)

```
var numbers = [3, 56, 2, 48, 5];

var hasil = numbers.map( (angka) =>
  angka * 10
);
```

Versi parameter ganda

```
var hasil = numbers.map( (x, y) =>
  x * y
);
```

Versi Function (Tingkat paling singkat) (Kusus untuk parameter tunggal)

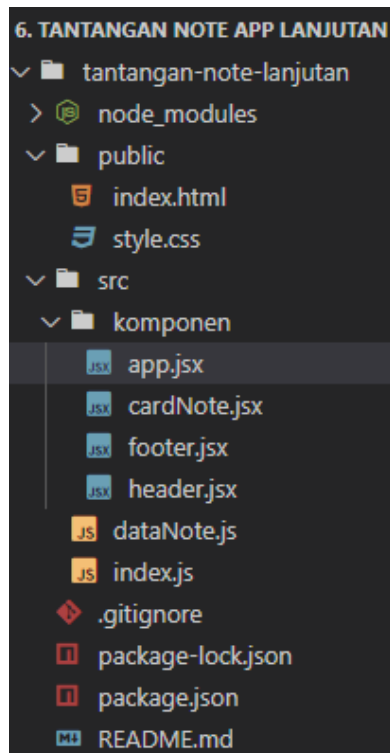
Kurang rekomdendasi (karena takut membingungkan progremers lain)

```
var numbers = [3, 56, 2, 48, 5];

var hasil = numbers.map( angka => angka * 10 );
console.log(hasil);
```

- **Latihan Membuat Note Lanjutan**

Folder



File html

```
tantangan-note-lanjutan > public > index.html > html > body
1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <meta charset="utf-8" />
5      <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
6      <title>React App</title>
7      <link rel="stylesheet" href="style.css">
8    </head>
9    <body>
10     <div id="root"></div>
11     <script src="./src/index.js" type="text/jsx"></script>
12     <!-- menjalankan script index.js -->
13   </body>
14 </html>
```

File index.js

```
tantangan-note-lanjutan > src > index.js
1  import React from 'react';
2  import ReactDOM from 'react-dom';
3  import APP from './komponen/app';
4
5  ReactDOM.render(<APP />, document.getElementById('root'));
```

File cardNote.jsx

```
tantangan-note-lanjutan > src > komponen > JSX cardNote.jsx > ...
1  import React from "react";
2
3  function NOTEJSX(props) { //untuk memanggil attribute fungsi NOTEJSX
4  ✓  return (
5  ✓    <div className="note">
6      <h1>{props.title}</h1>
7      <p>{props.isi}</p>
8    </div>
9  );
10 //memanggil attribute yang bernama title
11 //memanggil attribute yang bernama isi
12 //nama atribut di app.jsx
13 }
14 export default NOTEJSX;
15 //export function NOTE untuk dijalankan di app.jsx
```

Versi 1 app.jsx (bisa seperti ini)

```
import React from "react";
import HEADER from "../header";
import FOOTER from "../footer";
import NOTEJSX from "../cardNote";
import NOTE from "../dataNote";

function Create(isi){
  return(
    <NOTEJSX
      key = {isi.key}
      title= {isi.title}
      isi= {isi.content}
    />
    // key, title, isi = nama attribute fungsi notejsx
    // nama attribute = bebas kita ingin buat nama attribute apa
    // isi.title/content/id = untuk ngambil dari isi array yang ada di
dataNote.js
  );
}
```

```
function APP(){
  // tambahkan div untuk menjalankan leih dari 1
  return(
    <div>
      <HEADER />
      {NOTE.map(Create)}
      <FOOTER />
    </div>
    //melakukan map untuk memanggil semua isi array pada file datanote.js
    //create = memanggil fungsi yang dibuat diatas sendiri
  );
}
export default APP;
//export function APP untuk dijalankan di index.js
```

Versi 2 app.jsx (lebih simpel dan singkat)

```
tantangan-note-lanjutan > src > komponen > app.jsx > APP > NOTE.map() callback
1  import React from "react";
2  import HEADER from "../header";
3  import FOOTER from "../footer";
4  import NOTEJSX from "../cardNote";
5  import NOTE from "../dataNote";
6
7  function APP(){
8    // tambahkan div untuk menjalankan leih dari 1
9    return(
10     <div>
11       <HEADER />
12       {NOTE.map(isi => (
13         <NOTEJSX
14           key = {isi.key}
15           title= {isi.title}
16           isi= {isi.content}
17         />
18         // (isi)=>{} = function(isi){ retrun() };
19         // key, title, isi = nama attribute fungsi notejsx
20         // nama attribute = bebas kita ingin buat nama attribute apa
21         // isi.title/content/id = untuk ngambil dari isi array yang ada di dataNote.js
22       )}}
23       <FOOTER />
24     </div>
25     //Note.map = untuk meamnggil isi array (mirip forEach)
26   );
27 }
28 export default APP;
29 //export function APP untuk dijalankan di index.js
```

Hasil

Keeper

Delegation

Q. How many programmers does it take to change a light bulb? A. None – It's a hardware problem

Loops

How to keep a programmer in the shower forever. Show him the shampoo bottle instructions: Lather. Rinse. Repeat.

Arrays

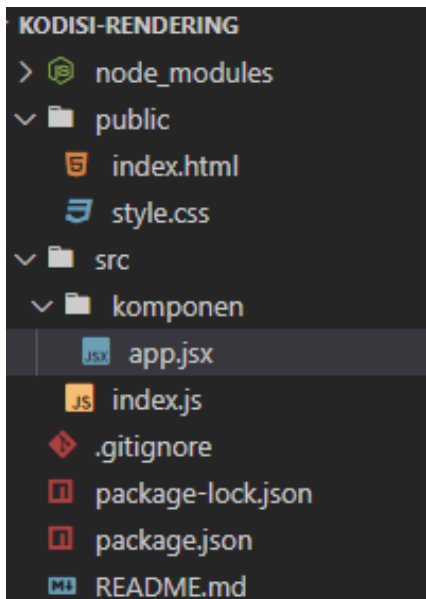
Q. Why did the programmer quit his job? A. Because he didn't get arrays.

Hardware vs. Software

What's the difference between hardware and software? You can hit your hardware with a hammer, but you can only curse at your software.

- **Kondisi Rendering**

Folder



File html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>React App</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <div id="root"></div>
    <script src="../src/index.js" type="text/jsx"></script>
    <!-- panggil index.js untuk dijalankan -->
  </body>
</html>
```

File index.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import APP from '../komponen/app'; //panggil fungsi APP dari file app.jsx

ReactDOM.render(<APP/>, document.getElementById('root'));
// jalankan function APP dari app.jsx
```

File app.jsx

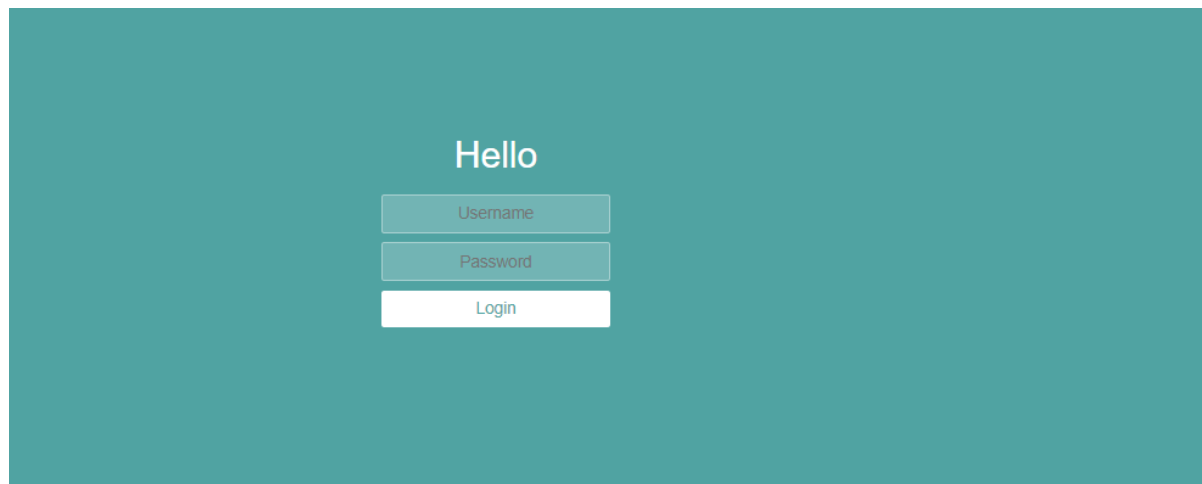
```
import React from "react";

var login = false;
function kondisiRender(){
  if(login === true){ //jika var login true
    return (<h1>Hello World</h1>); //jalankan ini
  }else{ // jika var login bukan true jalankan ini
    return(
      <div className="container">
        <h1>Hello</h1>
        <form className="form">
          <input type="text" placeholder="Username" />
          <input type="password" placeholder="Password" />
          <button type="submit">Login</button>
        </form>
      </div>
    );
  }
}

function APP() {
  // kasih div untuk menjalankan retrun fungsi
  //jalankan fungsi kodsirender
  return (
    <div>
      {kondisiRender()}
    </div>
  );
}

export default APP;
// export untuk dijalankan di index.js
```

Hasil



- **Kondisi Rendering Dengan Ternary Operator (kondisi ? IF true : Else False)**

Pengenalan

Kondisi ? IF true : Else false

Contoh

Var login = false;

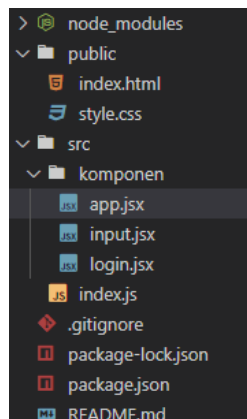
Login ? { tes() } : { tes2() }

Ket:

Jika login = true, jalankan function tes(), jika salah maka jalankan function tes2()

Karena var login false, maka yang akan dijalankan yang function tes2()

Folder



File html

```
public > index.html > html > body
1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <meta charset="utf-8" />
5      <title>React App</title>
6      <link rel="stylesheet" href="style.css">
7    </head>
8    <body>
9      <div id="root"></div>
10     <script src="../src/index.js" type="text/jsx"></script>
11     <!-- panggil file index.js untuk dijalankan -->
12   </body>
13 </html>
```

File index.js

```
JS index.js
import React from 'react';
import ReactDOM from 'react-dom';
import APP from './komponen/app'; //panggil fungsi APP dari app.jsx

ReactDOM.render(<APP />, document.getElementById('root'));
//jalankan fungsi APP dari app.jsx
```

File login.jsx

```
import React from "react";

function LOGIN(){
  return(
    <h1>Hallo world</h1>
  );
}
export default LOGIN;
//export function LOGIN untuk dijalankan di app.jsx
```

File input.jsx

```
import React from "react";

function INPUT(){
  return(
    <div className="container">
      <h1>Hello</h1>
      <form className="form">
        <input type="text" placeholder="Username" />
        <input type="password" placeholder="Password" />
        <button type="submit">Login</button>
      </form>
    </div>
  );
}
export default INPUT;
```

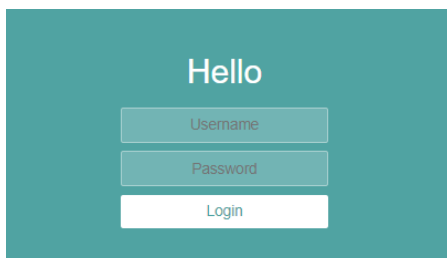
File app.jsx

```
import React from "react";
import LOGIN from "../login"; //panggil function LOGIN dari login.app
import INPUT from "../input"; //panggil function INPUT dari input.app

var login = false;

function APP() {
  return (
    //ternary Operator
    //jika var login true
    //jalankan fungsi login
    //jika var login bukan true jalankan ini
    //jalankan fungsi input
    login === true ? <LOGIN /> : <INPUT />
  );
}
export default APP;
// export untuk dijalankan di index.js
```

Hasil



The screenshot shows the rendered output of the application. It features a teal background with the word "Hello" in white text at the top. Below the text is a login form consisting of three input fields: "Username", "Password", and "Login". The "Username" and "Password" fields are teal with white text, and the "Login" field is white with teal text.

- **Kondisi Rendering Dengan Ternary Operator dan And Operator (&&)**

Pengenalan AND Operator di React

Var login = true;

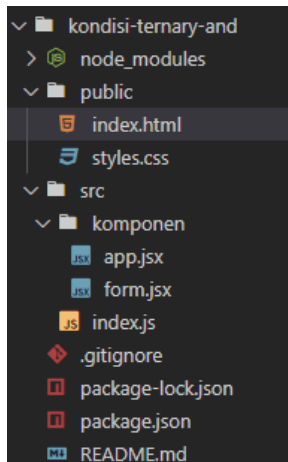
Login && "Hello world"

Ket:

Jika login = true maka jalankan "hello world"

Tapi jika login tidak sama dengan true maka null atau kosong

Folder



File index.js

```
kondisi-ternary-and > src > index.js
1  import React from 'react';
2  import ReactDOM from 'react-dom';
3  import APP from './komponen/app'; //panggil fungsi APP dari app.jsx
4
5  ReactDOM.render(<APP />, document.getElementById('root'));
6  //jalankan fungsi APP
```

File app.jsx

```
kondisi-ternary-and > src > komponen > app.jsx > APP
1  import React from "react";
2  import FORM from "../form";
3
4  var login = false;
5  function APP() {
6    return (
7      <FORM
8        panjul = {login}
9      />
10     //manggil variabel / function wajib pake { }
11     //panjul = nama antribut yang dibuat untuk dipanggil di form.jsx
12   );
13 }
14 export default APP;
```

File form.jsx

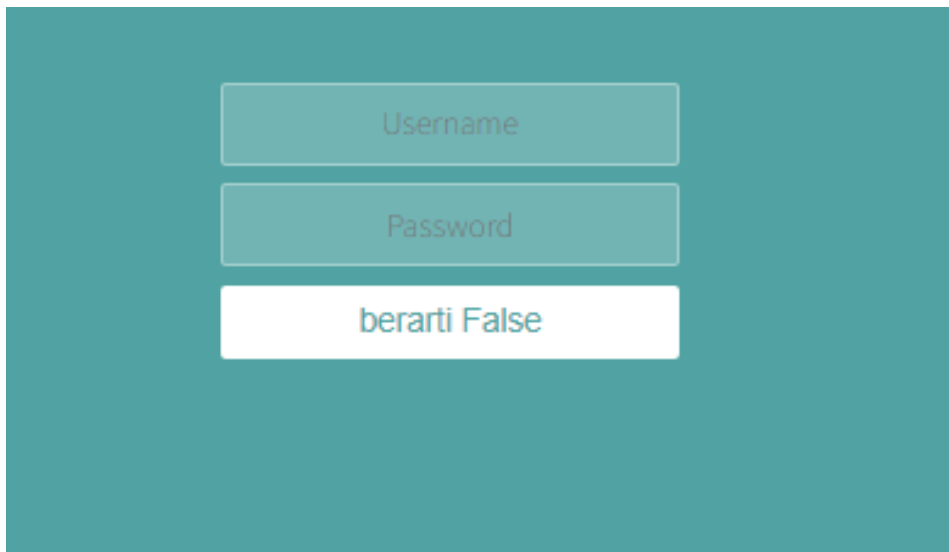
```
import React from "react";

function FORM(props) {
  //props untuk memanggil atribut yang dimiliki function FORM
  return (
    <div className="container">
      <form className="form">
        <input type="text" placeholder="Username" />
        <input type="password" placeholder="Password" />
        { props.panjul && <input type="password" placeholder="Confirm Password" /> }
        <button type="submit">
          { props.panjul ? "hello World" : "berarti False" }
        </button>
      </form>
    </div>
    // props.panjul input (And Operator react js)
    // jika atribut panjul berisi true, maka jalankan input,
    //jika atribut panjul berisi false, maka jalankan null/kosong (input hilang)

    //props.panjul di button (ternary operator)
    // jika attribute panjul di app.jsx = true,
    //isi button dengan "hello world", jika berisi false maka isi button dg "berarti false"
  );
}

export default FORM; // export fungsi FORM untuk dijalankan di app.jsx
```

Hasil

The image shows a web form rendered on a teal background. The form consists of three vertically stacked elements: a text input field with the placeholder 'Username', a password input field with the placeholder 'Password', and a white button with the text 'berarti False'. The 'Confirm Password' field mentioned in the code is not visible, indicating it was correctly rendered as null when the 'panjul' prop was false.

Hasilnya input konfirmasi password hilang

- State (Deklaratif VS Impresif)

Deklaratif

Tergantung pada status variabel (state)

```
import React from "react";

var tes =true;

function APP(){
  return(
    <h1 style={ tes ? {textDecoration:"Line-through"} : null }>Hello World</h1>
  );
  //jika variabel tes berisi true maka jalankan style textdecorasi
  // jika variabel tes tidak berisi true makan style dihilangkan (null)
}
export default APP;
//export function APP untuk dijalankan di index.js
```

Hasil

~~Hello World~~

Impresif

```
import React from "react";

//Impresif
function ganti(){
  document.getElementById("root").style.textDecoration = "line-through";
}
function hilang(){
  document.getElementById("root").style.textDecoration = null;
}
function APP(){
  return(
    <div>
      <p>Hello World</p>
      <button onClick={ganti}>Tombol Tambah garis</button>
      <button onClick={hilang}>Tombol hilang garis</button>
    </div>
  );
}
export default APP;
```

Hasil

Ketika klik button 1 maka style text ada garis, jika klik button 2 maka style text ilang

Hello World

Tombol Tambah garis Tombol hilang garis

- Hooks (useState)

file app.jsx

```
src > komponen > app.jsx > APP > decrease
1  import React, {useState} from "react"; //memanggil useState
2
3  function APP(){
4      const [count, setAngka] = useState(0);
5      //useState(0) = mengisi count diawal dengan nilai 0
6
7      function decrease(){
8          setAngka (count- 1);
9      }
10     function increase(){
11         setAngka(count + 1);
12     }
13     return(
14         <div className="container">
15             <h1>{count}</h1>
16             <button onClick={decrease}>-</button>
17             <button onClick={increase}>+</button>
18         </div>
19     )
20     //{count} = memanggil isi array = dimana berisi 0 diatur dalam useState(0)
21     //div untuk css
22     // {decrease} dan {increase} = memanggil fungsi
23 }
24 export default APP;
```

Hasil

Ketika klik tambah akan jadi 1, ketika klik kurang akan jadi -1



- Latihan hooks useState (toLocaleTimeString)

File app.jsx

```
import React, {useState} from "react";

function APP(){
  const sekarang = new Date().toLocaleTimeString();
  const [waktu, setwaktu] = useState(sekarang);
  // waktu berisi new Date().toLocaleString()
  // diatur dalam useState(sekarang)

  function update(){
    const time = new Date().toLocaleTimeString();
    setwaktu(time);
  }

  return(
    <div className="container">
      <h1>{waktu}</h1>
      <button onClick={update}>Get Time</button>
    </div>
  );
}

export default APP;
```

Hasil

Ketika klik get time maka waktu akan update kewaktu sekarang

10PM33:39

Get Time

- **Destructuring Javascript ES6**

Destructuring adalah memecah komponen yang kompleks menjadi komponen yang lebih kecil.

File paraktek.js

```
.js praktek.js > [0] cars > speedStats
const cars = [
  {
    model: "Honda Civic",
    //The top colour refers to the first item in the array below:
    //i.e. hondaTopColour = "black"
    coloursByPopularity: ["black", "silver"],
    speedStats: {
      topSpeed: 140,
      zeroToSixty: 8.5
    }
  },
  {
    model: "Tesla Model 3",
    coloursByPopularity: ["red", "white"],
    speedStats: {
      topSpeed: 150,
      zeroToSixty: 3.2
    }
  }
];
export default cars;
```

File index.js

```
import React from "react";
import ReactDOM from "react-dom";
import cars from "./praktek";

const [civicYa, TeslaYa] = cars;
// pada const cars di praktek terdapat 2 data array
// civicYa = memberi nama data array ke-0
// teslaYa = memeberi nama data array ke-1

const { speedStats : {topSpeed : kecepatanCivic} } = civicYa;
// memanggil topspeed dari data yang bernama civicya pada cars
// speedStats: {
//   topSpeed: 140,
//   zeroToSixty: 8.5
// }
// topspeed : kecepatanCivic = menggantikan nama data topspeed menjadi
// kecepatanCivic
//
const { speedStats : {topSpeed : kecepatanTesla} } = TeslaYa;
```

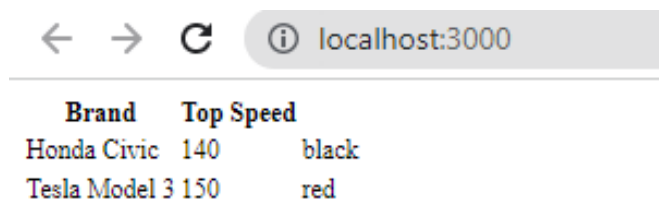
```

const { coloursByPopularity : [warnacivic] } = civicYa ;
//      coloursByPopularity: ["black", "silver"],
const { coloursByPopularity : [warnatesla] } = TeslaYa;

ReactDOM.render(
  <table>
    <tr>
      <th>Brand</th>
      <th>Top Speed</th>
    </tr>
    <tr>
      <td>{civicYa.model}</td>
      <td>{kecepatancivic}</td>
      <td>{warnacivic}</td>
    </tr>
    <tr>
      <td>{TeslaYa.model}</td>
      <td>{kecepatantesla}</td>
      <td>{warnatesla}</td>
    </tr>
  </table>,
  document.getElementById("root"));

```

Hasil



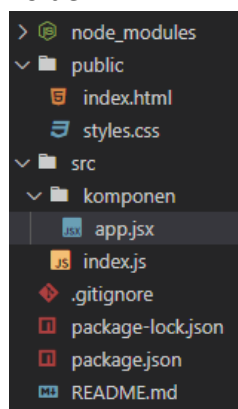
The screenshot shows a web browser window with the address bar displaying 'localhost:3000'. Below the address bar, a table is rendered with the following content:

Brand	Top Speed	
Honda Civic	140	black
Tesla Model 3	150	red

- **Event Handling React (onClick, onMouseOver, onMouseOut)**

Link event html = https://www.w3schools.com/tags/ref_eventattributes.asp

Folder



File app.jsx

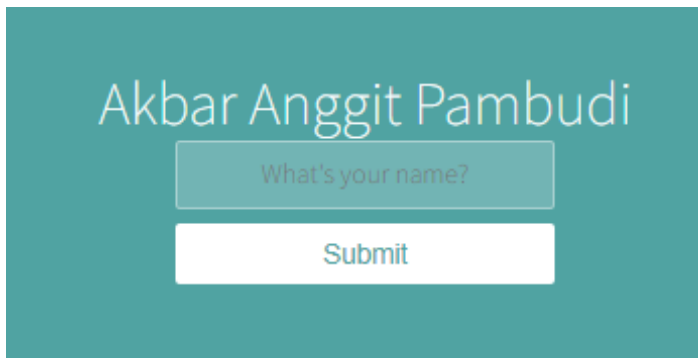
```
import React, {useState} from "react";

function APP() {
  const [title, setTitle] = useState ("Nama Anda Siapa ?");
  //title berisi "siapa anda"
  function AturTitle(){
    setTitle("Akbar Anggit Pambudi");
  }
  //setTitle = mengubah isi title jadi "Akbar Anggit Pambudi" ketika function jalan

  const [warnatombol, setwarna] = useState(false);
  //isi warnatombol = false
  function Aturwarna(){
    setwarna(true);
  }
  //setwarna = ubah isi warnatombol jadi true ketika function dijalankan
  function OutYa(){
    setwarna(false);
  }
  //setwarna = ubah isi warnatombol jadi true ketika function dijalankan
  return (
    <div className="container">
      <h1> {title} </h1>
      <input type="text" placeholder="What's your name?" />
      <button
        onClick={AturTitle} //ketika klik jalankan fungsi Aturtitle
        style={{backgroundColor: warnatombol ? "black" : "white"}}
        onMouseOver={Aturwarna}
        onMouseOut={OutYa}
      >
        Submit
      </button>
    </div>
  );
}

//{title} = memanggil isi array title
// onclick = ketika diklik jalankan function aturtitle
// background warna yaitu warnatombol dimana warna tombol false berarti jalankan white
// onmouseover = ketika cursor berada pada tombol jalankan function,
// dimana fungsi mengubah warnatombol = true berarti jalankan warna black
// onmouseout = ketika cursor tidak berada pada tombol jalankan fungsi
// dimana fungsi mengubah warnatombol = false berarti jalankan warna putih lagi
export default APP; // export function APP untuk dijalankan diindex.js
```

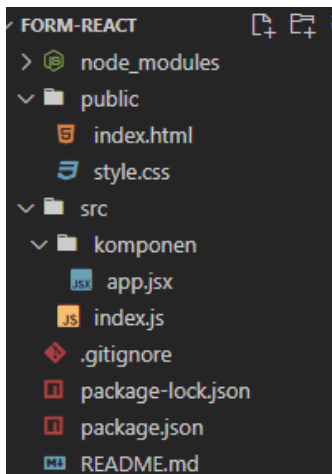

Hasil



A screenshot of a web application with a teal background. At the top, the name "Akbar Anggit Pambudi" is displayed in white. Below it is a light blue input field containing the placeholder text "What's your name?". Underneath the input field is a white button with the text "Submit" in teal.

- **Form React (Effect Button)**

Folder



File html

```
index.html > html > body
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>React App</title>
    <link rel="stylesheet" href="style.css" />
  </head>

  <body>
    <div id="root"></div>
    <script src="../../src/index.js" type="text/javascript"></script>
    <!-- panggil dan jalankan javascript -->
  </body>
</html>
```

File index.js

```
index.js
import React from "react";
import ReactDOM from "react-dom";
import APP from "../komponen/app"; // panggil function APP di app.jsx

ReactDOM.render(<APP />, document.getElementById("root"));
//jalankan function APP
```

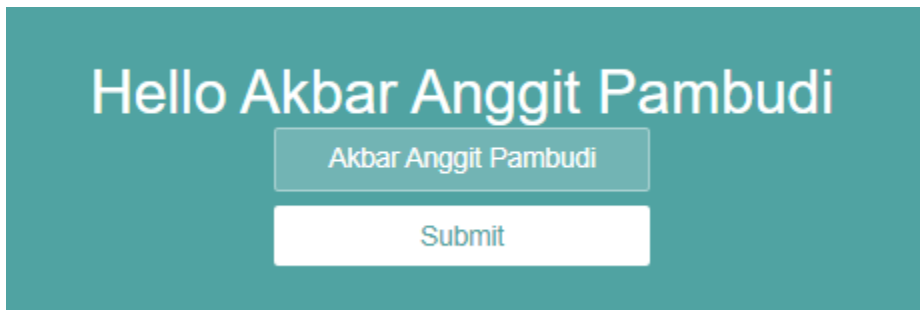
File app.jsx

```
import React, {useState} from "react";

function APP() {
  const [judul, setjudul] = useState("");
  // judul berisi kosong (")
  function ketikaKetik(tes){
    setjudul(tes.target.value);
    //ketika fungsi dijalankan,
    //mengganti isi judul dengan value (isi dari ketikan)
  }
  const [fix, setfix] = useState("");
  //fix = berisi kosong
  function ketikasubmit(tes){
    setfix(judul);
    tes.preventDefault();
    //ketika fungsi dijalankan
    // mengganti isi fix dengan judul
    // preventDefault() = untuk mencegah setelah dijalankan semuanya kembali
    kedefault
  }
  return (
    <div className="container">
      <form onSubmit={ketikasubmit}>
        <h1>Hello {fix} </h1>
        <input onChange={ketikaKetik} value={judul} type="text"
placeholder="hallo" />
        <button>Submit</button>
      </form>
    </div>
  );
}

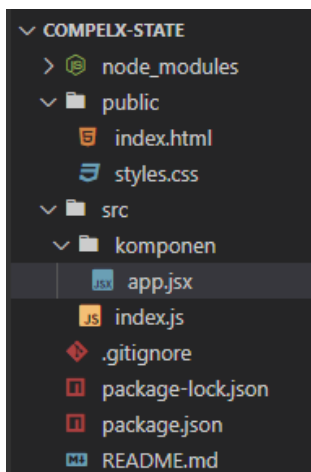
//value = isi yang diketik dalam input
//onchange = ketika merubah isi value, jalankan function ketikaKetik
// onsubmit = ketika tekan submit jalan kan functon ketikaSubmit
export default APP; //export function APP untuk djlankan di index.js
```

Hasil: ketikan sesuatu submit maka hello akan berubah



- **State complex (Hooks)**

Folder



File app.jsx

```
import React, { useState } from "react";

function APP() {
  const [contact, setContact] = useState({
    fName: "",
    lName: "",
  });
  function handleChange(tes) {
    const { name, value } = tes.target;
    setContact(kemana => {
      if (name === "Namadpn") {
        return {
          fName: value,
          lName: kemana.lName,
        };
      } else if (name === "Namablkg") {
```

```

        return {
          fName: kemana.fName,
          lName: value,
        };
      }
    });
  }
  return (
    <div className="container">
      <h1>
        Hello {contact.fName} {contact.lName}
      </h1>
      <p>{contact.email}</p>
      <form>
        <input
          onChange={handleChange}
          value={contact.fName}
          name="Namadpn"
          placeholder="First Name"
        />
        <input
          onChange={handleChange}
          value={contact.lName}
          name="Namablkg"
          placeholder="Last Name"
        />
        <button>Submit</button>
      </form>
    </div>
  );
}
export default APP;

```

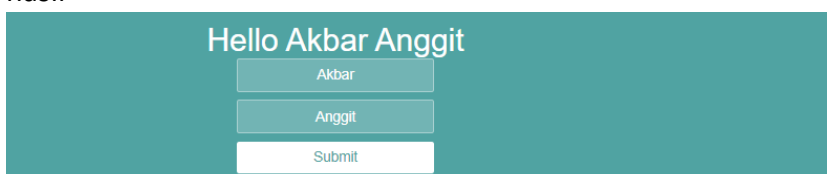
Hasil



- Latihan State Complex Sendiri

```
import React, { useState } from "react";
function APP() {
  const [index1, setIndex] = useState({
    fName: "",
    lName: "",
  });
  function ketikaKetik(panggil){
    const {name, value} = panggil.target;
    setIndex(function(ganti){
      if(name === "namaDpn"){
        return{
          fName: value,
          lName: ganti.lName
        };
      }else if(name === "namaBlkng"){
        return{
          fName: ganti.fName,
          lName: value
        };
      }
    });
  }
  return (
    <div className="container">
      <h1>
        Hello {index1.fName} {index1.lName}
      </h1>
      <form>
        <input onChange={ketikaKetik} name="namaDpn" value={index1.fName}
placeholder="First Name" />
        <input onChange={ketikaKetik} name="namaBlkng" value={index1.lName}
placeholder="Last Name" />
        <button>Submit</button>
      </form>
    </div>
  );
}
export default APP;
```

hasil



The screenshot shows the rendered output of the React application. It features a teal background container. At the top, the text "Hello Akbar Anggit" is displayed in white. Below the text are two input fields, each with a light blue border and a light blue background. The first input field contains the text "Akbar" and the second contains "Anggit". Below these input fields is a white button with the text "Submit" in black.

- **Javascript ES6 Spread (var = [2, 4, 4, ..spread])**

Contoh

Var a = [3, 4, 5];

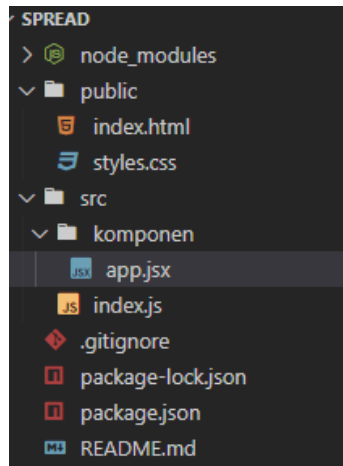
Var b = [3, ..a, 1, 9];

Console.log(a);

Hasilnya = 3, 3, 4, 5, 1,9

Ket: var a masuk kedalam variabel b

Folder



File app.jsx

```
import React, {useState} from "react";

function APP() {
  const [pasInput, setpas] = useState("");
  const [list, setList] = useState([]);

  function ketikaNgisi(tes){
    const iniValue = tes.target.value;
    setpas(iniValue);
  }
  function klik(){
    setList( rubah =>{
      return [...rubah, pasInput];
    });
    setpas("");
  }
  return (
    <div className="container">
      <div className="heading">
        <h1>To-Do List</h1>
      </div>
```

```

<div className="form">
  <input type="text" value={pasInput} onChange={ketikaNgisi} />
  <button onClick={klik}>
    <span>Add</span>
  </button>
</div>
<div>
  <ul>
    {list.map(todolist=>(
      <li>{todolist}</li>
    ))}
  </ul>
</div>
</div>
);
}
export default APP;

```

Hasil

